

Network Computing

(058172)



POLITECNICO
MILANO 1863

Datacenter Monitoring

Some parts of the slides are inspired from Yuilang Li's talk at NSDI 2016 and Marco Chiesa lectures on datacenter networks

Content

- Why we need network monitoring
- We will look at three specific solutions:
 - Everflow: a system running in production at Microsoft that uses legacy switches
 - FlowRadar: a proposal from the research community that requires programmable switches
 - In-band Network Telemetry: a solution from Barefoot/Intel that needs programmable switches

Recap on datacenters (1)

- *Single* administrative domain
- control over network and endpoint(s)
- control over placement of traffic source/sink

Full control:

- can deviate from standards (invent your own!)
- can change addressing/congestion control
- can control what traffic crosses which links

Recap on datacenter - 2

- They are characterized by a massive scale!
 - 1M servers/site [Microsoft/Amazon/Google]
 - 100K switches/routers [Microsoft]
 - Google is serving billion of queries per day
 - Facebook/Instagram count billion of active users

“In such large systems, software and hardware failures are the norm rather than the exception”
[Microsoft Paper from SIGCOMM 2015]

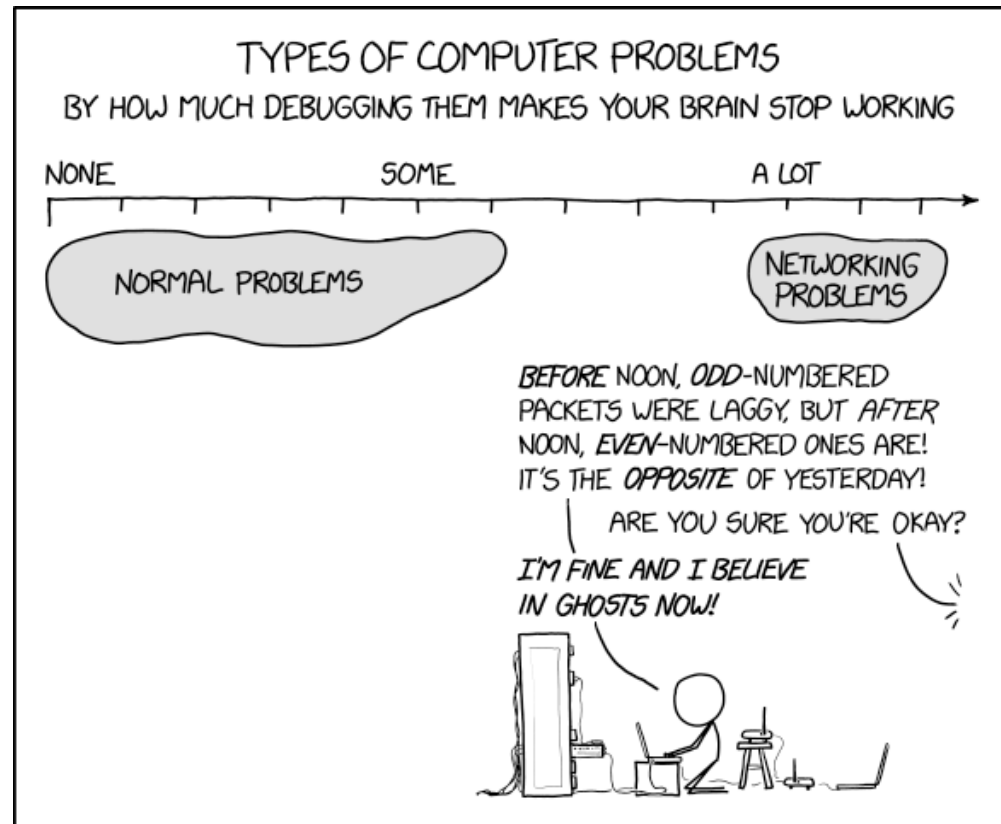
The motto for this lecture (1)

Understanding that something is happening is hard



The motto for this lecture (2)

Understanding what's happening in a big network is very hard!!



<https://xkcd.com/2259/>

A thought example

- You are running a computing application in a datacenter. This application spawns several servers inside the same datacenter. Suddenly the application starts to run slower than expected. What can cause this problem?

A thought example

- You are running a computing application in a datacenter. This application spawns several servers inside the same datacenter. Suddenly the application starts to run slower than expected. What can cause this problem?
 - Software bug
 - Network congestion
 - Broken fiber
 - Broken switch
 - Network mis-configuration
 - Routing protocol bug
 - ...and many more

A thought example

- You are running a computing application in a datacenter. This application spawns several servers inside the same datacenter. Suddenly the application starts to run slower than expected. What can cause this problem?
 - Software bug
 - Network congestion
 - Broken fiber
 - Broken switch
 - Network mis-configuration
 - Routing protocol bug
 - ...and many more

Takeaway: many can be the potential causes!

Is it a network problem?

Pingmesh: A Large-Scale System for Data Center Network Latency Measurement and Analysis*

Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, Zhi-Wei Lin, Varugis Kurien¹
 Microsoft, ¹Midfin Systems
 {chguo, lyuan, dxiang, yidang, rayhuang, dmaltz, zhaoyi, vinwang, bipang, stchen, linzw}@microsoft.com, vkurien@midfinsystems.com

ABSTRACT

Can we get network latency between any two servers at any time in large-scale data center networks? The collected latency data can then be used to address a series of challenges: telling if an application perceived latency issue is caused by the network or not, defining and tracking network service level agreement (SLA), and automatic network troubleshooting.

We have developed the Pingmesh system for large-scale data center network latency measurement and analysis to answer the above question affirmatively. Pingmesh has been running in Microsoft data centers for more than four years, and it collects tens of terabytes of latency data per day. Pingmesh is widely used by not only network software developers and engineers, but also application and service developers and operators.

CCS Concepts

•Networks → Network measurement; Cloud computing; Network monitoring; •Computer systems organization → Cloud computing;

Keywords

Data center networking; Network troubleshooting; Silent packet drops

work interface cards (NICs), switches and routers, cables and fibers, which form large-scale intra and inter data center networks. The scale of the data center networks (DCNs) is growing even larger due to the rapid development of cloud computing. On top of the physical data center infrastructure, various large-scale, distributed services are built, e.g., Search [5], distributed file systems [17] and storage [7], MapReduce [11].

These distributed services are large and evolving software systems with many components and have complex dependencies. All of these services are distributed and many of their components need to interact via the network either within a data center or across different data centers. In such large systems, software and hardware failures are the norm rather than the exception. As a result, the network team faces several challenges.

The first challenge is to determine if an issue is a network issue or not. Due to the distributed systems nature, many failures show as “network” problems, e.g., some components can only be reached intermittently, or the end-to-end latency shows a sudden increase at the 99th percentile, the network throughput degrades from 20MB/s per server to less than 5MB/s. Our experience showed that about 50% of these “network” problems are not caused by the network. However it is not easy to tell if a “network” problem is indeed caused by network failures or not.

Small problems can cause big issues

Understanding and Mitigating Packet Corruption in Data Center Networks

Danyang Zhuo
University of Washington

Monia Ghobadi
Microsoft Research

Ratul Mahajan
Microsoft Research & Intentionet

Klaus-Tycho Förster
Aalborg University

Arvind Krishnamurthy
University of Washington

Thomas Anderson
University of Washington

ABSTRACT

We take a comprehensive look at packet corruption in data center networks, which leads to packet losses and application performance degradation. By studying 350K links across 15 production data centers, we find that the extent of corruption losses is significant and that its characteristics differ markedly from congestion losses. Corruption impacts fewer links than congestion, but imposes a heavier loss rate; and unlike congestion, corruption rate on a link

1 INTRODUCTION

Packet losses in data center networks (DCNs) hurt applications and can lead to millions of dollars in lost revenue [20, 26, 37]. For instance, packet loss rate above 0.1% causes RDMA's throughput to drop by 25% for bulk transfer [36]. For user-facing video traffic, loss rates of 0.01% can cause TCP CUBIC's throughput to drop by 50% [10]. Even sporadic packet losses can cause catastrophic virtual machine reboots [5].

Performance anomalies are common

Flow Event Telemetry on Programmable Data Plane

Yu Zhou^{*,†}, Chen Sun^{*}, Hongqiang Harry Liu^{*}, Rui Miao^{*}, Shi Bai^{*}, Bo Li^{*}, Zhilong Zheng^{*,†},
Lingjun Zhu^{*}, Zhen Shen^{*}, Yongqing Xi^{*}, Pengcheng Zhang^{*}, Dennis Cai^{*}, Ming Zhang^{*}, Mingwei Xu[†]
^{}Alibaba Group [†]INSC and BNRist, Tsinghua University*

ABSTRACT

Network performance anomalies (NPAs), *e.g.* long-tailed latency, bandwidth decline, *etc.*, are increasingly crucial to cloud providers as applications are getting more sensitive to performance. The fundamental difficulty to quickly mitigate NPAs lies in the limitations of state-of-the-art network monitoring solutions — coarse-grained counters, active probing, or packet telemetry either cannot provide enough insights on flows or incur too much overhead. This paper

1 INTRODUCTION

Cloud providers own networks that can comprise thousands of switches and hundreds of thousands of links. In such large and complex networks, bugs and failures in configuration, software, and hardware are inevitable. Therefore, applications often suffer from unceasing network performance anomalies (NPAs), such as bandwidth decline, long-tailed latency, bandwidth/latency jitters, and so forth.

The need for better visibility

CloudCluster: Unearthing the Functional Structure of a Cloud Service

Weiwu Pang
University of Southern California

Sourav Panda
University of California, Riverside

Jehangir Amjad
Google Inc.

Christophe Diot
Google Inc.

Ramesh Govindan
University of Southern California

Abstract

In their quest to provide customers with good tools to manage cloud services, cloud providers are hampered by having very little visibility into cloud service functionality; a provider often only knows where VMs of a service are placed, how the virtual networks are configured, how VMs are provisioned, and how VMs communicate with each other. In this paper, we show that, using the VM-to-VM traffic matrix, we can unearth the *functional structure* of a cloud service and use it to aid cloud service management. Leveraging the observation that cloud services use well-known design patterns for scaling (e.g., replication, communication locality), we show that *clustering* the VM-to-VM traffic matrix yields the functional structure of the cloud service. Our clustering algorithm, CloudCluster, must overcome challenges imposed by each

well the provider provisions the underlying infrastructure.

Cloud service monitoring. Cloud providers struggle to provide customers with insights on the performance and reliability of a cloud service. This is because, while a VM provides a very convenient abstraction for computing and communication, the provider has (by design) very little *visibility* into cloud service logic embedded in the VM. This lack of visibility prevents providers from being able to relate problems observed at the service level to issues in the underlying infrastructure. For a given service, a provider often only knows where the VMs are, how much compute and storage each VM is provisioned with, customer-supplied names for the VMs, and how much traffic each VM exchanges with other VMs in the service. Customers are often loath to reveal more, for business and privacy reasons.¹

Many issues might show up

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²UC. Santa Barbara ³Princeton University

ABSTRACT

Debugging faults in complex networks often requires capturing and analyzing traffic at the packet level. In this task, datacenter networks (DCNs) present unique challenges with their scale, traffic volume, and diversity of faults. To troubleshoot faults in a timely manner, DCN administrators must *a)* identify affected packets inside large volume of traffic; *b)* track them across multiple network components; *c)* analyze traffic traces for fault patterns; and *d)* test or confirm potential causes. To our knowledge, no tool today can achieve both the specificity and scale required for this task.

We present Everflow, a packet-level network telemetry system for large DCNs. Everflow traces specific packets by implementing a powerful packet filter on top of “match and mirror” functionality of commodity switches. It shuffles captured packets to multiple analysis servers using load balancers built on switch ASICs, and it sends “guided probes” to test or confirm potential faults. We present experiments

proactive model of DCN management, where infrastructure observes, analyzes, and corrects faults in near-real time.

Understanding and debugging faults in DCNs is challenging because faults come in all shapes and sizes. For example:

1. Some packets may experience abnormally high delay between servers *A* and *B*, but it may not be clear which of the many links is responsible.
2. Packets destined for a specific set of servers may be dropped, even when the packet-drop counters at switches exhibit no abnormality.
3. TCP connections to a VIP (Virtual IP) may encounter intermittent timeouts, and traceroute probes to debug the issue may be blocked by the load balancers.
4. Load may not be balanced among a group of ECMP (Equal Cost Multi Path) links; the network administrators do not know if this issue is due to flow size differences or a deeper problem (e.g., a poor hash function).

Many issues might show up

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²UC. Santa Barbara ³Princeton University

ABSTRACT

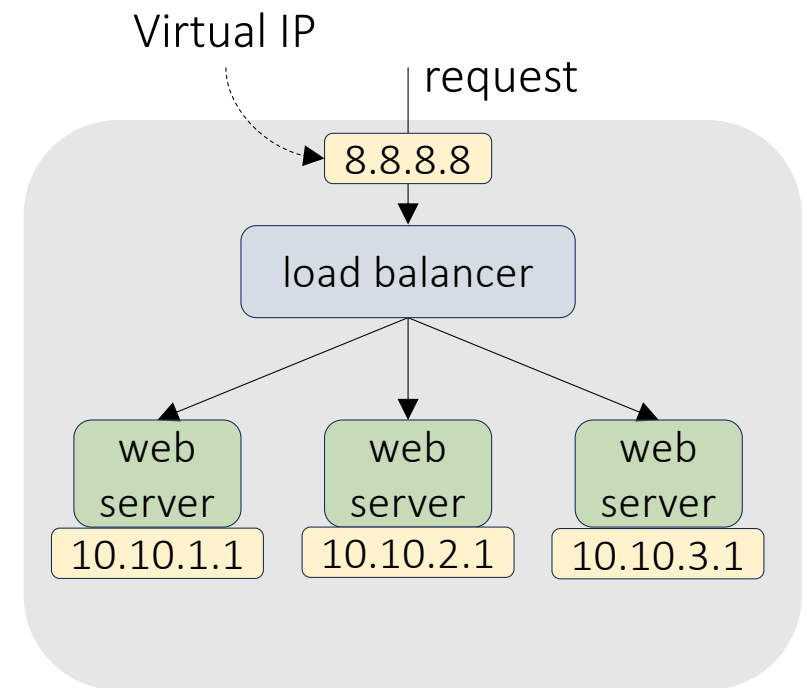
Debugging faults in complex networks often requires capturing and analyzing traffic at the packet level. In this task, datacenter networks (DCNs) present unique challenges with their scale, traffic volume, and diversity of faults. To troubleshoot faults in a timely manner, DCN administrators must *a)* identify affected packets inside large volume of traffic; *b)* track them across multiple network components; *c)* analyze traffic traces for fault patterns; and *d)* test or confirm potential causes. To our knowledge, no tool today can achieve both the specificity and scale required for this task.

We present Everflow, a packet-level network telemetry system for large DCNs. Everflow traces specific packets by implementing a powerful packet filter on top of “match and mirror” functionality of commodity switches. It shuffles captured packets to multiple analysis servers using load balancers built on switch ASICs, and it sends “guided probes” to test or confirm potential faults. We present experiments

proactive model of DCN management, where infrastructure observes, analyzes, and corrects faults in near-real time.

Understanding and debugging faults in DCNs is challenging because faults come in all shapes and sizes. For example:

1. Some packets may experience abnormally high delay between servers *A* and *B*, but it may not be clear which of the many links is responsible.
2. Packets destined for a specific set of servers may be dropped, even when the packet-drop counters at switches exhibit no abnormality.
3. TCP connections to a VIP (Virtual IP) may encounter intermittent timeouts, and traceroute probes to debug the issue may be blocked by the load balancers.
4. Load may not be balanced among a group of ECMP (Equal Cost Multi Path) links; the network administrators do not know if this issue is due to flow size differences or a deeper problem (e.g., a poor hash function).



Many issues might show up

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U.C. Santa Barbara ³Princeton University

ABSTRACT

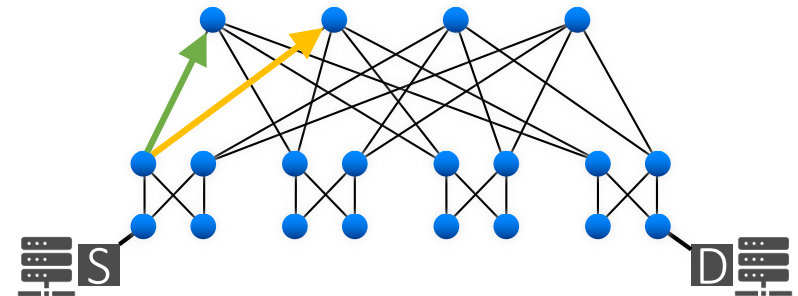
Debugging faults in complex networks often requires capturing and analyzing traffic at the packet level. In this task, datacenter networks (DCNs) present unique challenges with their scale, traffic volume, and diversity of faults. To troubleshoot faults in a timely manner, DCN administrators must *a)* identify affected packets inside large volume of traffic; *b)* track them across multiple network components; *c)* analyze traffic traces for fault patterns; and *d)* test or confirm potential causes. To our knowledge, no tool today can achieve both the specificity and scale required for this task.

We present Everflow, a packet-level network telemetry system for large DCNs. Everflow traces specific packets by implementing a powerful packet filter on top of “match and mirror” functionality of commodity switches. It shuffles captured packets to multiple analysis servers using load balancers built on switch ASICs, and it sends “guided probes” to test or confirm potential faults. We present experiments

proactive model of DCN management, where infrastructure observes, analyzes, and corrects faults in near-real time.

Understanding and debugging faults in DCNs is challenging because faults come in all shapes and sizes. For example:

1. Some packets may experience abnormally high delay between servers *A* and *B*, but it may not be clear which of the many links is responsible.
2. Packets destined for a specific set of servers may be dropped, even when the packet-drop counters at switches exhibit no abnormality.
3. TCP connections to a VIP (Virtual IP) may encounter intermittent timeouts, and traceroute probes to debug the issue may be blocked by the load balancers.
4. Load may not be balanced among a group of ECMP (Equal Cost Multi Path) links; the network administrators do not know if this issue is due to flow size differences or a deeper problem (e.g., a poor hash function).



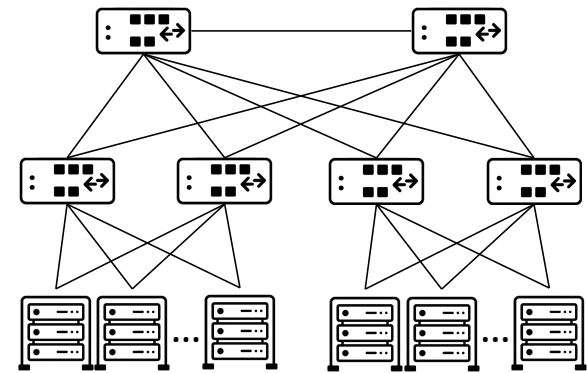
More in the next lecture 😊

I hope I convinced you that...

Understanding what's happening and when in a big network is very hard!!

Network Monitoring to the rescue

- Network monitoring is the use of a system that constantly monitors a computer network for slow or failing components
- Many ways to monitor a network



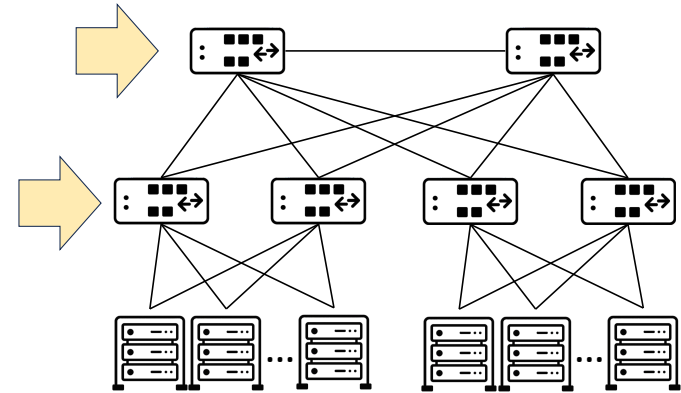
Network Monitoring to the rescue

- From switches

- using their features: NetFlow, Mirroring, SNMP (Simple Network Management Protocol)
- building new features with data plane programmability

- From servers

- Using standard Linux/Windows tools: netstat, tcpdump, traceroute
- Using ad-hoc services



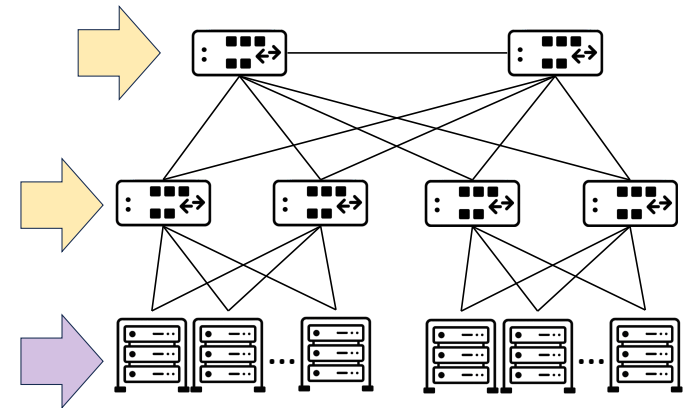
Network Monitoring to the rescue

- From switches

- using their features: NetFlow, Mirroring, SNMP (Simple Network Management Protocol)
- building new features with data plane programmability

- From servers

- Using standard Linux/Windows tools: netstat, tcpdump, traceroute
- Using ad-hoc services



Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

- A solution proposed by Microsoft and running in production in their datacenters
- Built to work with legacy switches on top of the commodity feature “match and mirror”
- Work presented at ACM SIGCOMM 2015 (London)

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

Several faults impact only a subset of traffic and can be identified only by examining the network behavior at the granularity of packets

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

Several faults impact only a subset of traffic and can be identified only by examining the network behavior at the granularity of packets

Example 1: silent packet drops

what: drops not reported by switches

when: software bug or faulty hardware

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

Several faults impact only a subset of traffic and can be identified only by examining the network behavior at the granularity of packets

Example 1: silent packet drops

what: drops not reported by switches

when: software bug or faulty hardware

Example 2: silent blackhole

what: routing blackhole that does not show up in forwarding tables

when: corrupted entries in TCAMs

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

Several faults impact only a subset of traffic and can be identified only by examining the network behavior at the granularity of packets

Example 1: silent packet drops

what: drops not reported by switches

when: software bug or faulty hardware

Example 3: inflated end-to-end latency

what: high latency for a flow

when: packets get delayed

Example 2: silent blackhole

what: routing blackhole that does not show up in forwarding tables

when: corrupted entries in TCAMs

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

Several faults impact only a subset of traffic and can be identified only by examining the network behavior at the granularity of packets

Example 1: silent packet drops

what: drops not reported by switches

when: software bug or faulty hardware

Example 3: inflated end-to-end latency

what: high latency for a flow

when: packets get delayed

Example 2: silent blackhole

what: routing blackhole that does not show up in forwarding tables

when: corrupted entries in TCAMs

Example 4: loops

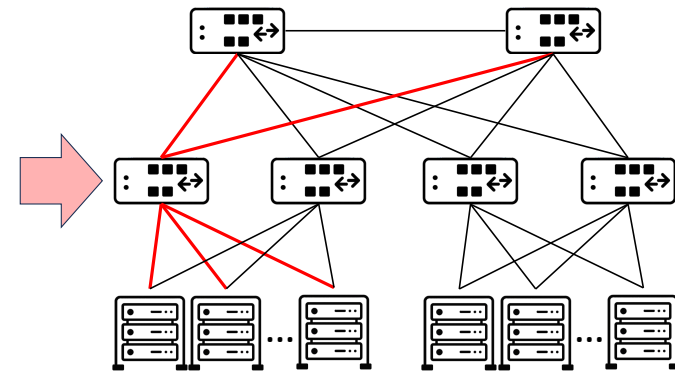
what: routing problems

when: middlebox incorrectly modify a packet and its forwarding behavior

Packet-Level Telemetry in Large Datacenter Networks

All those problems could be easily debugged if we have per packet visibility on all network links

Example: if you have the network traces from all the red links then you can detect *silent packet drops* at the red switch



Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

- Unfortunately tracing all packets in a datacenter network is hard!
 - At high utilization levels, aggregate traffic can easily exceed 100 Tbps. Collecting and analysing this amount of traffic can be prohibitive (Microsoft estimated the need for approximately 3200 servers, back in 2015!)

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²U. C. Santa Barbara ³Princeton University

- Unfortunately tracing all packets in a datacenter network is hard!
 - At high utilization levels, aggregate traffic can easily exceed 100 Tbps. Collecting and analysing this amount of traffic can be prohibitive (Microsoft estimated the need for approximately 3200 servers, back in 2015!)
 - Effective diagnosis requires intelligently tracing packets over the network: you might need to correlate the behavior over multiple hops

Packet-Level Telemetry in Large Datacenter Networks

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

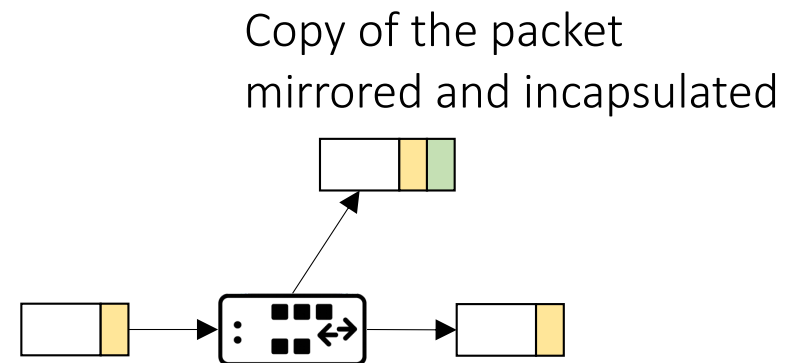
¹Microsoft ²U. C. Santa Barbara ³Princeton University

- Unfortunately tracing all packets in a datacenter network is hard!
 - At high utilization levels, aggregate traffic can easily exceed 100 Tbps. Collecting and analysing this amount of traffic can be prohibitive (Microsoft estimated the need for approximately 3200 servers, back in 2015!)
 - Effective diagnosis requires intelligently tracing packets over the network: you might need to correlate the behavior over multiple hops
 - Passive tracing alone has limited effectiveness: the traces observed in the snapshot may not be enough to identify whether the problem is transient or persistent, and they may not provide enough information to localize the problem

Three main ideas behind Everflow

1. Match and mirror on switch

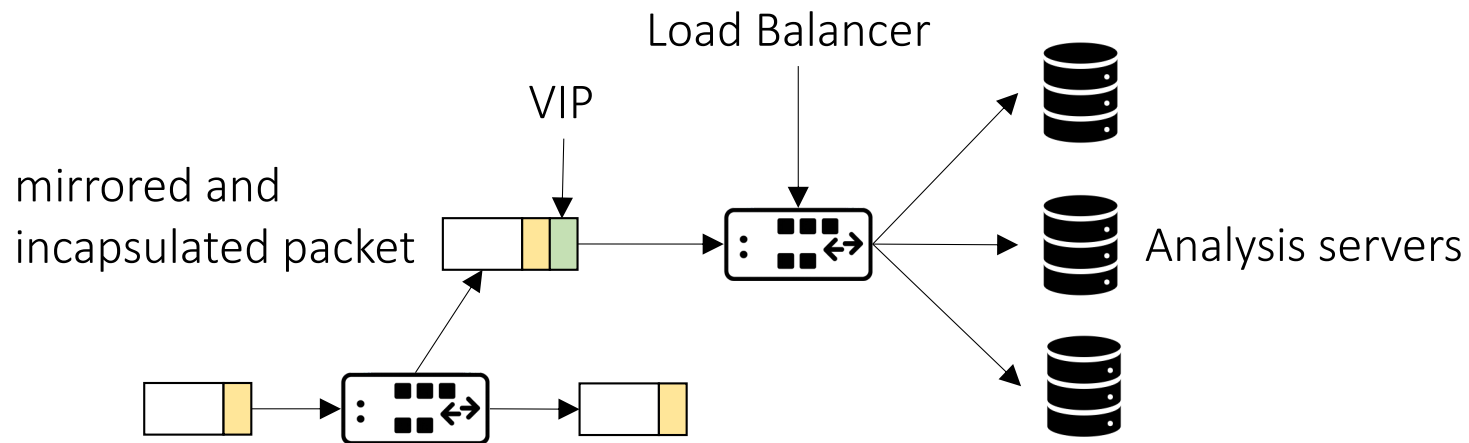
- Commodity switches can match based on pre-defined rules and then execute actions (copy the packet and send it out encapsulated to another interface)
- Three matching rules:
 - TCP SYN, FIN and RST (traffic is dominated by TCP)
 - Special debug bit in the header
 - Protocol traffic (BGP and others)



Three main ideas behind Everflow

2. Switch-based reshuffler

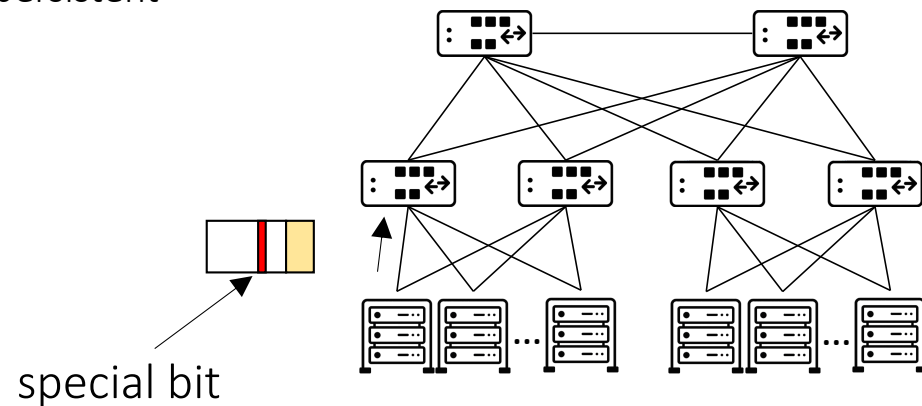
- Tracing traffic must be sent to different collectors as a single machine cannot cope with the amount of data



Three main ideas behind Everflow

3. Guided Probing

- This allows to inject any desired packet into the network and trace its behavior
- It can be used to recover lost information with match and mirror (remember you might need the behavior of the full TCP flow and not just of the first/last packets)
- It allows to check if a problem is transient or persistent
- Which bit to use?

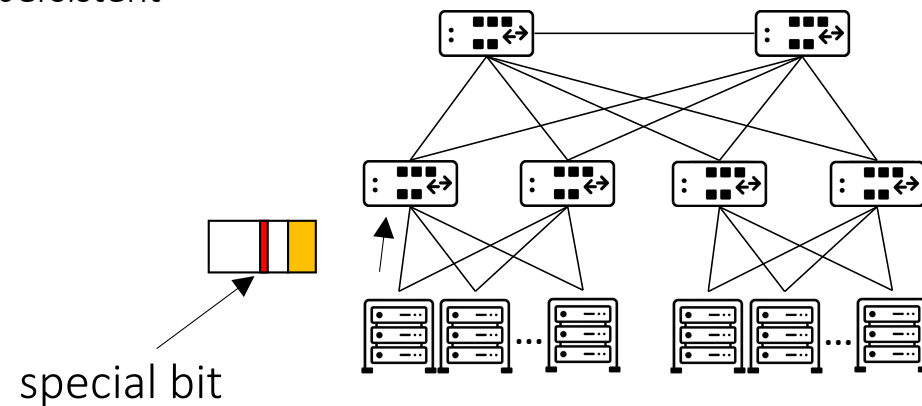


Three main ideas behind Everflow

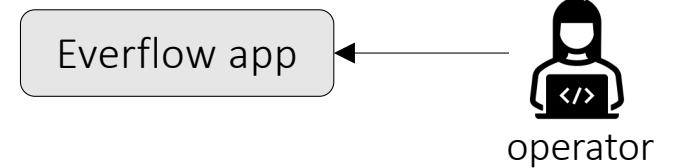
3. Guided Probing

- This allows to inject any desired packet into the network and
- It can be used to recover lost information with match and mirror (remember you might need the behavior of the full TCP flow and not just of the first/last packets)
- It allows to check if a problem is transient or persistent
- Which bit to use?

Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offset
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					



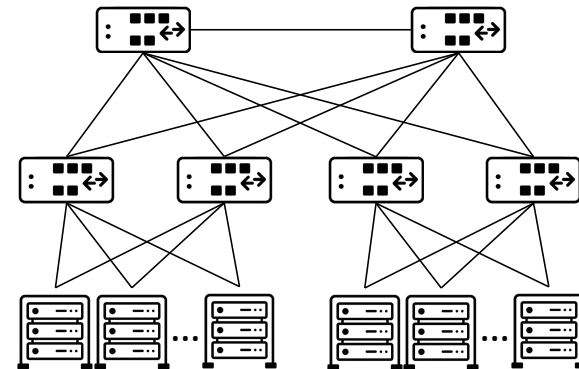
Putting this together



Everflow is extensible and based on the specific “application” the operator wants to run

Example of applications are:

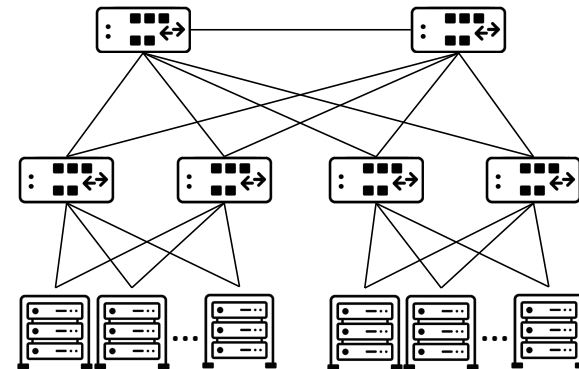
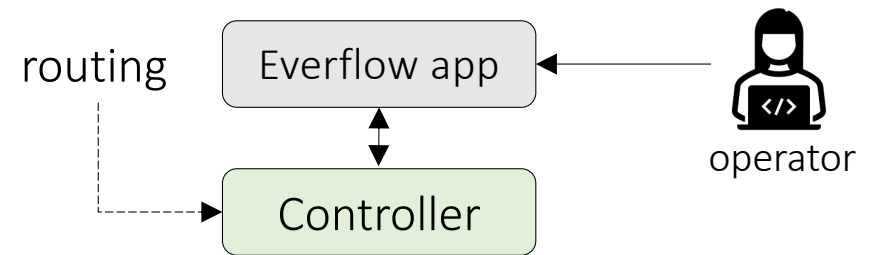
- Latency profiler
- Packet drop debugger
- Loop debugger
- ...



Putting this together

Everflow applications interact with a controller that co-ordinates the overall debugging process

The controller gets as input the operator request and the *supposed* network routing and then:

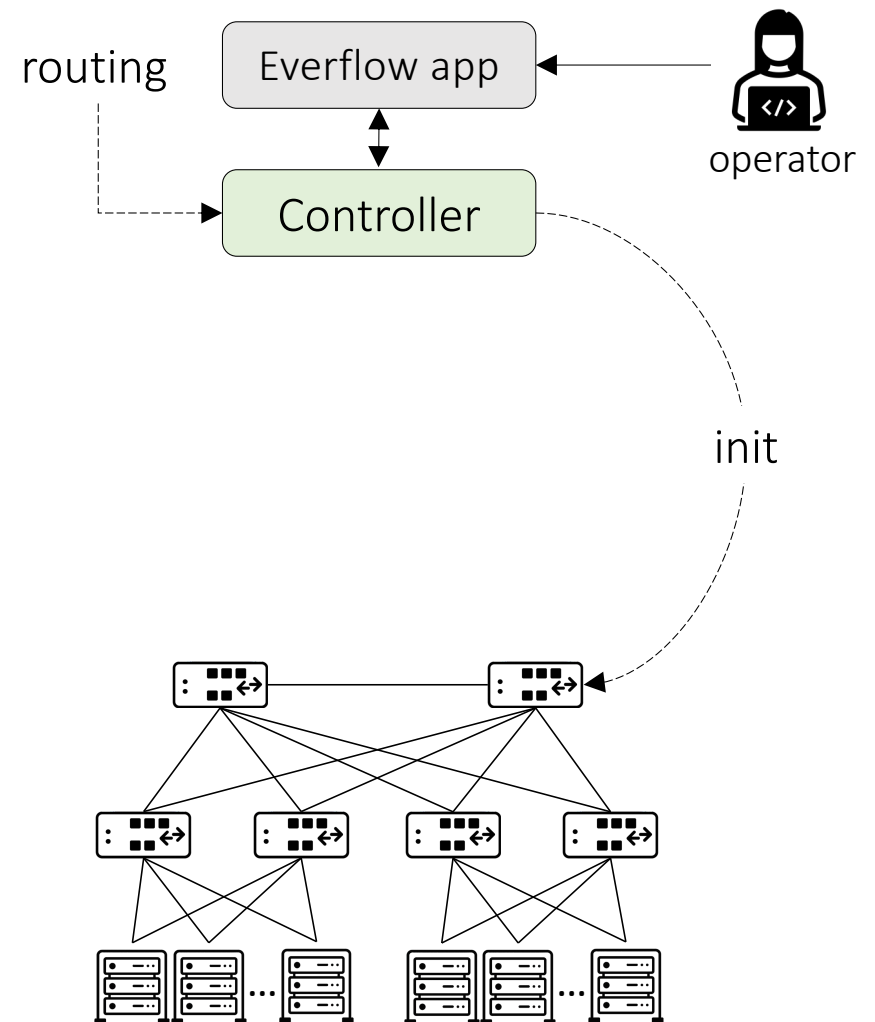


Putting this together

Everflow applications interact with a controller that co-ordinates the overall debugging process

The controller gets as input the operator request and the *supposed* network routing and then:

- Install rules into switches

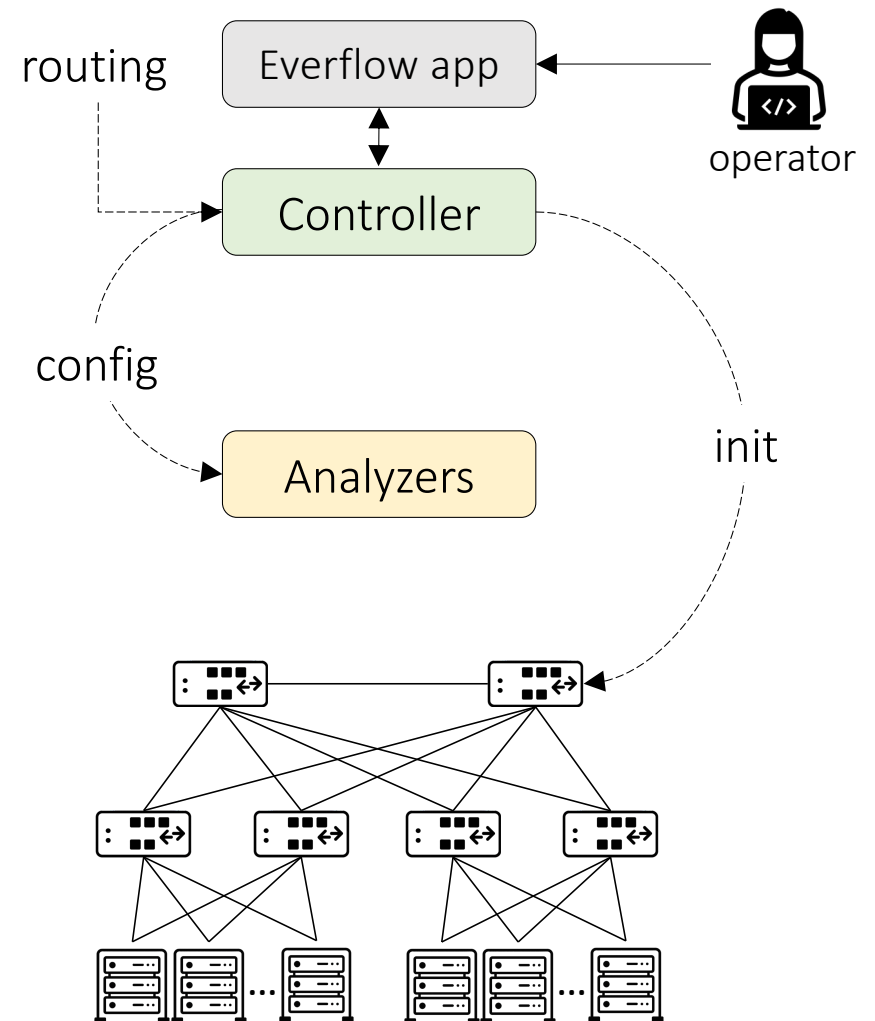


Putting this together

Everflow applications interact with a controller that co-ordinates the overall debugging process

The controller gets as input the operator request and the *supposed* network routing and then:

- Install rules into switches
- Configure the analyzers

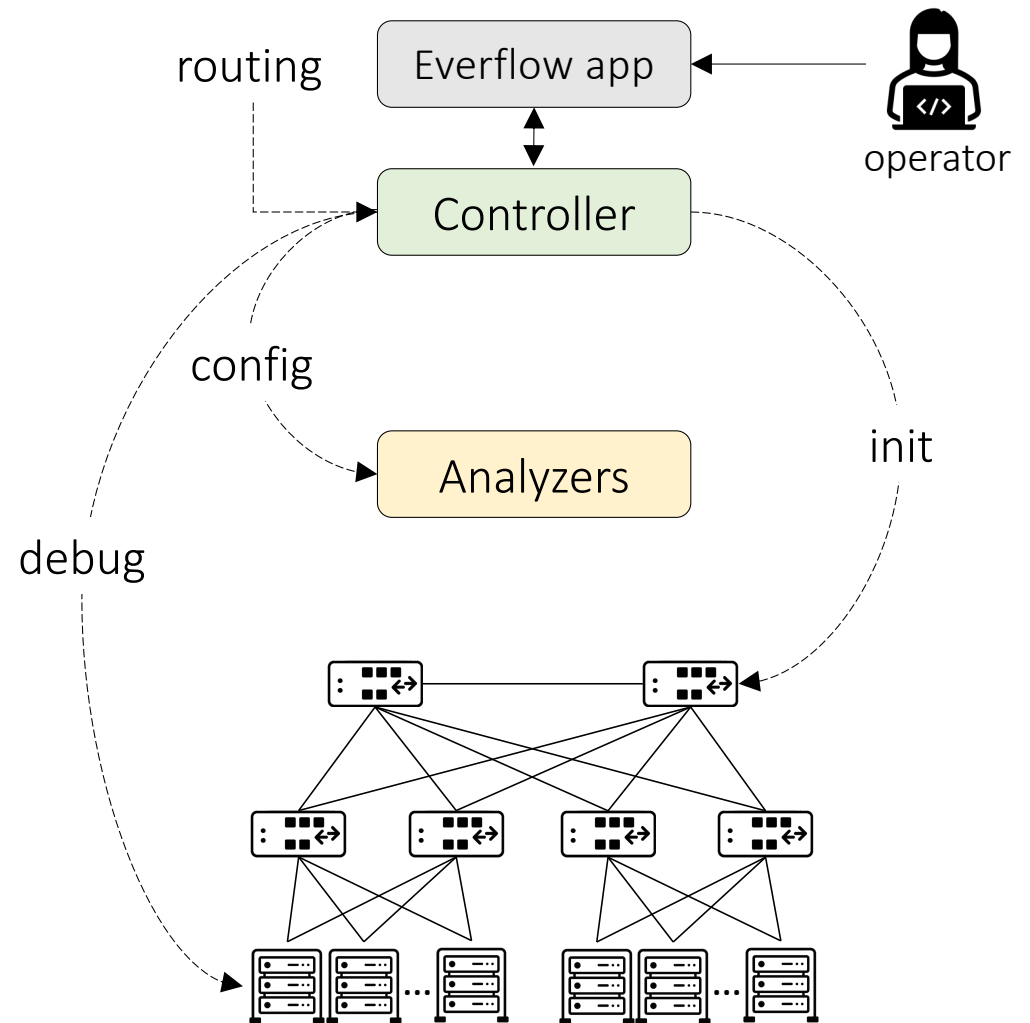


Putting this together

Everflow applications interact with a controller that co-ordinates the overall debugging process

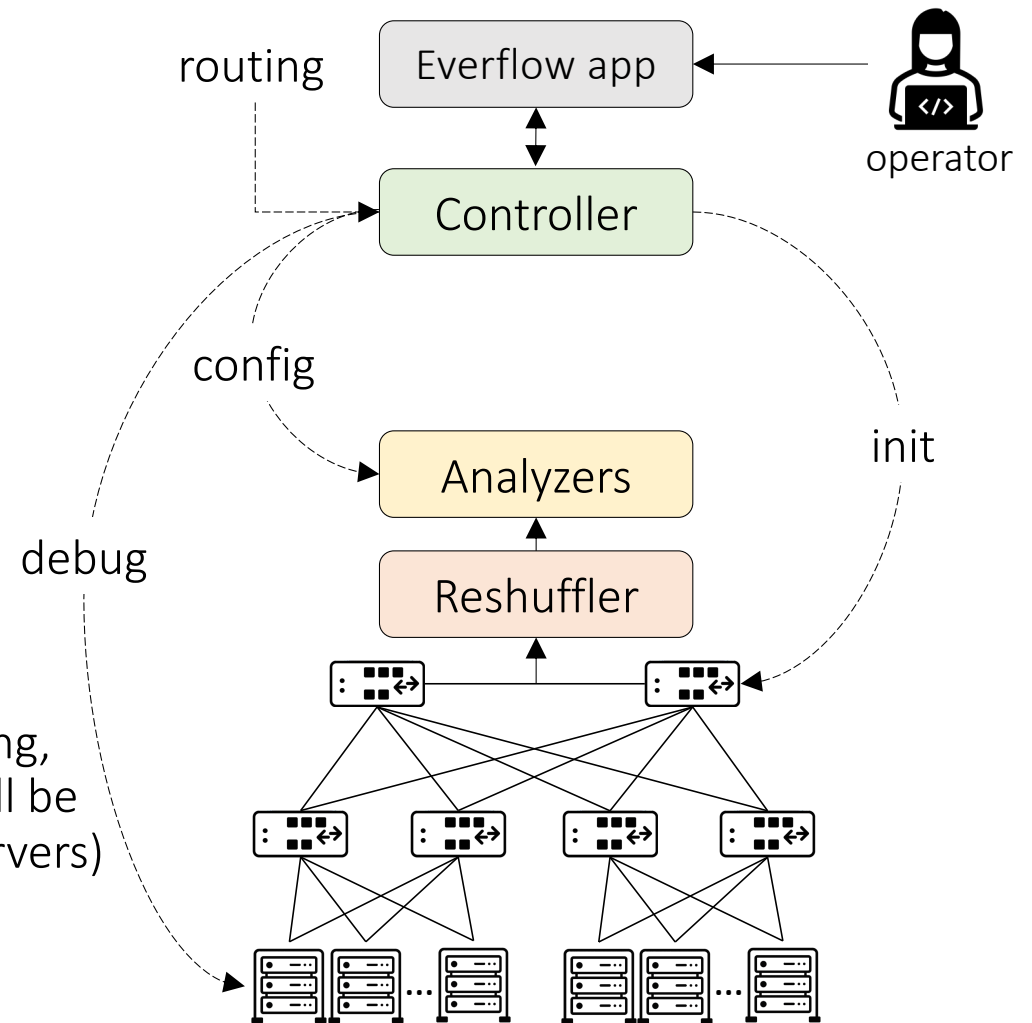
The controller gets as input the operator request and the *supposed* network routing and then:

- Install rules into switches
- Configure the analyzers
- Set appropriate debug bit into probes



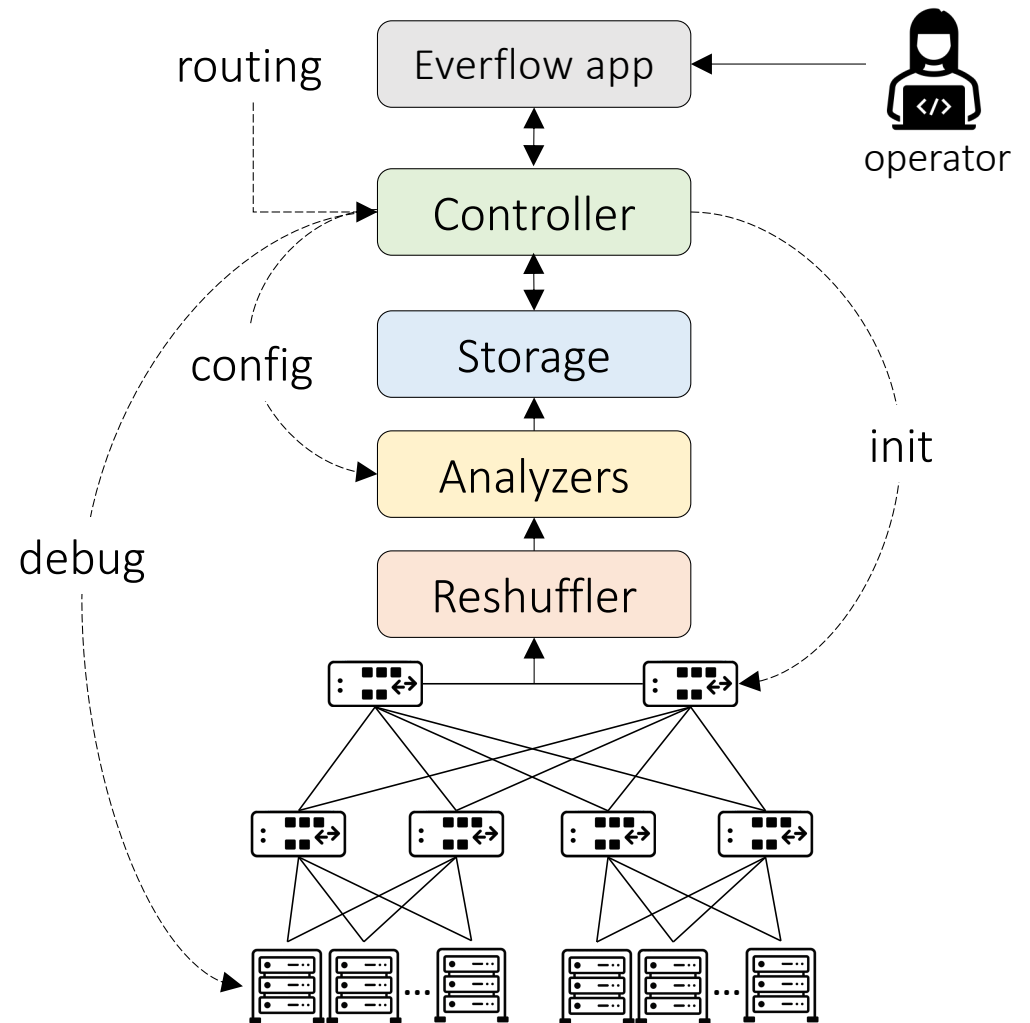
Putting this together

Once the controller has configured everything, mirrored traffic will hit the reshuffler and will be sent to the analyzers (a distributed set of servers)



Putting this together

Despite of the use of match and mirror, the amount of packet traces can still be huge. To further reduce the storage overhead, each analyzer will only write to storage the traces that exhibit abnormal behaviors or have the debug bit set

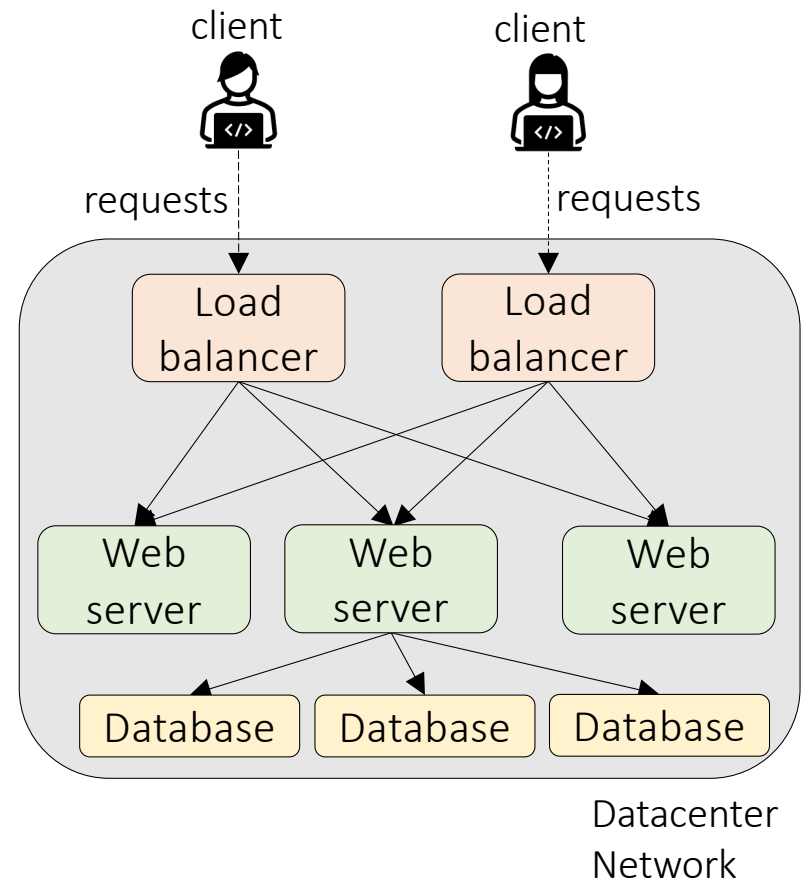


Real example of Everflow

- Many clients of an internal web service reported that a fraction of their connection requests were encountering timeouts
- This led to the violation of SLAs (Service Level Agreement)
- The suspect was the presence of packet drops, but where?

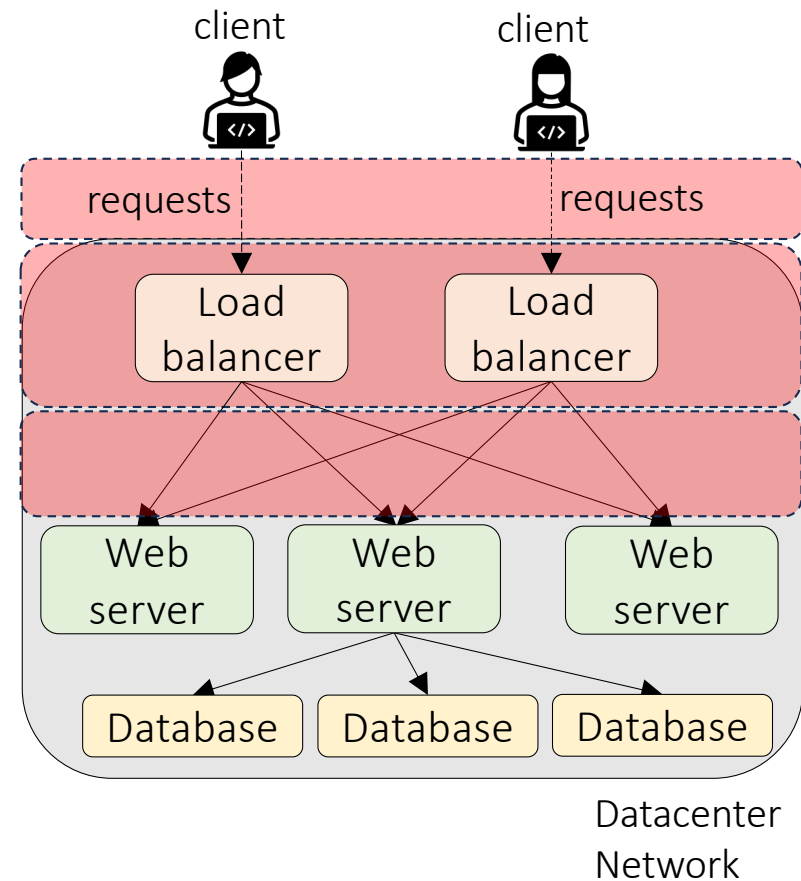
Real example of Everflow

- The drops could be anywhere!



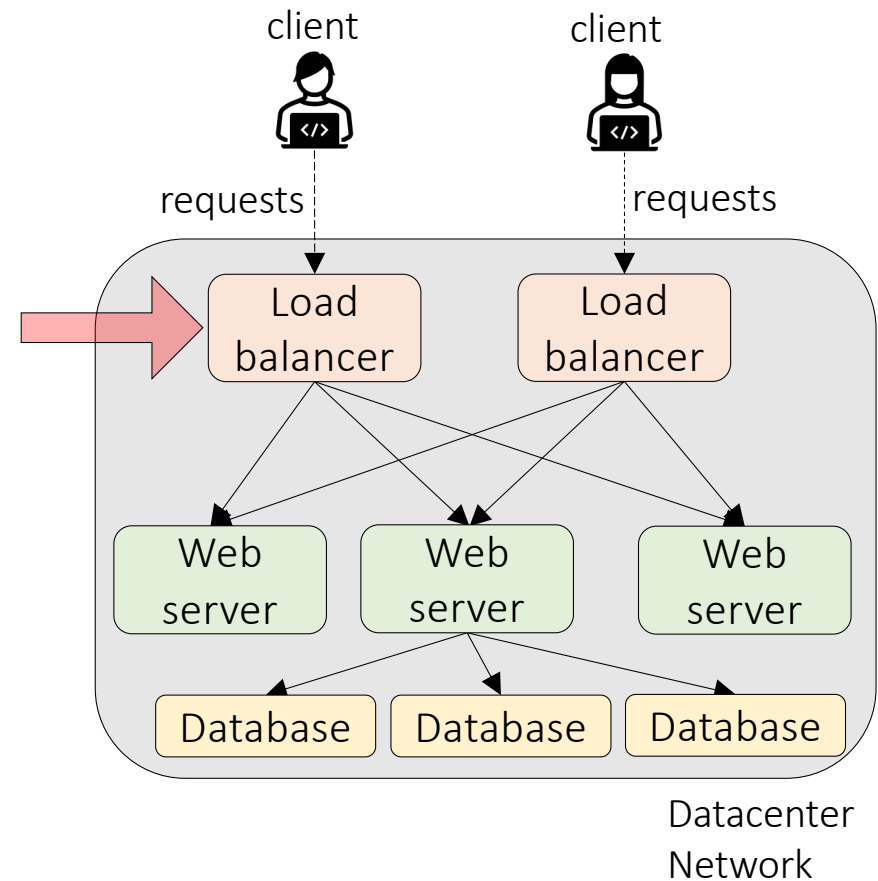
Real example of Everflow

- The drops could be anywhere!



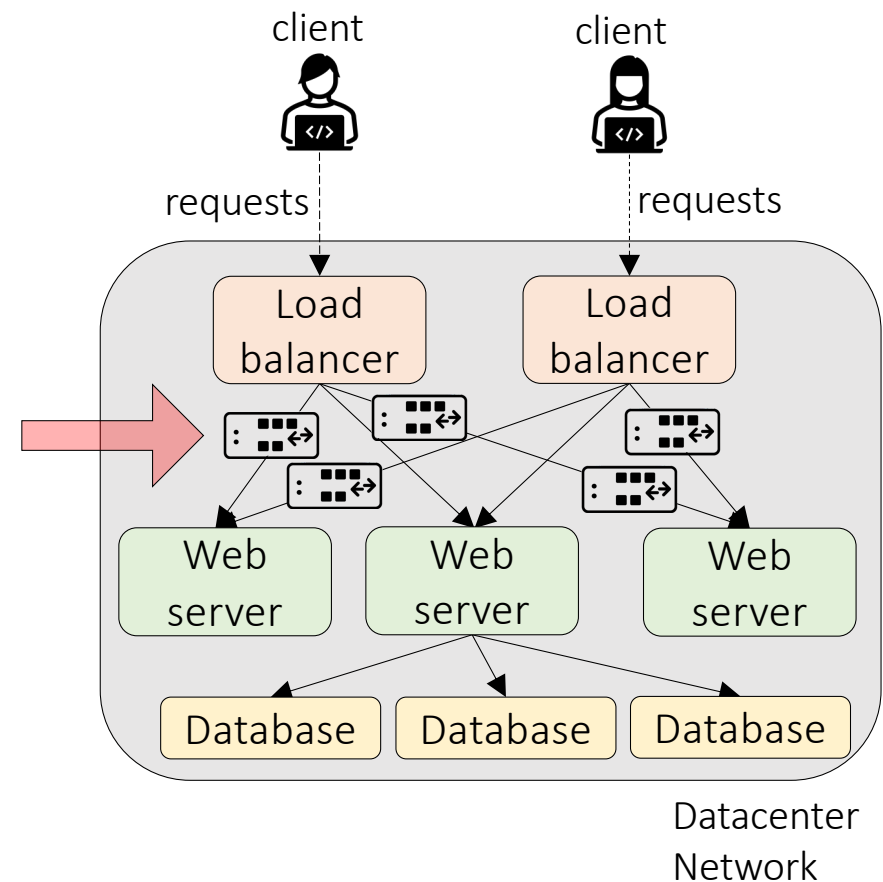
Real example of Everflow

- The drops could be anywhere!
- What would you do?
- No problems found at the error counters of the load balancers



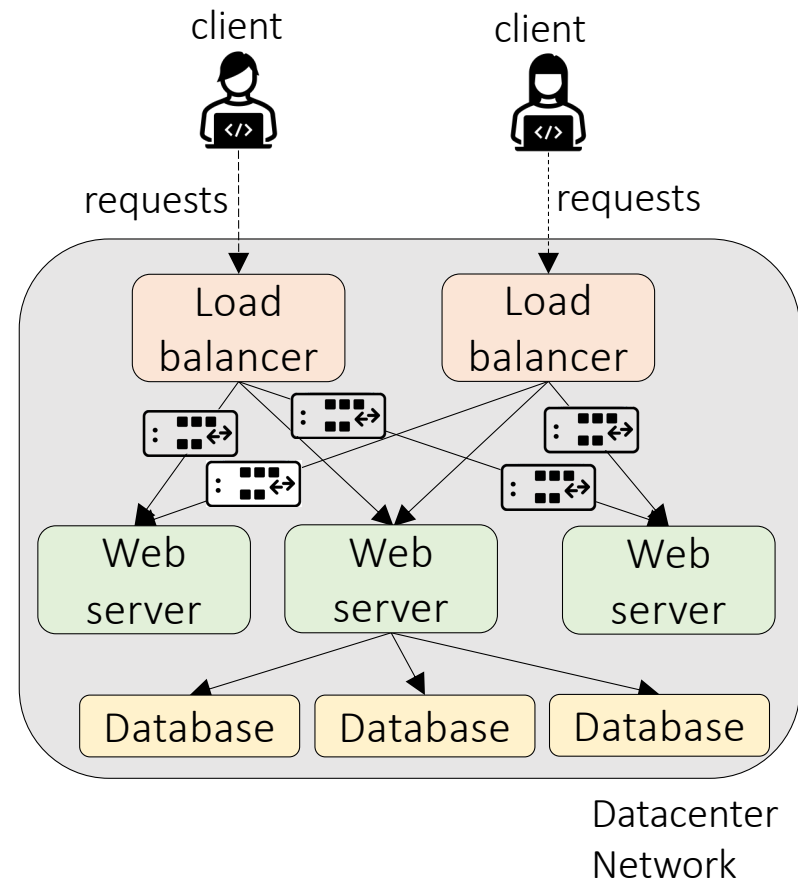
Real example of Everflow

- The drops could be anywhere!
- What would you do?
- No problems found at the error counters of the load balancers
- Some of the switches (too many to try all of them) checked and no problem found



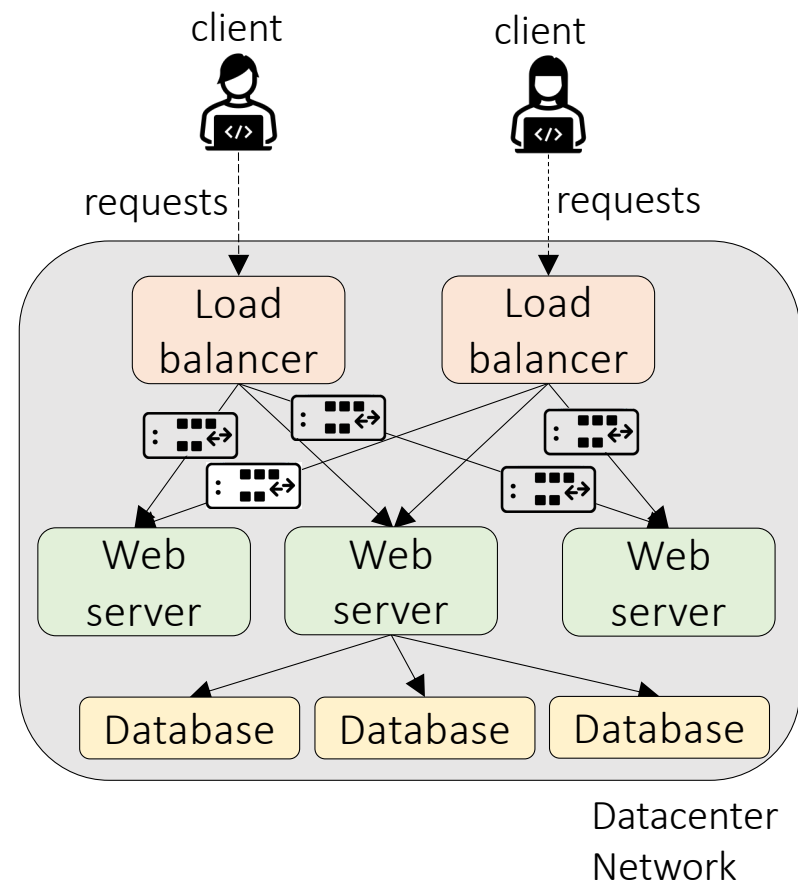
Real example of Everflow

- Everflow observed that many SYN packets did not reach the web server after the load balancers
- The trace analysis showed that all the SYN packets destined to a specific web server were dropped by a specific switch



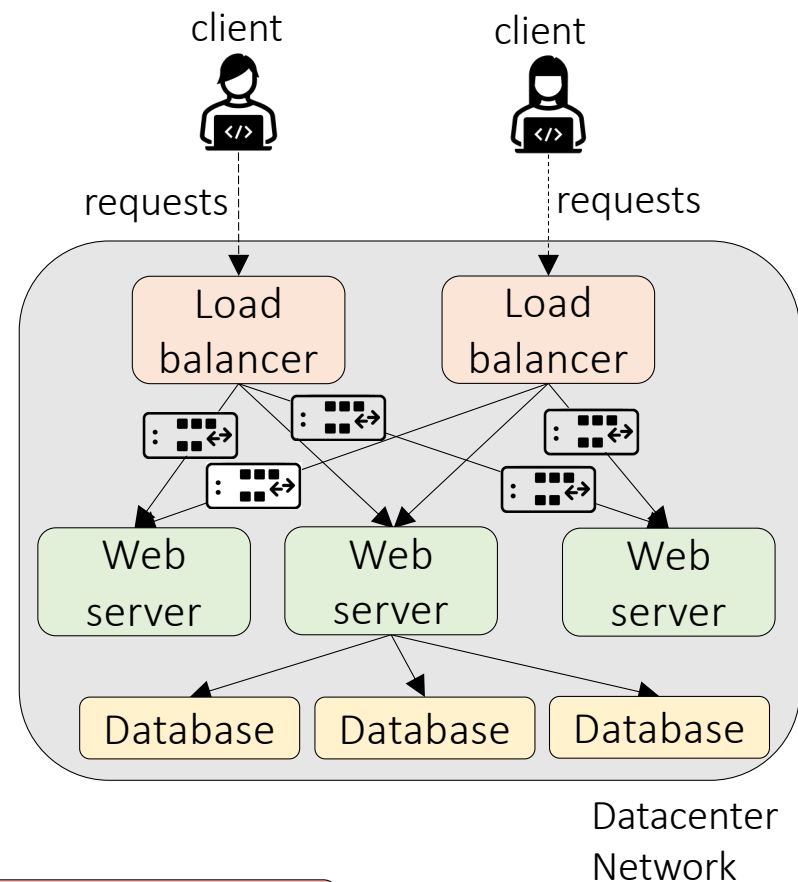
Real example of Everflow

- Everflow observed that many SYN packets did not reach the web server after the load balancers
- The trace analysis showed that all the SYN packets destined to a specific web server were dropped by a specific switch
- The operator found that the TCAM table was corrupted for that entry and the switch did not report any drops



Real example of Everflow

- Everflow observed that many SYN packets did not reach the web server after the load balancers
- The trace analysis showed that all the SYN packets destined to a specific web server were dropped by a specific switch
- The operator found that the TCAM table was corrupted for that entry and the switch did not report any drops

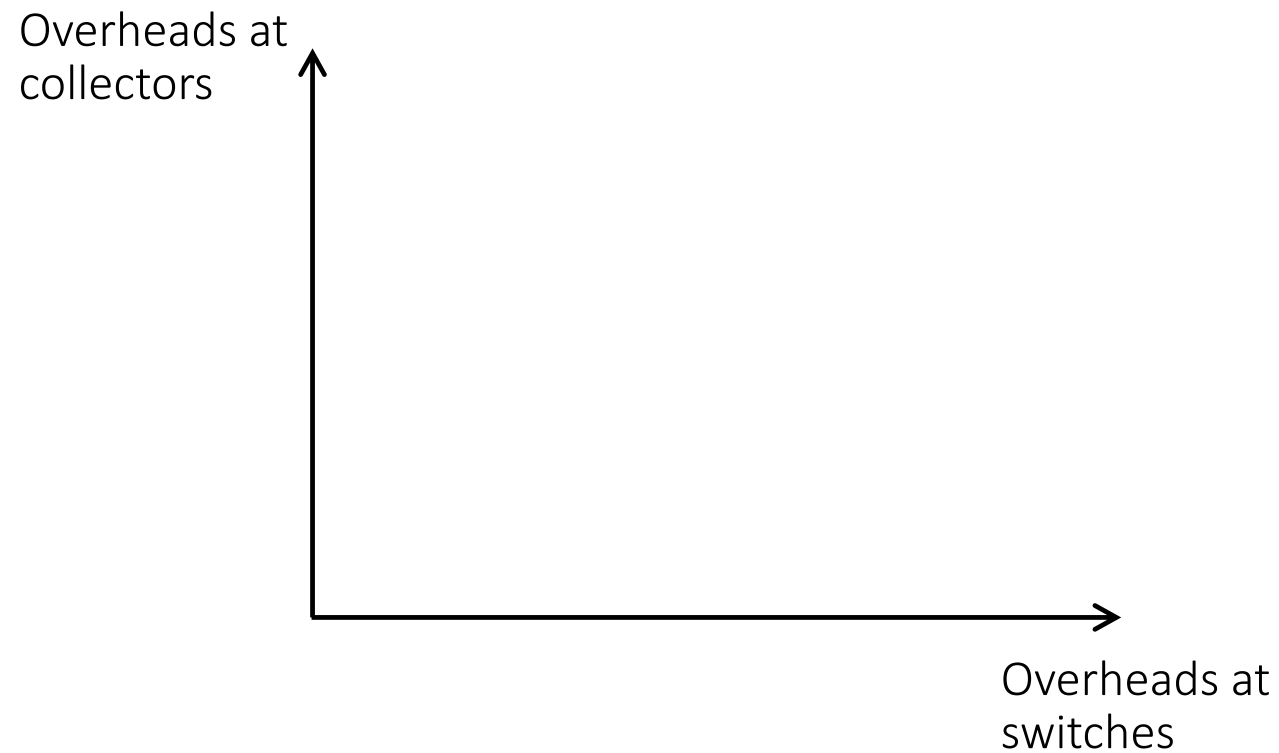


after a switch reboot all was fine 😊

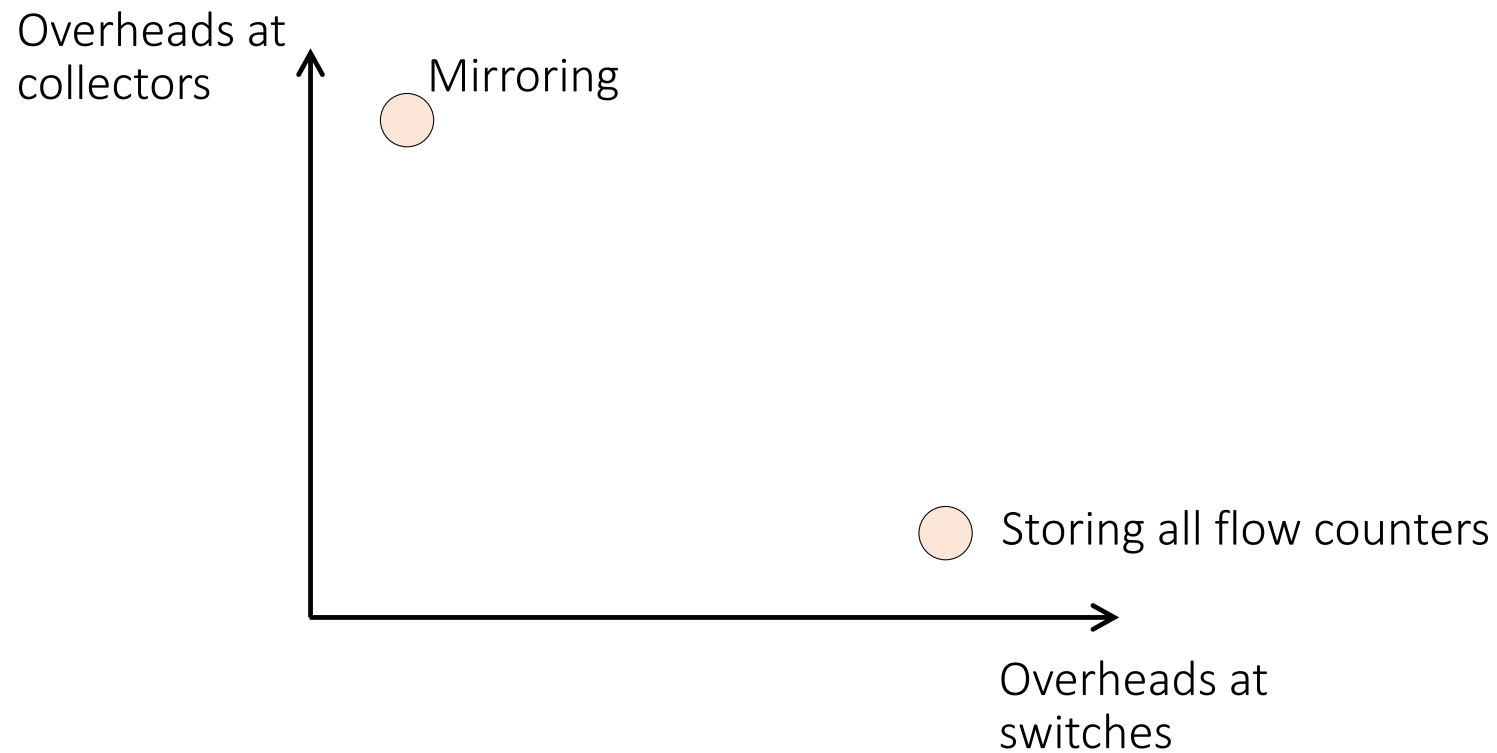
Takeaways

- Everflow is an example of a production solution
- Although powerful, it is still limited by the capabilities exposed by switches
- What if we can be wild and change programmatically switch operation?

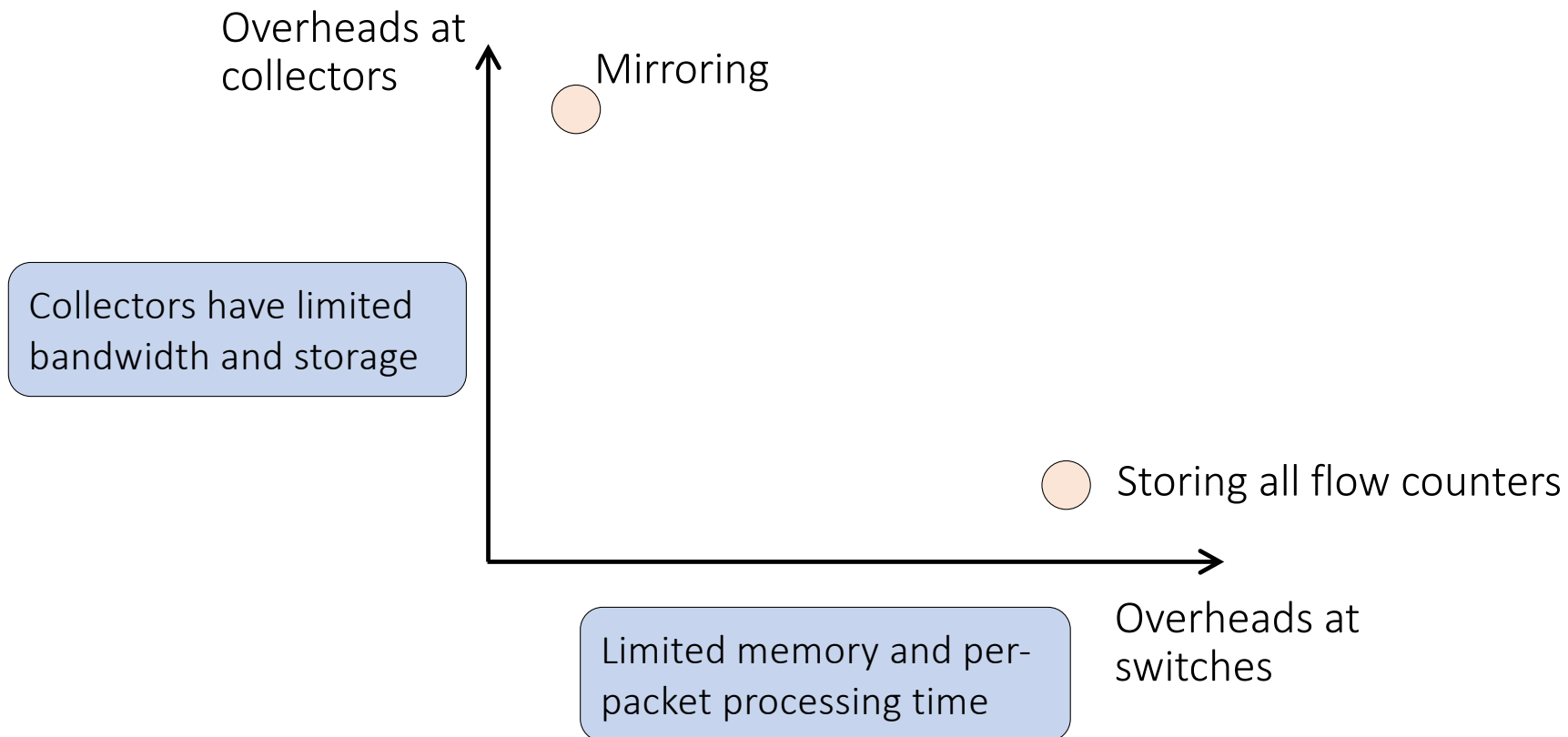
An important trade-off to consider



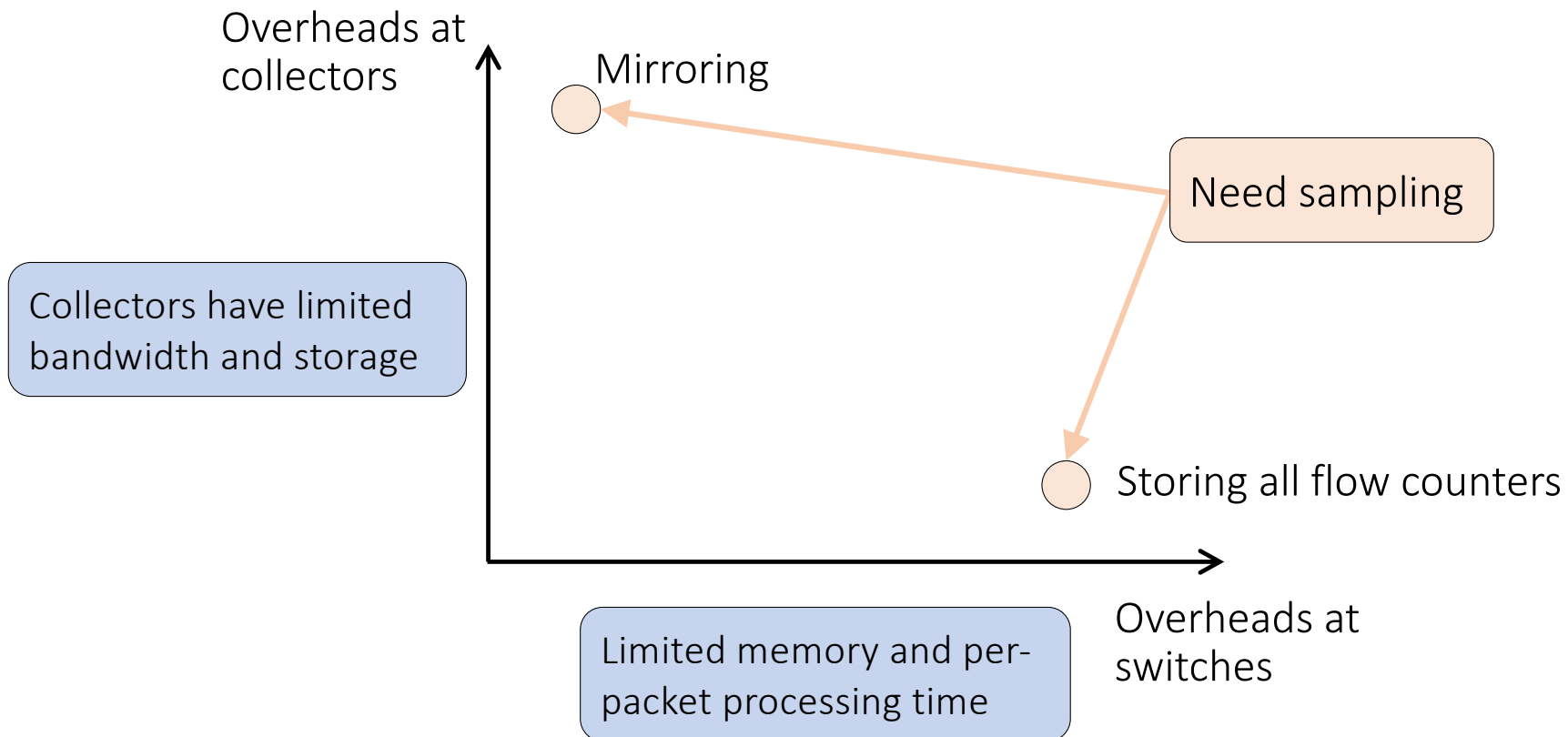
An important trade-off to consider



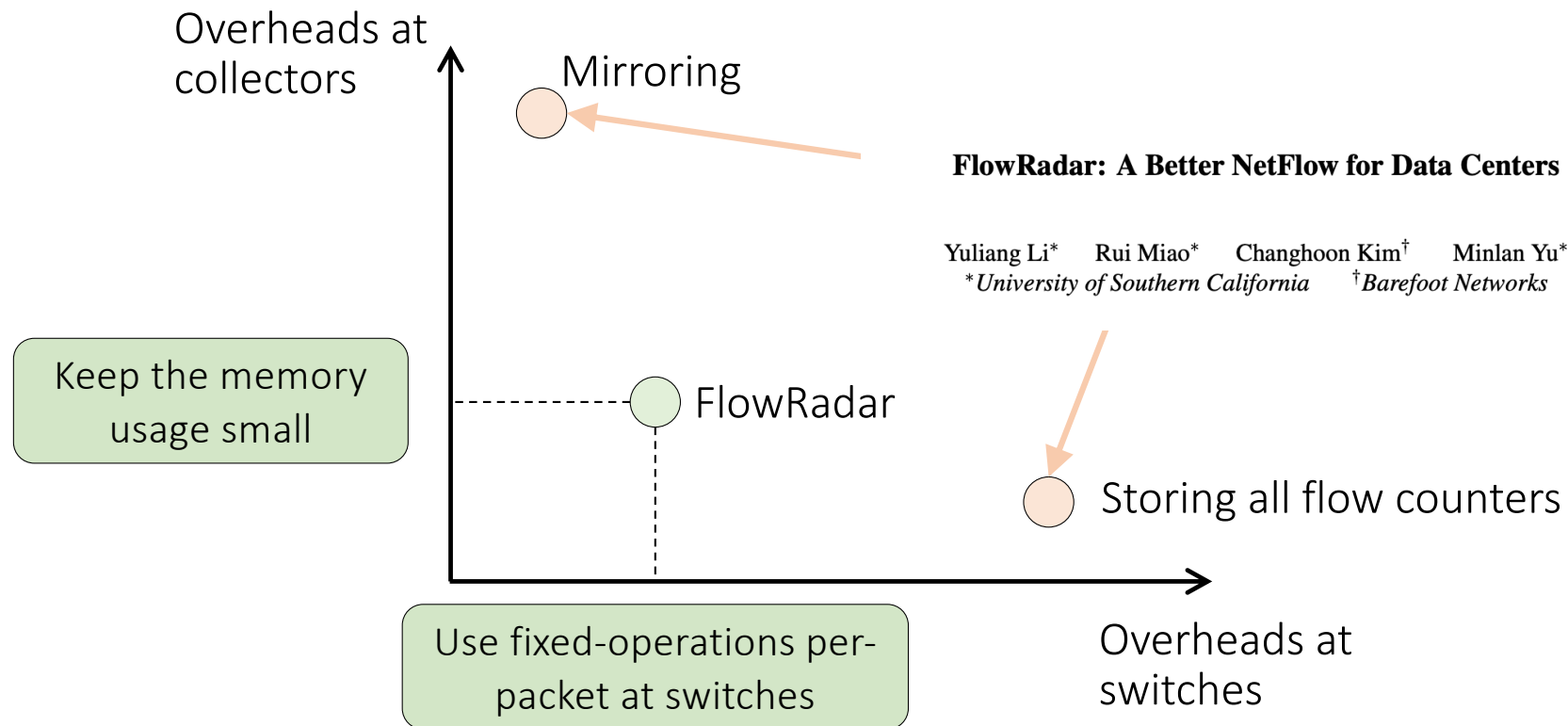
An important trade-off to consider



An important trade-off to consider



An important trade-off to consider

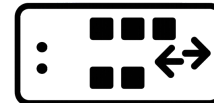
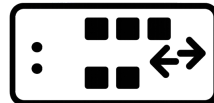
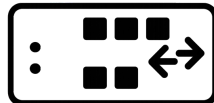


FlowRadar Architecture

Collectors



Network

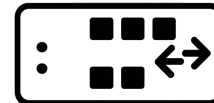
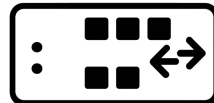
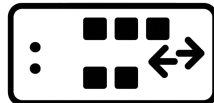


FlowRadar Architecture

Collectors

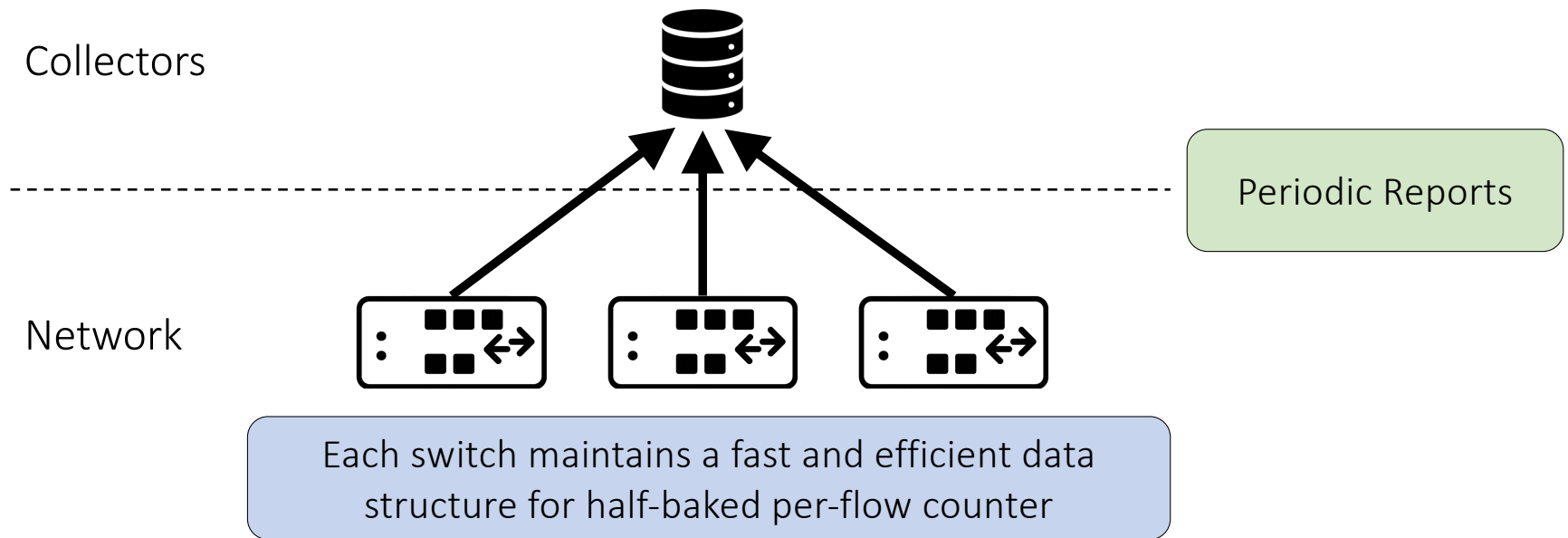


Network

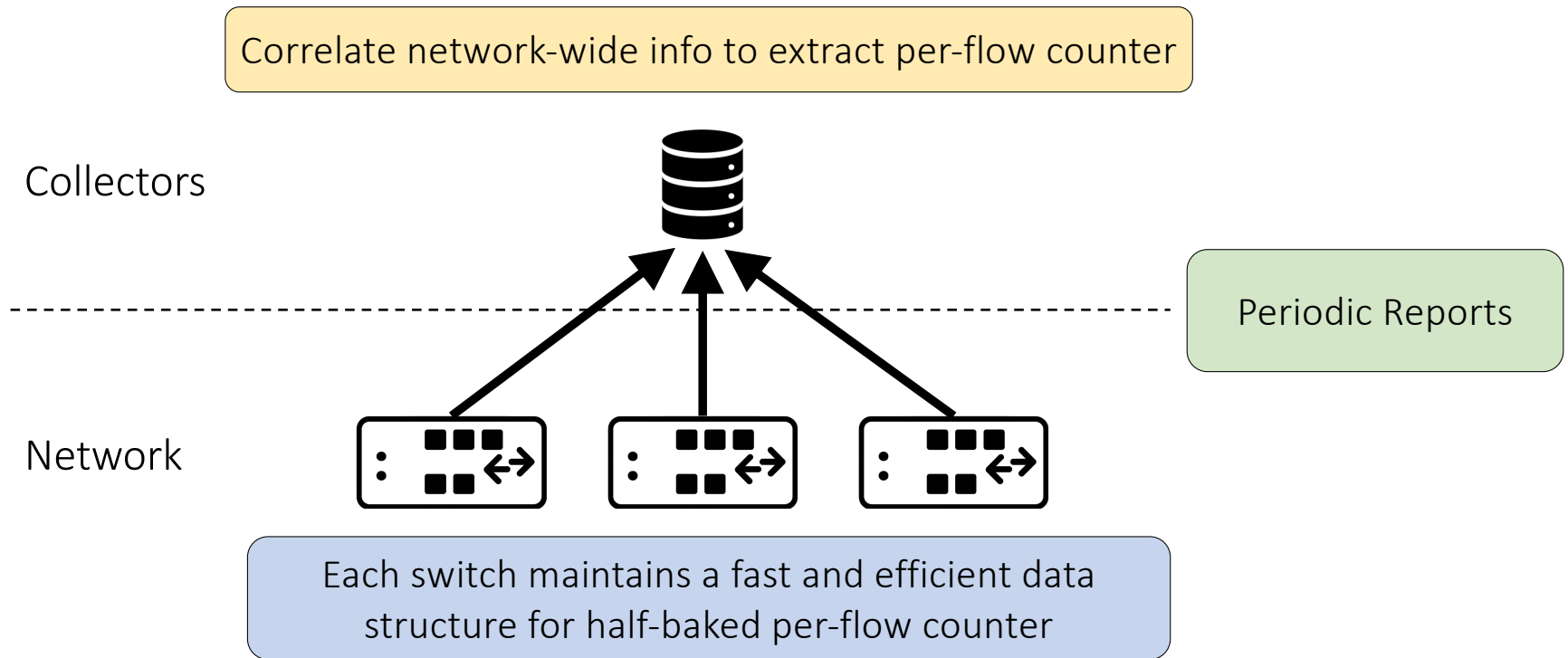


Each switch maintains a fast and efficient data structure for half-baked per-flow counter

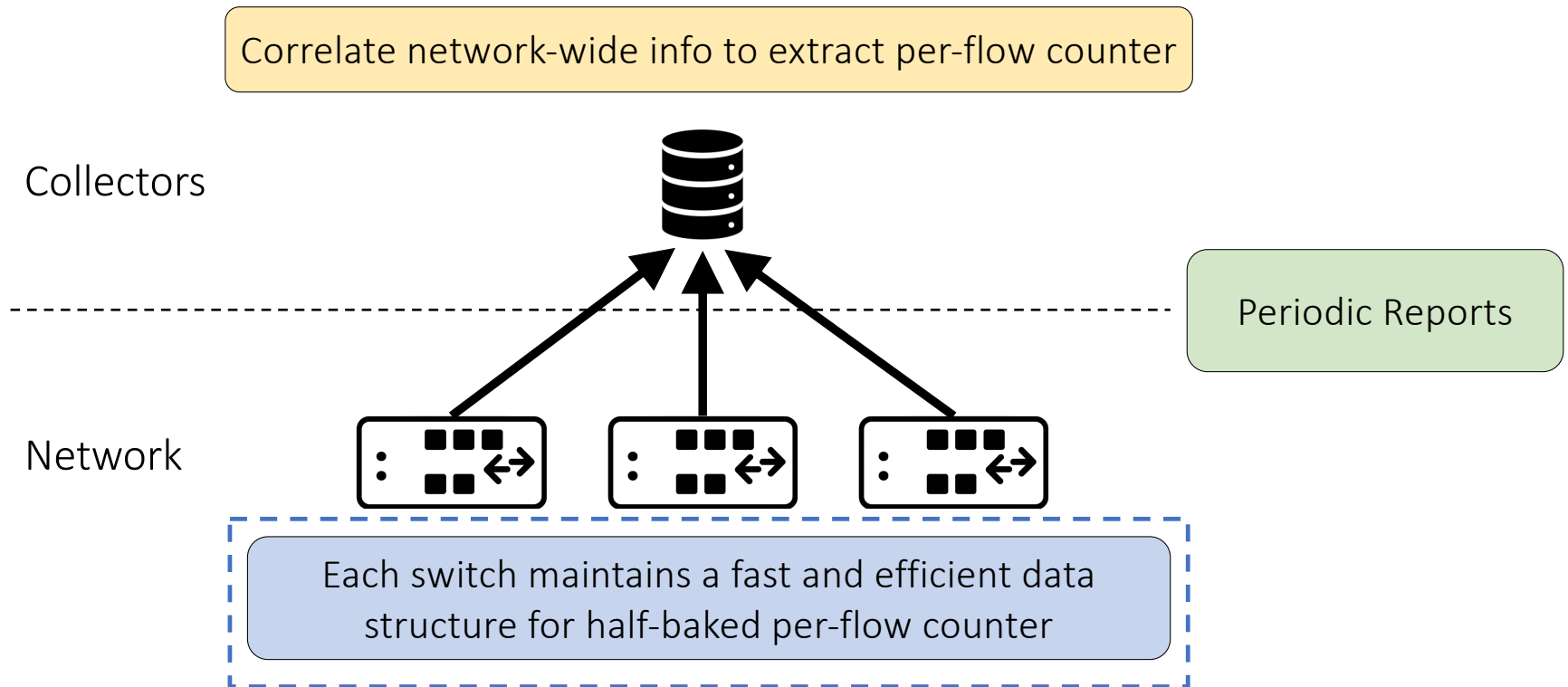
FlowRadar Architecture



FlowRadar Architecture



FlowRadar Architecture

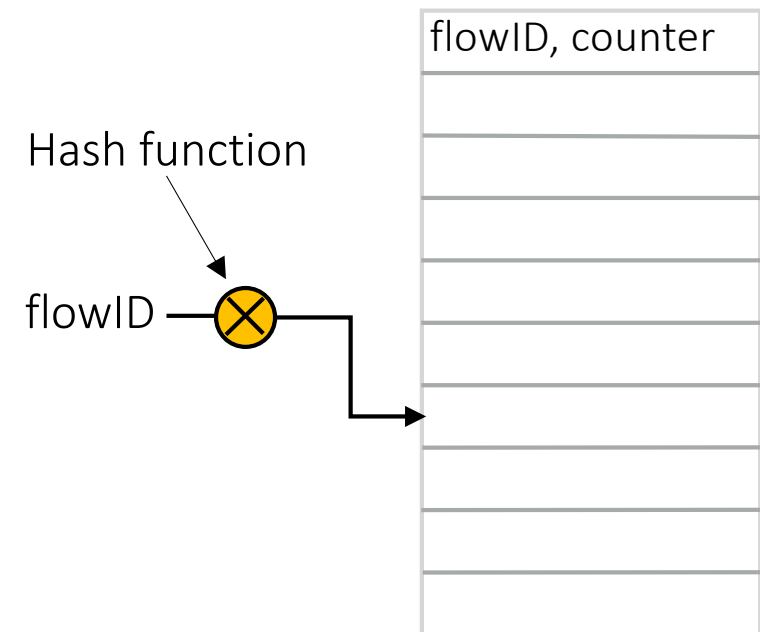


How to store the flow counters in the switch?

- Hash table?

How to store the flow counters in the switch?

- Hash table? To avoid collisions this might require too much memory ⚠

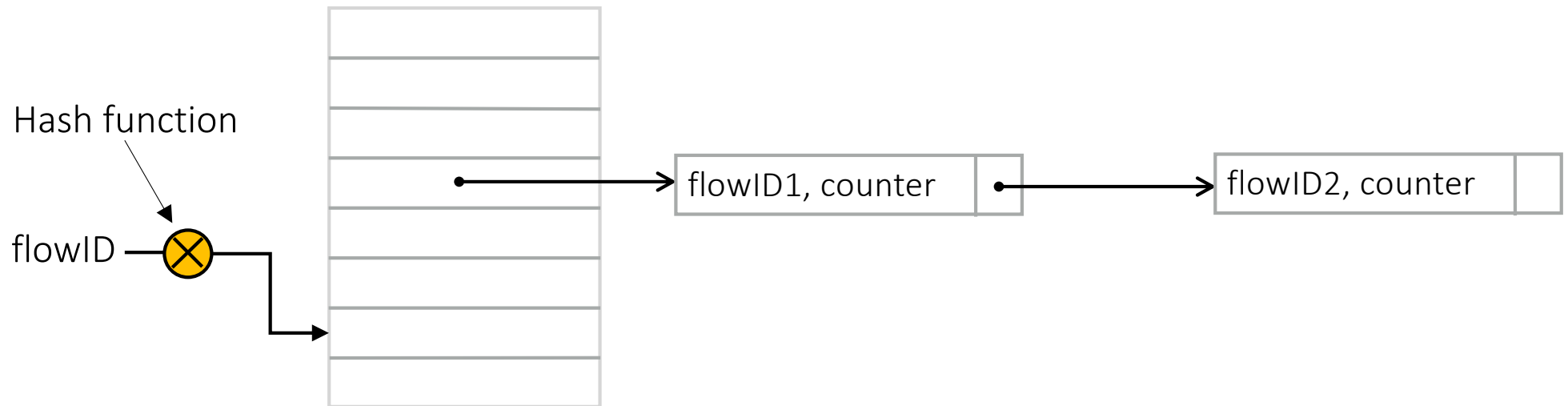


How to store the flow counters in the switch?

- Hash table? To avoid collisions this might require too much memory 
- Linked List?

How to store the flow counters in the switch?

- Hash table? To avoid collisions this might require too much memory ⚠
- Linked List? Multiple non-constant memory accesses ⚠



How to store the flow counters in the switch?

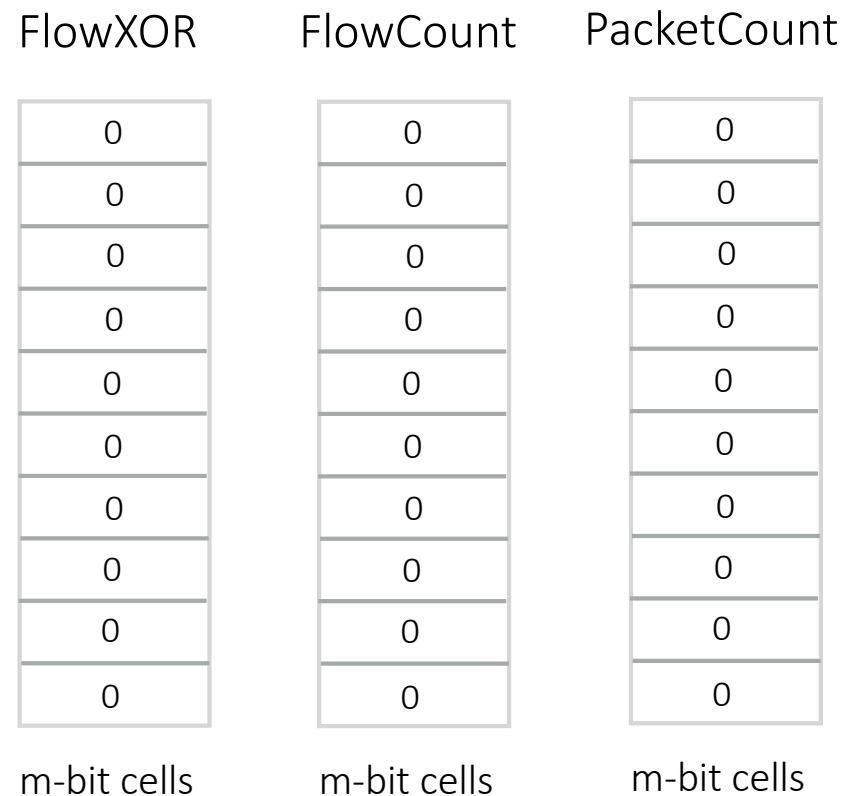
- The idea from FlowRadar is to use Invertible Bloom Lookup Tables
- In the last lecture we learned that each cell contains three fields:
 - Count: counts the number of entries mapped to this cell
 - KeySum: it is the sum of all the keys mapped to this cell
 - ValueSum: it is the sum of all the values mapped to this cell

Count	KeySum	ValueSum
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0

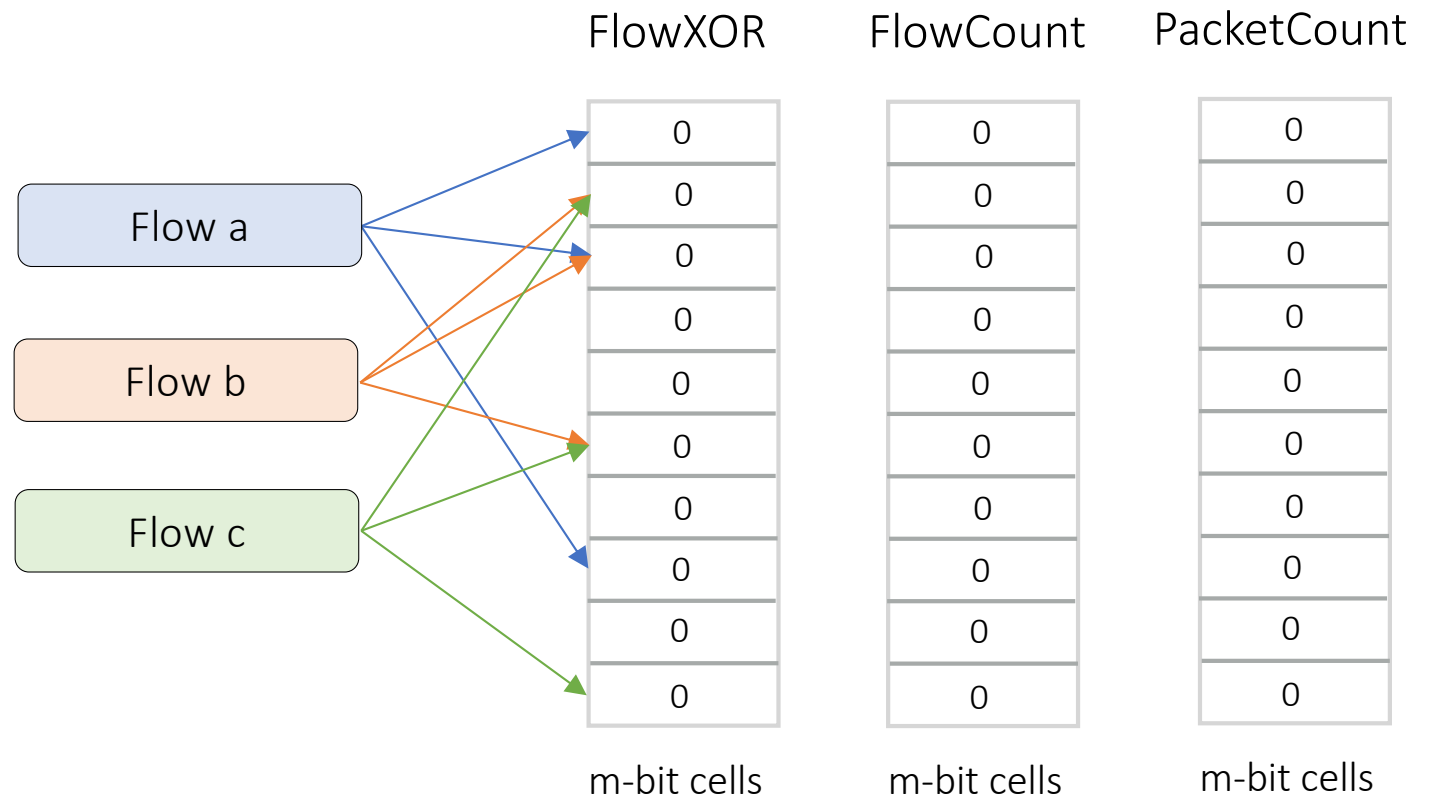
m-bit cells m-bit cells m-bit cells

How to store the flow counters in the switch?

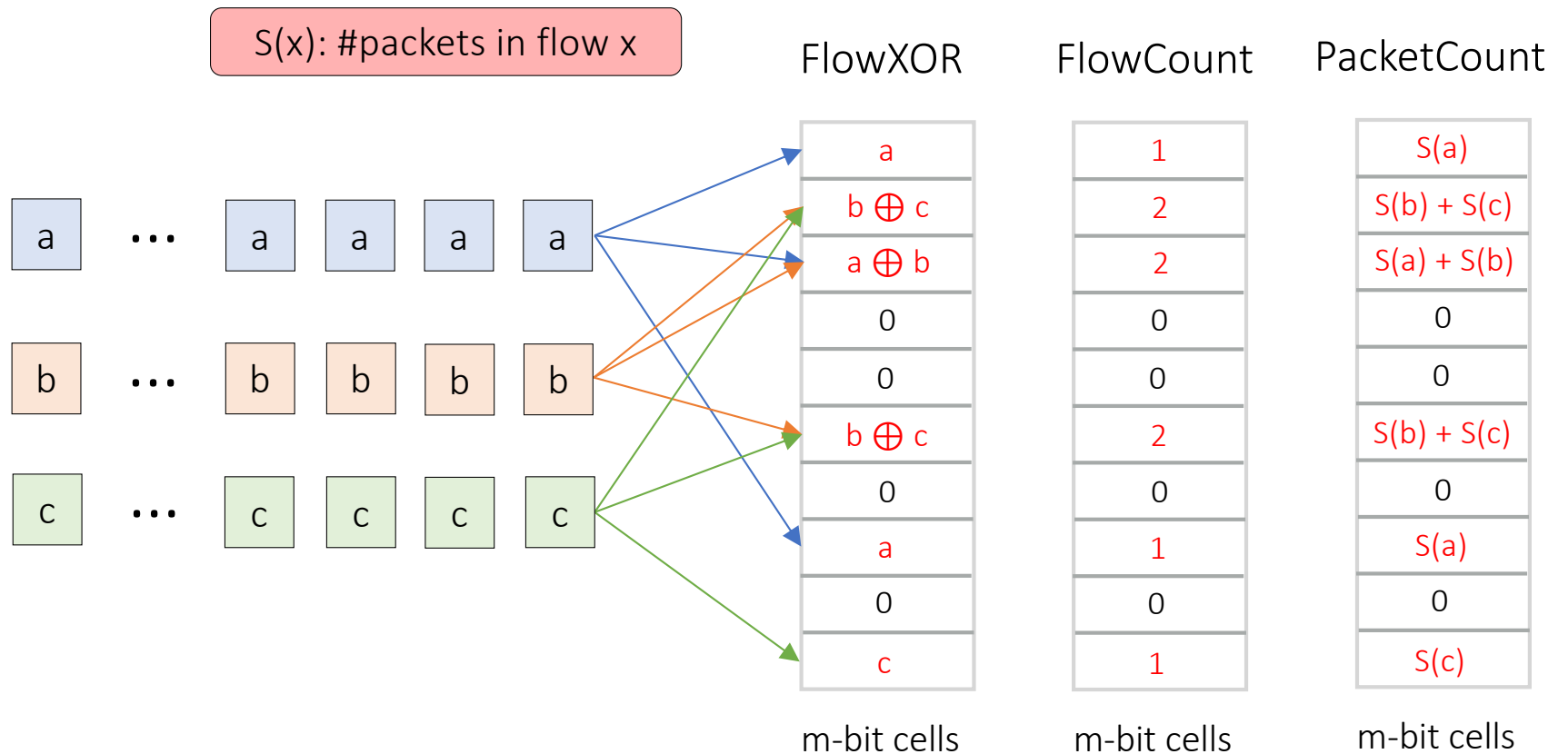
- In FlowRadar the idea is to use:
 - **FlowXOR**: keeps the XOR of all the flows mapped in the bin
 - **FlowCount**: keeps the number of flow mapped in the bin
 - **PacketCount**: keeps the number of packets of all the flows mapped in the bin



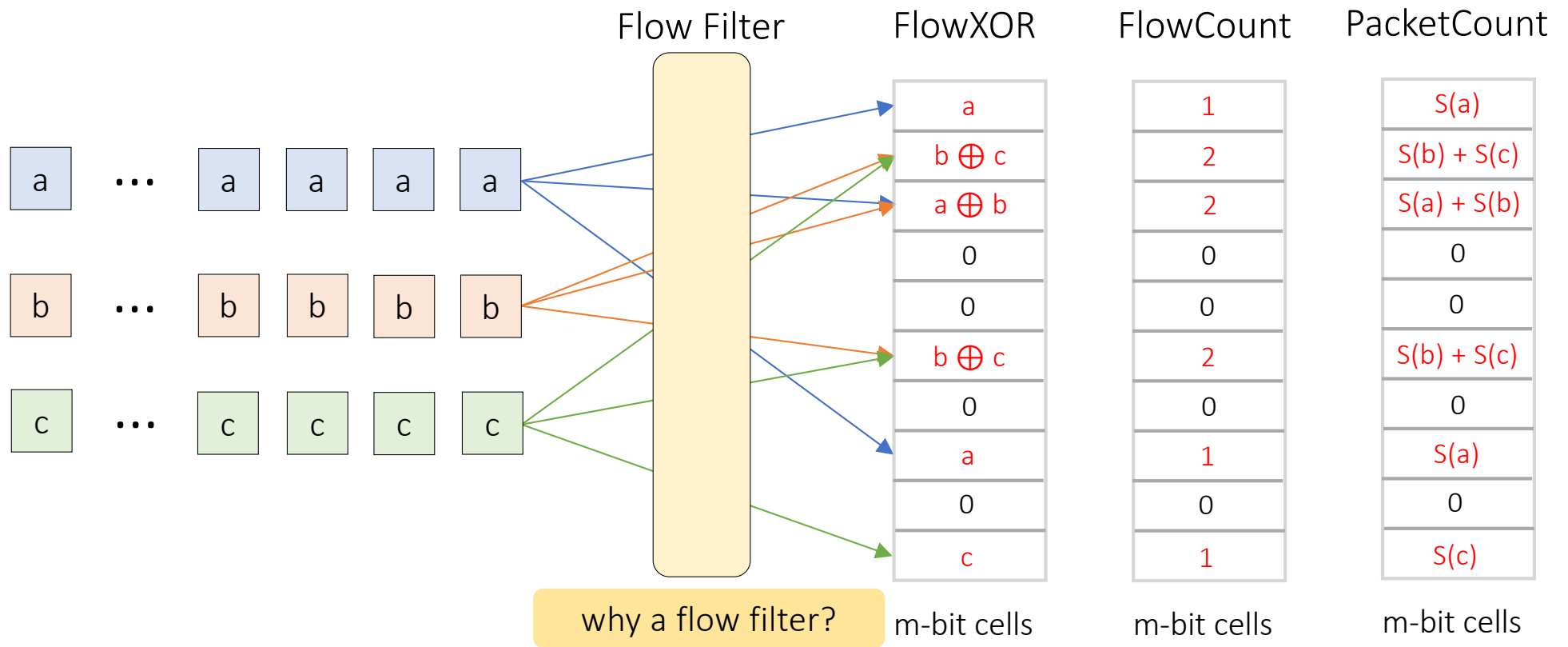
How to store the flow counters in the switch?



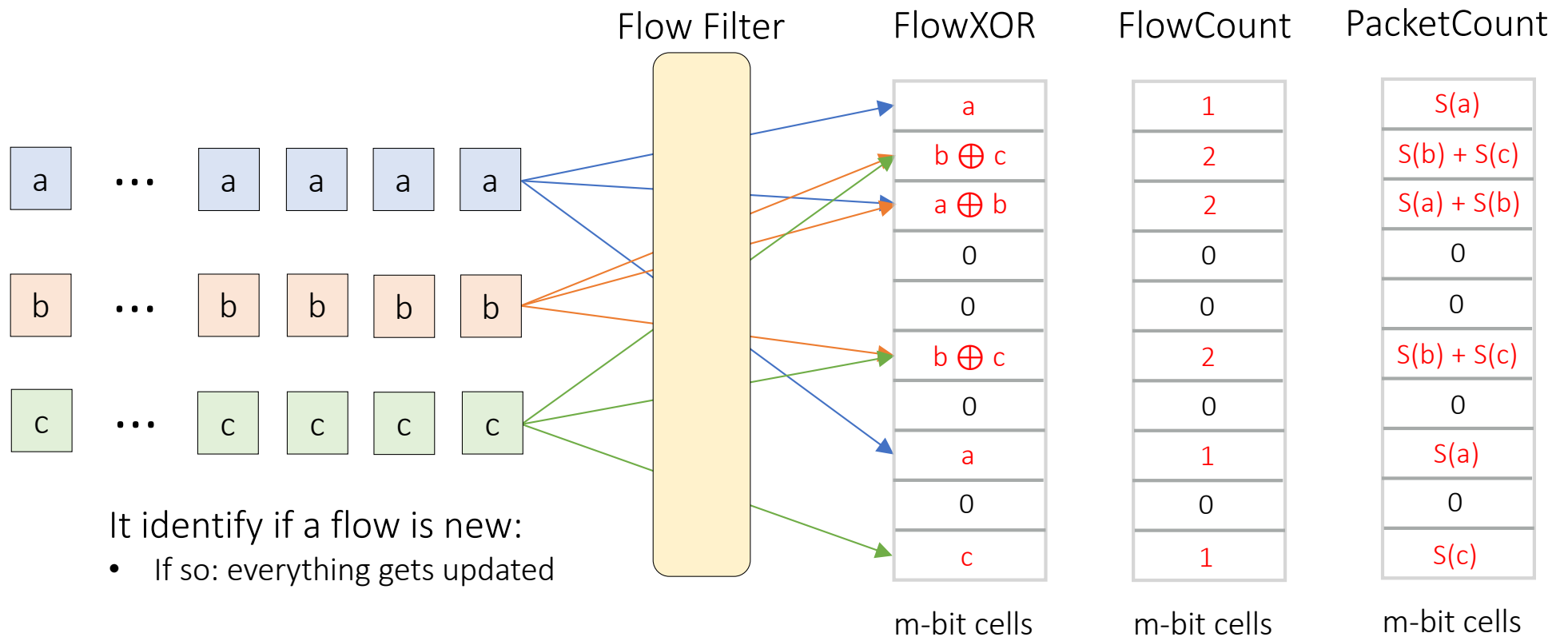
How to store the flow counters in the switch?



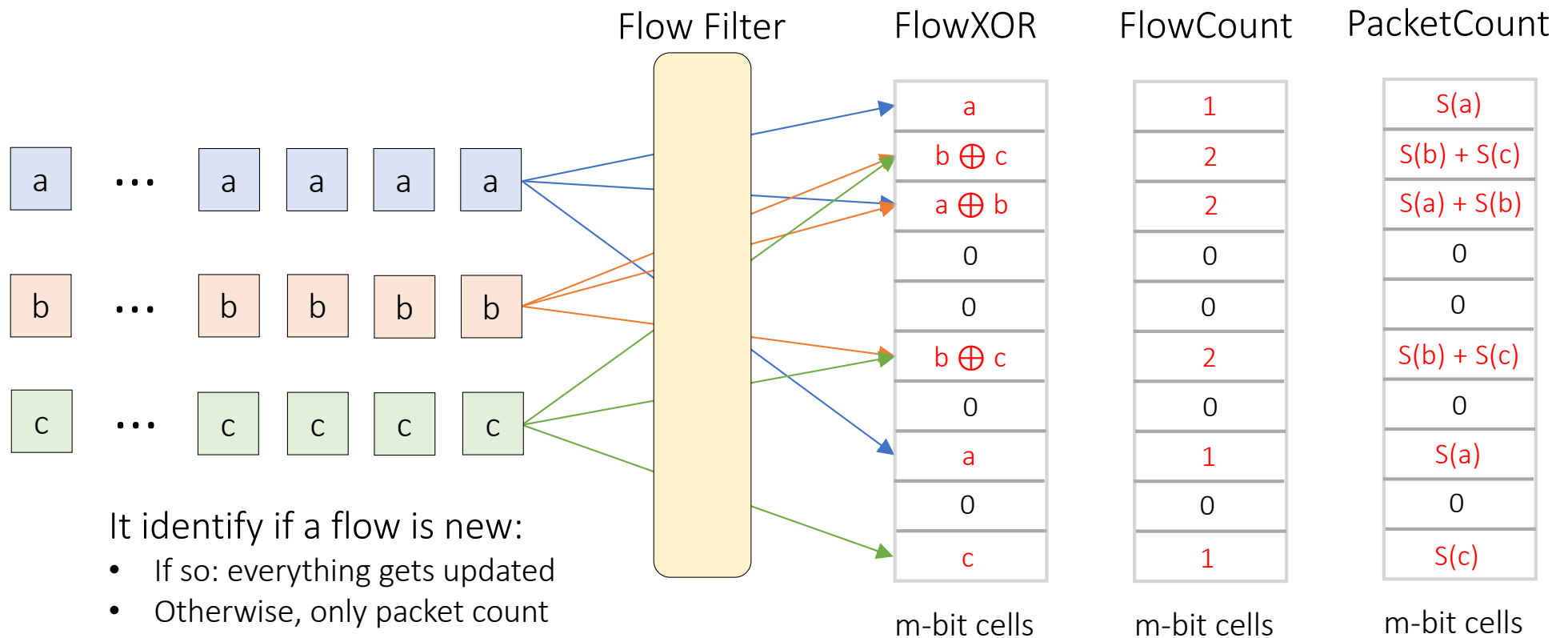
How to store the flow counters in the switch?



How to store the flow counters in the switch?



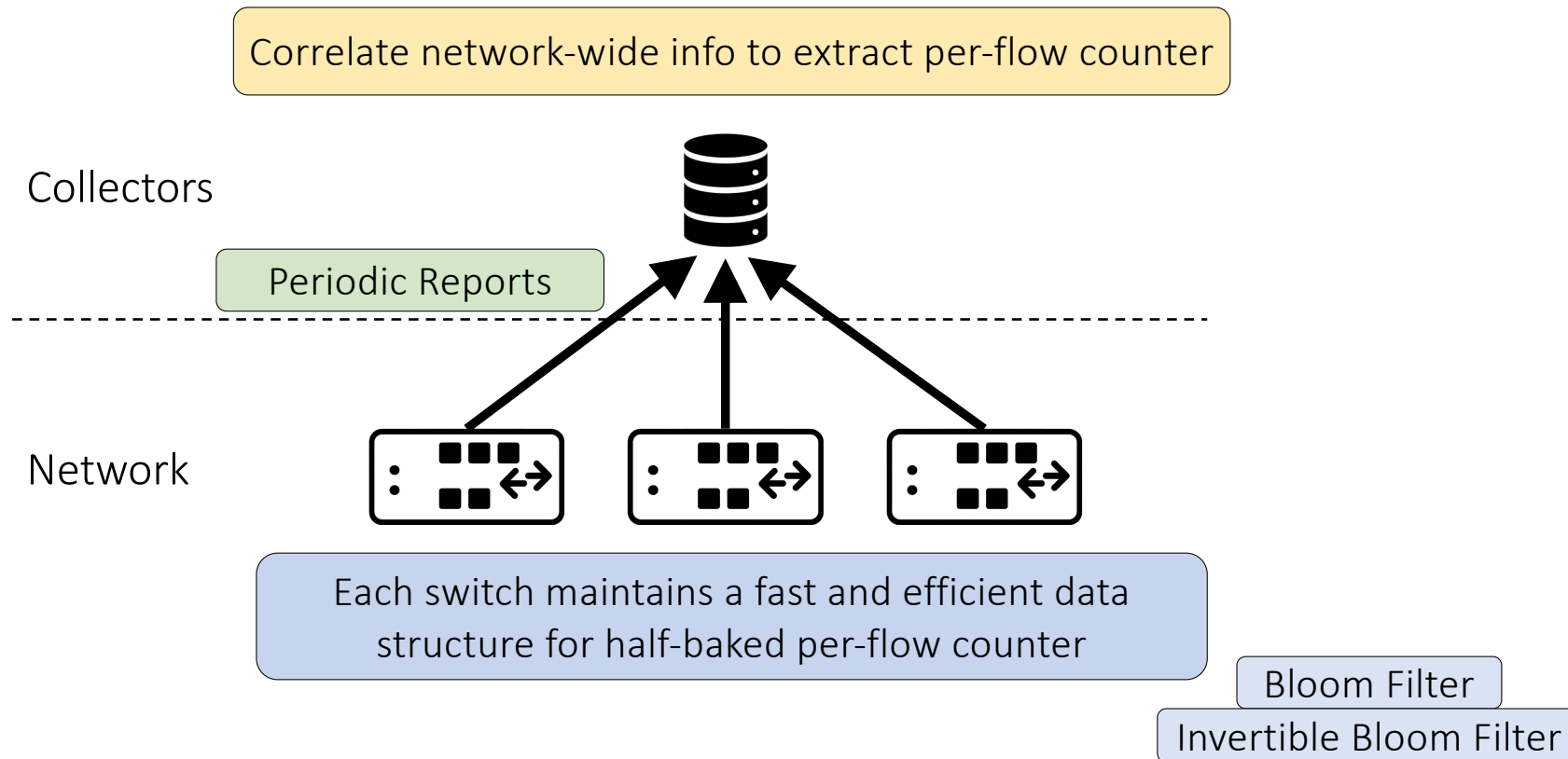
How to store the flow counters in the switch?



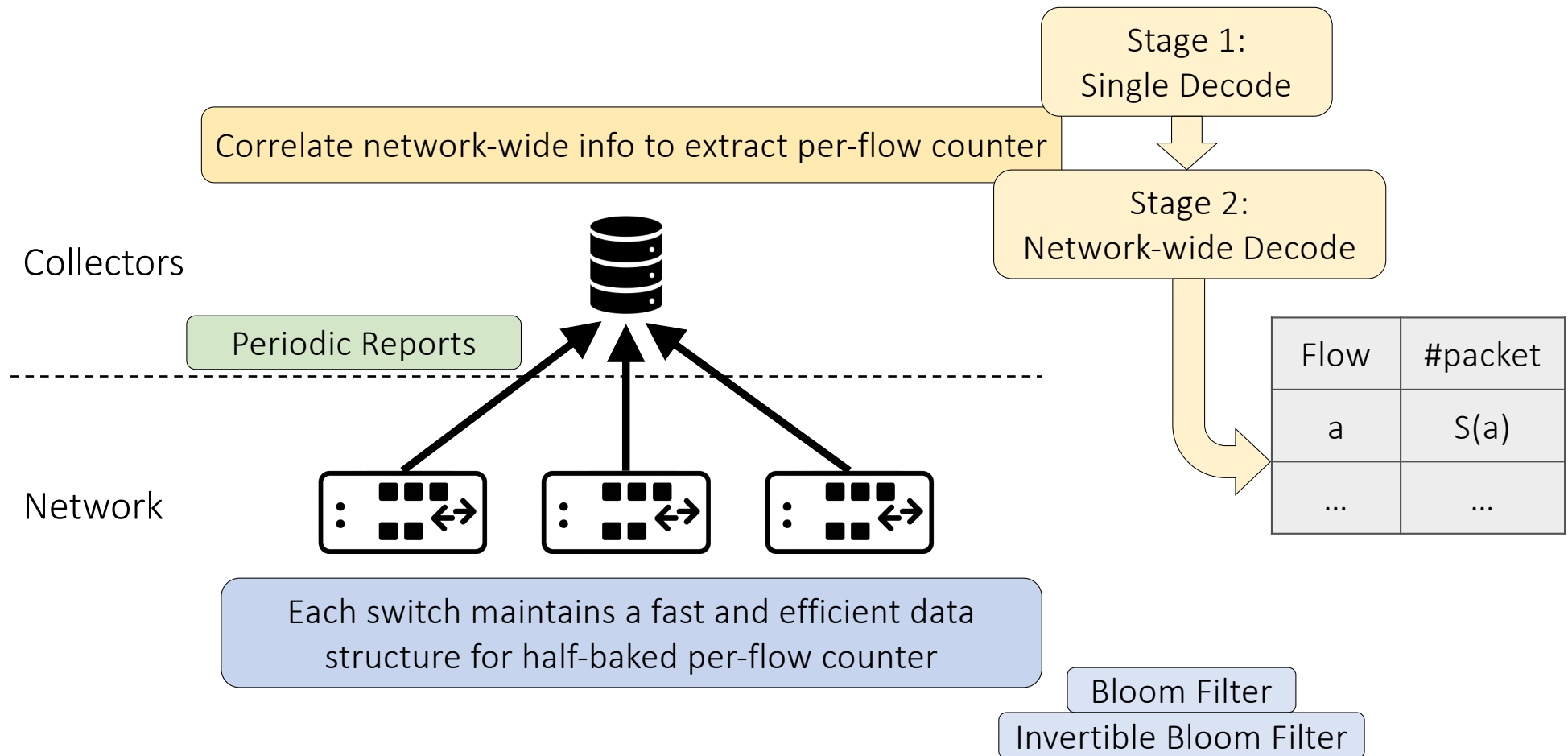
Questions

- How would you implement the flow filter?

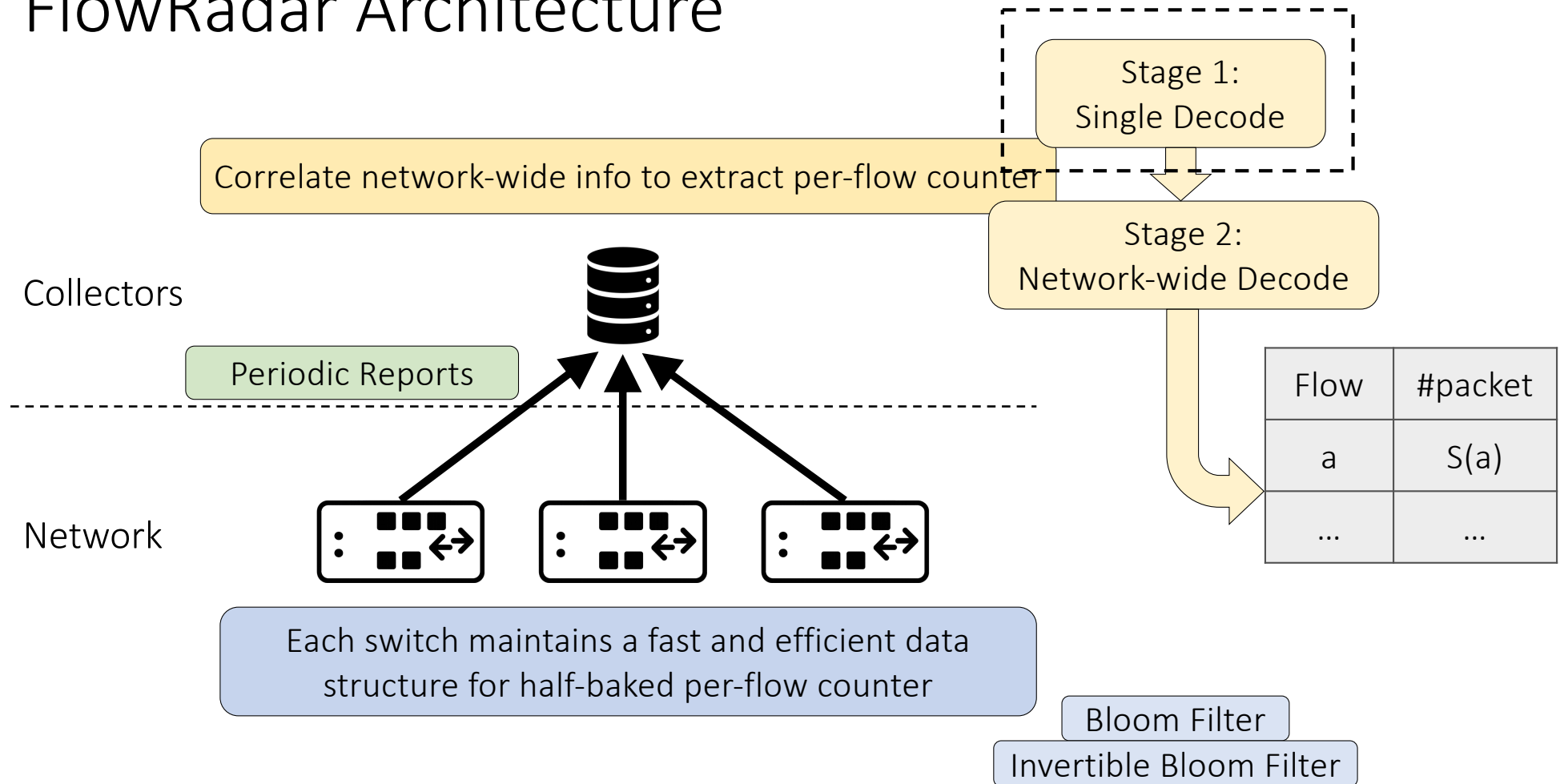
FlowRadar Architecture



FlowRadar Architecture



FlowRadar Architecture



FlowRadar: Single Decode

- Find a pure cell: a cell with one flow

FlowXOR	FlowCount	PacketCount
a	1	5
$a \oplus b$	2	12
$b \oplus c \oplus d$	3	13
0	0	0
0	0	0
$b \oplus c \oplus d$	3	13
0	0	0
a	1	5
0	0	0
$c \oplus d$	2	6

m-bit cells m-bit cells m-bit cells

FlowRadar: Single Decode

- Find a pure cell: a cell with one flow
- Remove the flow from all the cells

Decoded:

Flow	#packet
a	5

FlowXOR

0
b
$b \oplus c \oplus d$
0
0
$b \oplus c \oplus d$
0
0
0
$c \oplus d$

m-bit cells

FlowCount

0
1
3
0
0
3
0
0
0
2

m-bit cells

PacketCount

0
7
13
0
0
13
0
0
0
6

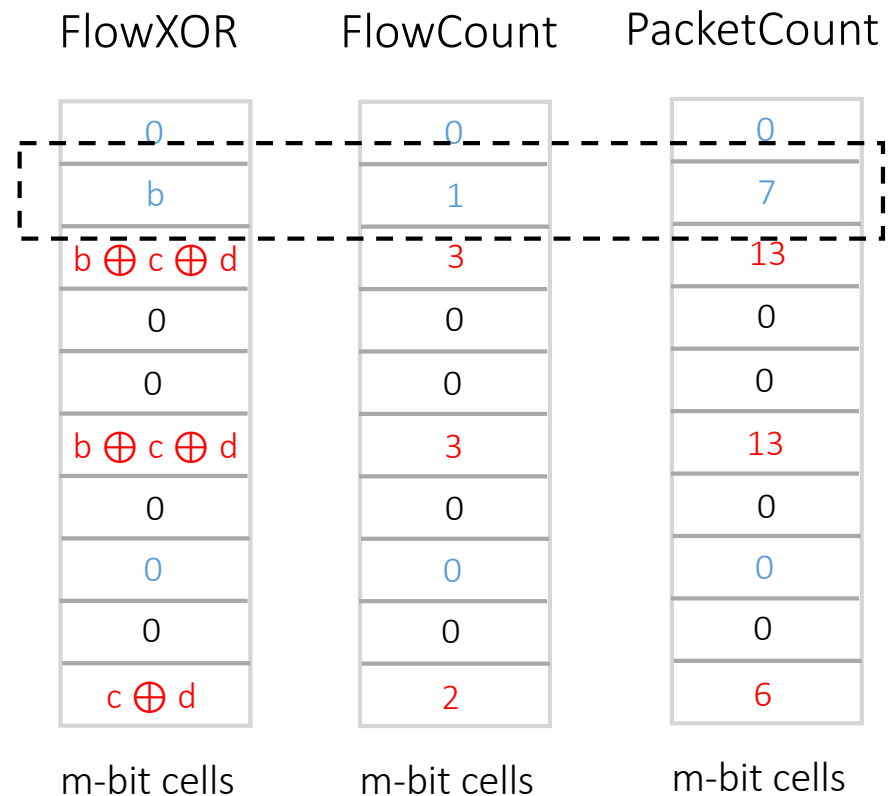
m-bit cells

FlowRadar: Single Decode

- Find a pure cell: a cell with one flow
- Remove the flow from all the cells
- This can create more pure cells

Decoded:

Flow	#packet
a	5



FlowRadar: Single Decode

- Find a pure cell: a cell with one flow
- Remove the flow from all the cells
- This can create more pure cells

Decoded:

Flow	#packet
a	5
b	7

FlowXOR

0
0
$c \oplus d$
0
0
$c \oplus d$
0
0
0
$c \oplus d$

m-bit cells

FlowCount

0
0
2
0
0
2
0
0
0
2

m-bit cells

PacketCount

0
0
6
0
0
6
0
0
0
6

m-bit cells

FlowRadar: Single Decode

- What about now? ☹️

Decoded:

Flow	#packet
a	5
b	7

FlowXOR

0
0
$c \oplus d$
0
0
$c \oplus d$
0
0
0
$c \oplus d$

m-bit cells

FlowCount

0
0
2
0
0
2
0
0
0
2

m-bit cells

PacketCount

0
0
6
0
0
6
0
0
0
6

m-bit cells

FlowRadar: Single Decode

- What about now? ☹️
- FlowRadar leverages network-wide info to decode more flow 😊

Decoded:

Flow	#packet
a	5
b	7

FlowXOR

0
0
$c \oplus d$
0
0
$c \oplus d$
0
0
0
$c \oplus d$

m-bit cells

FlowCount

0
0
2
0
0
2
0
0
0
2

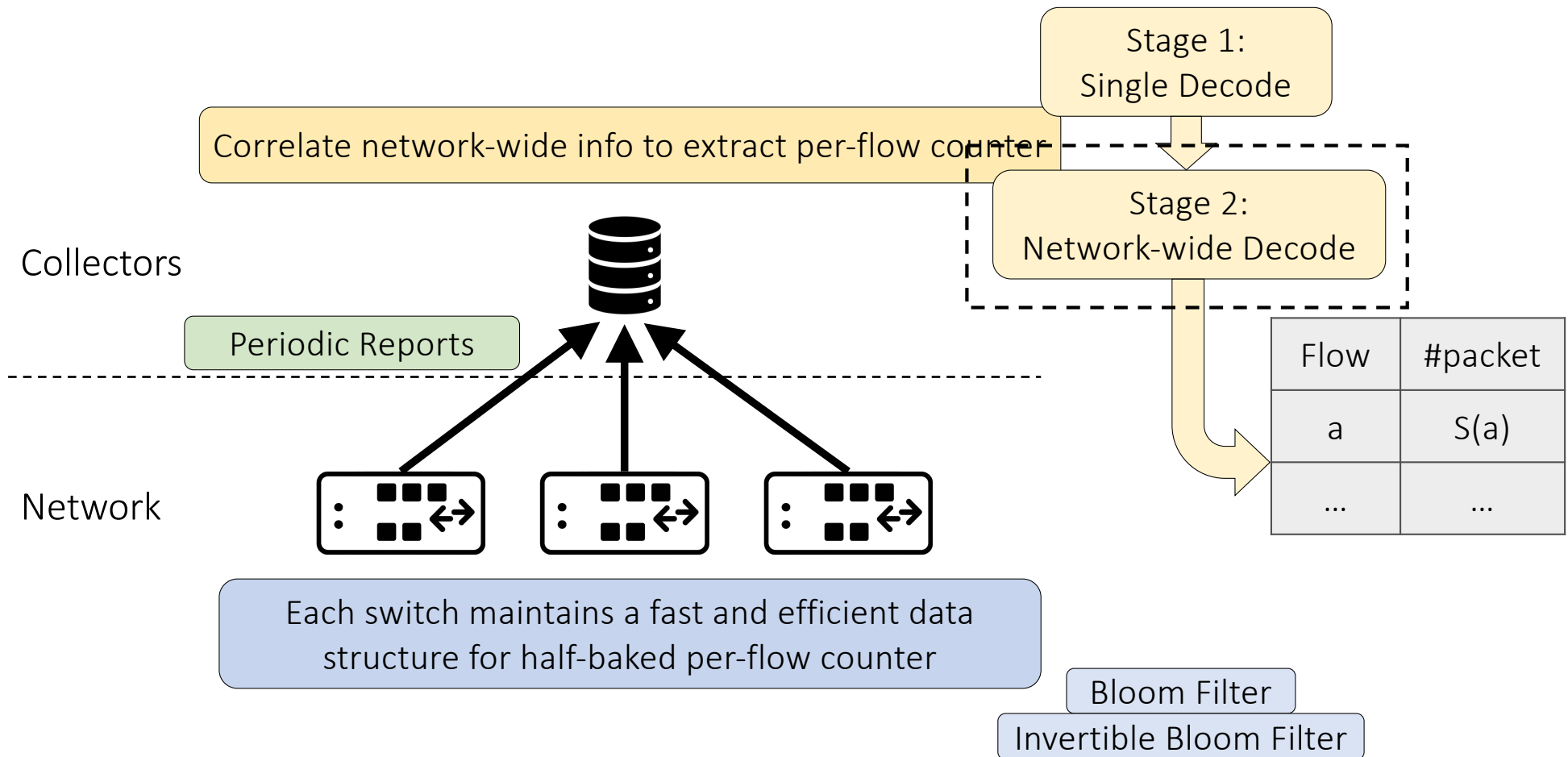
m-bit cells

PacketCount

0
0
6
0
0
6
0
0
0
6

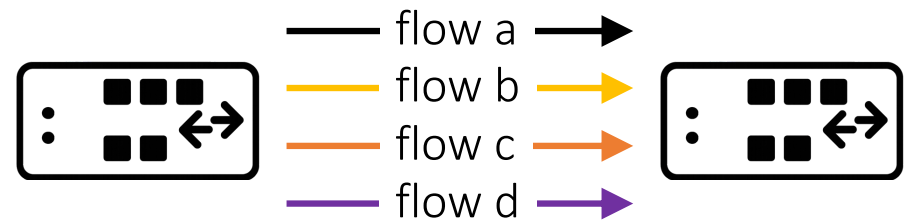
m-bit cells

FlowRadar Architecture



FlowRadar: Network-Wide decode

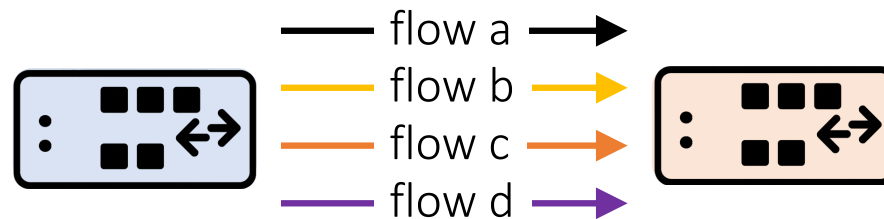
- The set of flows overlap across hops
- We can use the redundancy to decode more flows
 - Use different hash functions across flows



FlowRadar: Network-Wide decode

FlowXor	a	$a \oplus c \oplus d$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	1	3	3	3	2
PktCount					

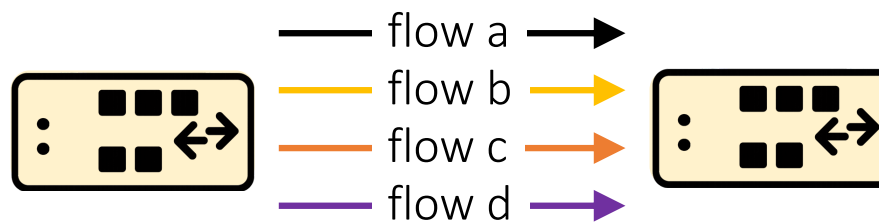
Instead of using 5 cells to decode 4 flows..



FlowXor	$a \oplus d$	$a \oplus c$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	2	2	3	3	2
PktCount					

FlowRadar: Network-Wide decode

FlowXor	$a \oplus d$	$a \oplus c$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$	a	$a \oplus c \oplus d$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	2	2	3	3	2	1	3	3	3	2
PktCount										



FlowRadar: Network-Wide decode

FlowXor	$a \oplus d$	$a \oplus c$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$	a	$a \oplus c \oplus d$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	2	2	3	3	2	1	3	3	3	2
PktCount										

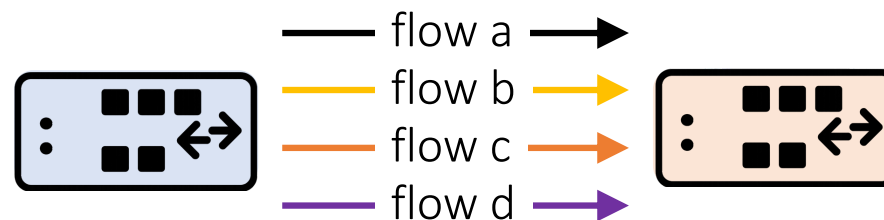
The idea behind this: provision memory based on $\text{avg}(\#\text{flows})$, not $\text{max}(\#\text{flows})$

- Single Decode for normal case
- Network-wide decoding for bursts of flows

How to find which switch processed which flow?

This switch processed
flow a

FlowXor	a	$a \oplus c \oplus d$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	1	3	3	3	2
PktCount					



How can we know that
this switch processed
flow a?

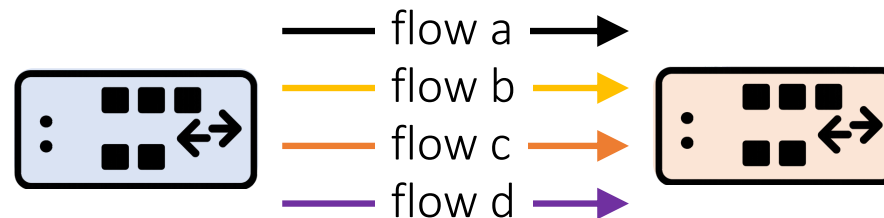
FlowXor	$a \oplus d$	$a \oplus c$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	2	2	3	3	2
PktCount					

How to find which switch processed which flow?

This switch processed flow a

FlowXor	a	$a \oplus c \oplus d$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	1	3	3	3	2
PktCount					

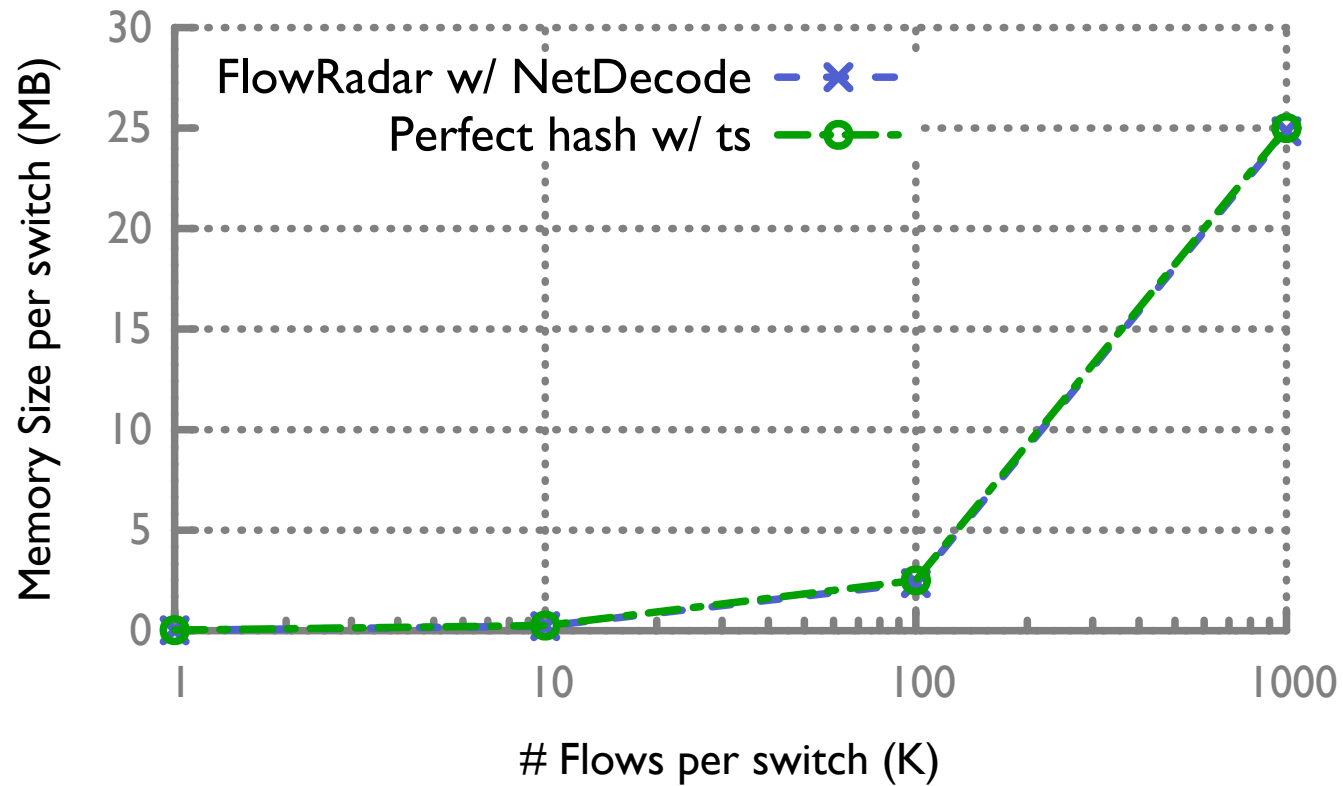
We need to query the Flow Filter of each switch first!



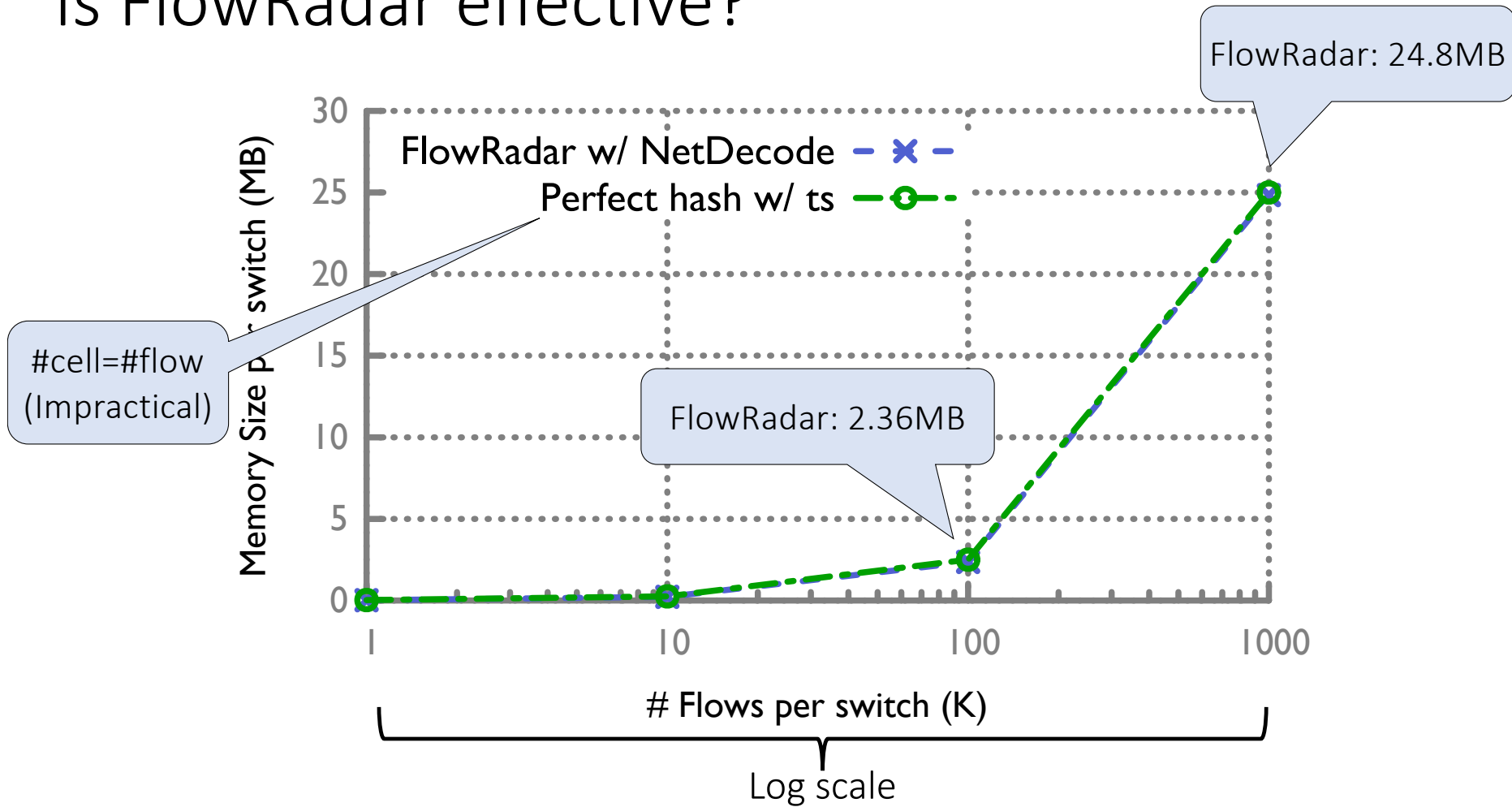
How can we know that this switch processed flow a?

FlowXor	$a \oplus d$	$a \oplus c$	$b \oplus c \oplus d$	$a \oplus b \oplus c$	$b \oplus d$
FlowCount	2	2	3	3	2
PktCount					

Is FlowRadar effective?



Is FlowRadar effective?



Question

- What if there is inconsistencies between switches in the decode phase because of packet losses?

Question

- What if there is inconsistencies between switches in the decode phase because of packet losses?

We can get anyway the full list of flows. We can then solve a linear equation system for each switch and this will help to detect losses!

Takeaways

- FlowRadar is a solution that leverages programmable switches to report counters for all flows in fine-grained time granularity
- It also fully leverages the capability of both the switches and the collector
 - Switch: fixed per-packet processing time, memory-efficient
 - Collector: Network-wide decoding

Takeaways

- FlowRadar is a solution that leverages programmable switches to report counters for all flows in fine-grained time granularity
- It also fully leverages the capability of both the switches and the collector
 - Switch: fixed per-packet processing time, memory-efficient
 - Collector: Network-wide decoding
- What if we are interested in metrics that are more than just the tuple <flow,counter> (i.e., switch queue length)?

In-Band Network Telemetry

- It is a framework designed to allow the collection and report of network state, by the data plane
- In the INT architectural model, packets may contain header fields that are interpreted as *telemetry instructions* by network devices
- These instructions tell an INT-capable device what state to collect
- Does this remind you something we saw earlier? 😊

In-Band Network Telemetry

- Original idea presented in ACM SIGCOMM 2014, named “tiny packet program” (TPP)

Millions of Little Minions: Using Packets for Low Latency Network Programming and Visibility

Vimalkumar Jeyakumar¹, Mohammad Alizadeh², Yilong Geng¹, Changhoon Kim³,
David Mazières¹

¹Stanford University, ²Cisco Systems, ³Barefoot Networks
{jvimal@cs., alizade@, gengyl08@}stanford.edu, chkim@barefootnetworks.com

In-Band Network Telemetry

- Original idea presented in ACM SIGCOMM 2014, named “tiny packet program” (TPP)
- Then, the killer-application for P4

Millions of Little Minions: Using Packets for Low Latency Network Programming and Visibility

Vimalkumar Jeyakumar¹, Mohammad Alizadeh², Yilong Geng¹, Changhoon Kim³,
David Mazières¹
¹Stanford University, ²Cisco Systems, ³Barefoot Networks
{jvimal@cs., alizade@, gengyl08@}stanford.edu, chkim@barefootnetworks.com

In-Band Network Telemetry - A Powerful Analytics Framework for your Data Center

Roberto Mari
Dir. Product Management
Barefoot Networks



In-Band Network Telemetry

- Original idea presented in ACM SIGCOMM 2014, named “tiny packet program” (TPP)
- Then, the killer-application for P4
- Then, a specification

In-band Network Telemetry (INT) Dataplane Specification

Version 2.1

The P4.org Applications Working Group. Contributions from
Alibaba, Arista, CableLabs, Cisco Systems, Dell, Intel, Marvell, Netronome, VMware

2020-11-11

Millions of Little Minions: Using Packets for Low Latency Network Programming and Visibility

Vimalkumar Jeyakumar¹, Mohammad Alizadeh², Yilong Geng¹, Changhoon Kim³,
David Mazières¹
¹Stanford University, ²Cisco Systems, ³Barefoot Networks
{jvimal@cs.,alizade@,gengyl08@}stanford.edu, chkim@barefootnetworks.com

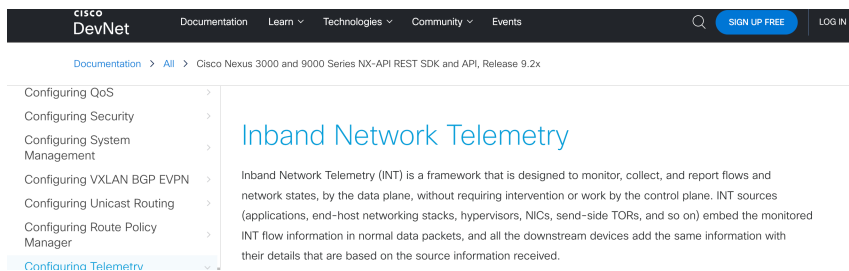
In-Band Network Telemetry - A Powerful Analytics Framework for your Data Center

Roberto Mari
Dir. Product Management
Barefoot Networks



In-Band Network Telemetry

Then....



The screenshot shows the Cisco DevNet website. The top navigation bar includes links for Documentation, Learn, Technologies, Community, and Events. A search bar and 'SIGN UP FREE' / 'LOG IN' buttons are also present. The breadcrumb trail reads: Documentation > All > Cisco Nexus 3000 and 9000 Series NX-API REST SDK and API, Release 9.2x. The left sidebar lists various configuration topics, with 'Configuring Telemetry' selected. The main content area is titled 'Inband Network Telemetry' and contains the following text: 'Inband Network Telemetry (INT) is a framework that is designed to monitor, collect, and report flows and network states, by the data plane, without requiring intervention or work by the control plane. INT sources (applications, end-host networking stacks, hypervisors, NICs, send-side TORs, and so on) embed the monitored INT flow information in normal data packets, and all the downstream devices add the same information with their details that are based on the source information received.'



Product Brief

In-band Telemetry

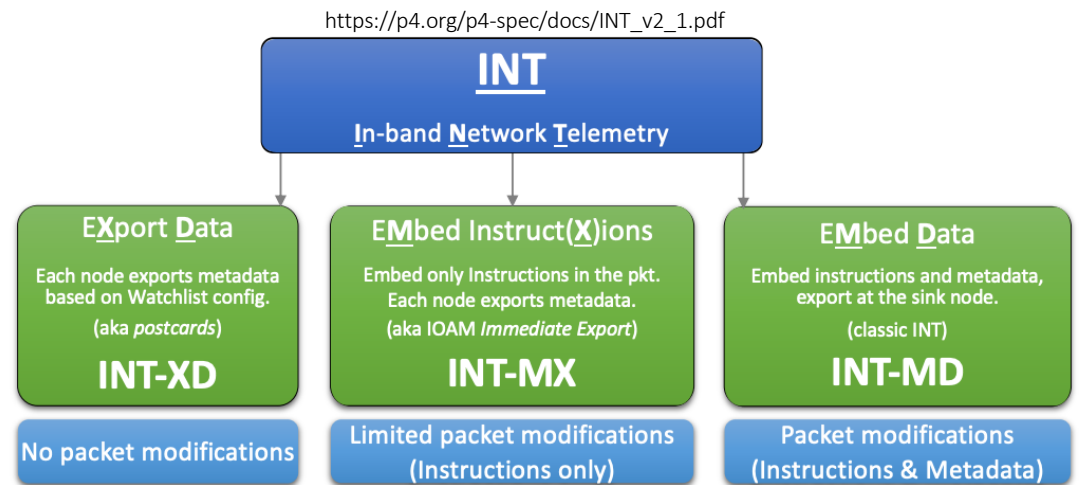
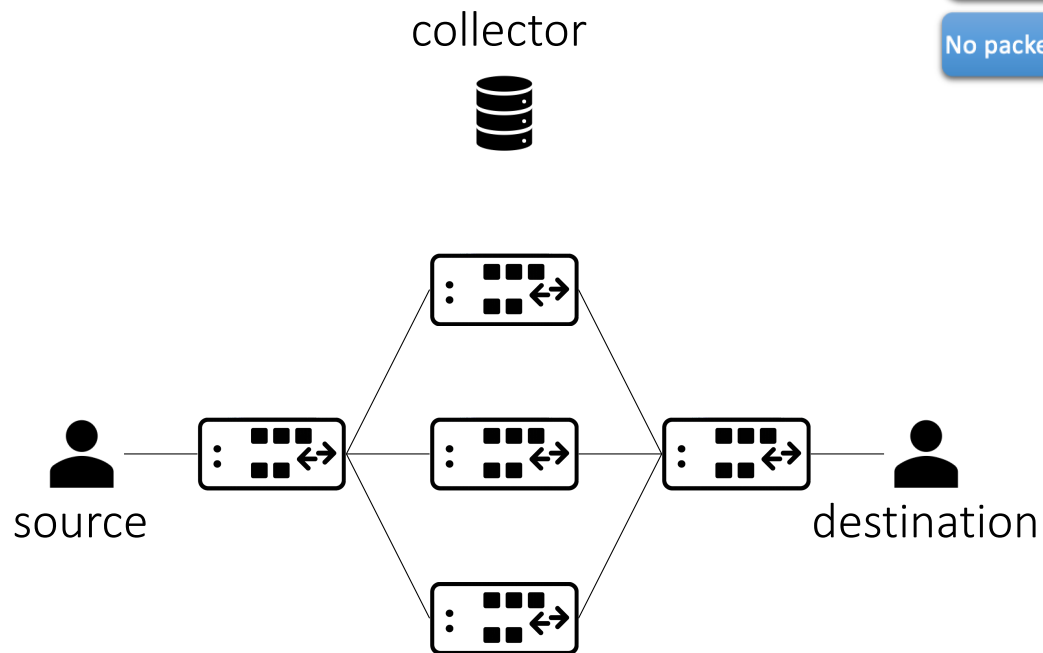
Introduction

Datacenter networks have become cloud scale. Deployment of hyper-converged networks is increasing. Telecom networks will enable connectivity everywhere with higher bandwidth delivering 5G wireless services. All of these next-generation networks not only require much higher bandwidth, but also require real-time telemetry to deliver services with good Quality of Experience (QoE).

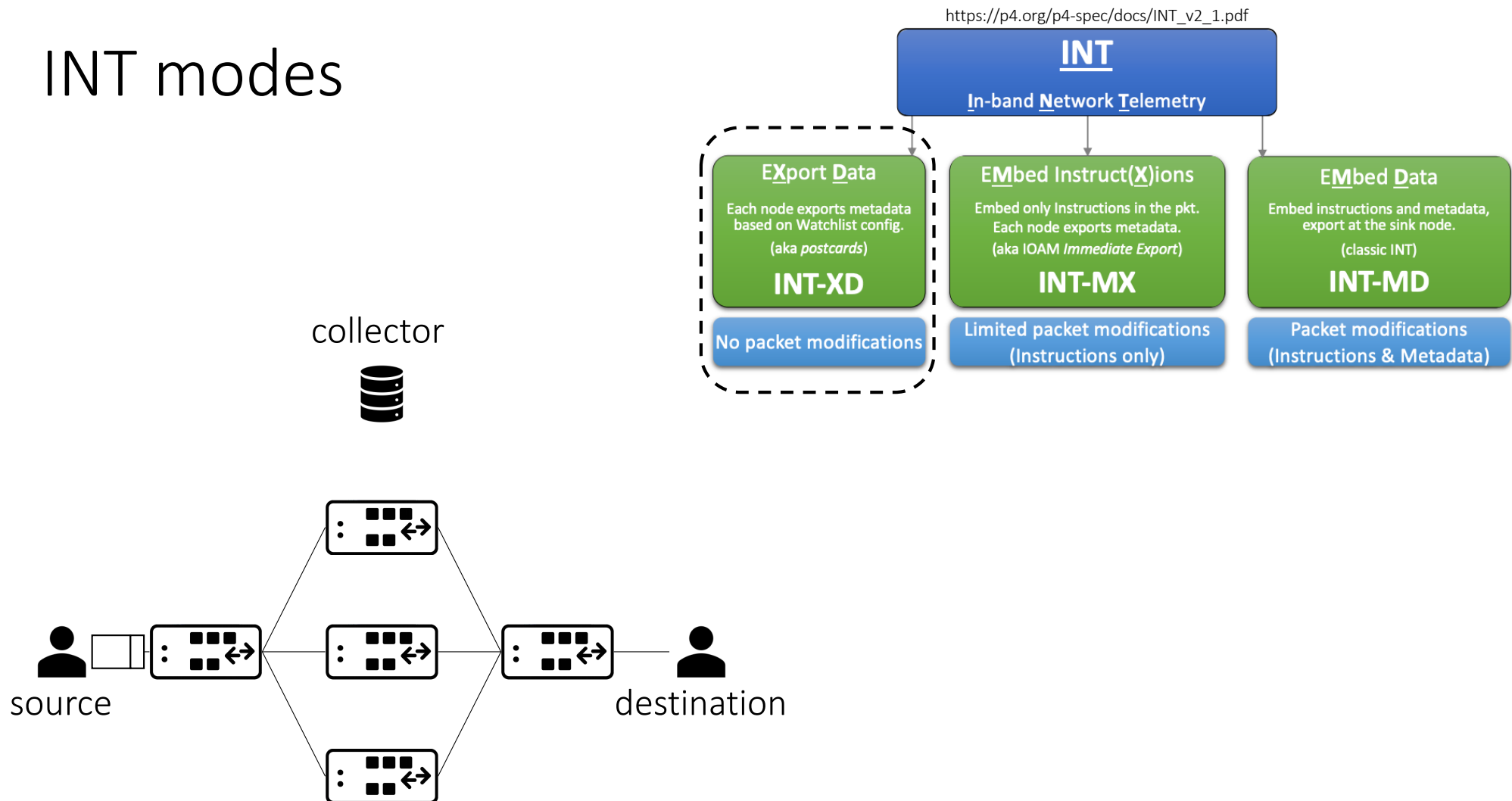
Key Use Cases

- Real-time congestion analysis

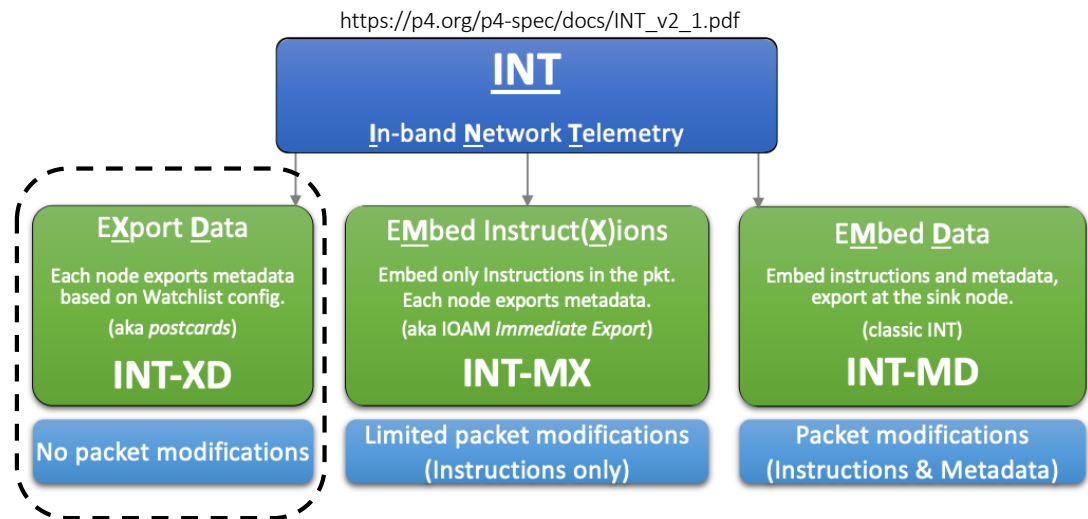
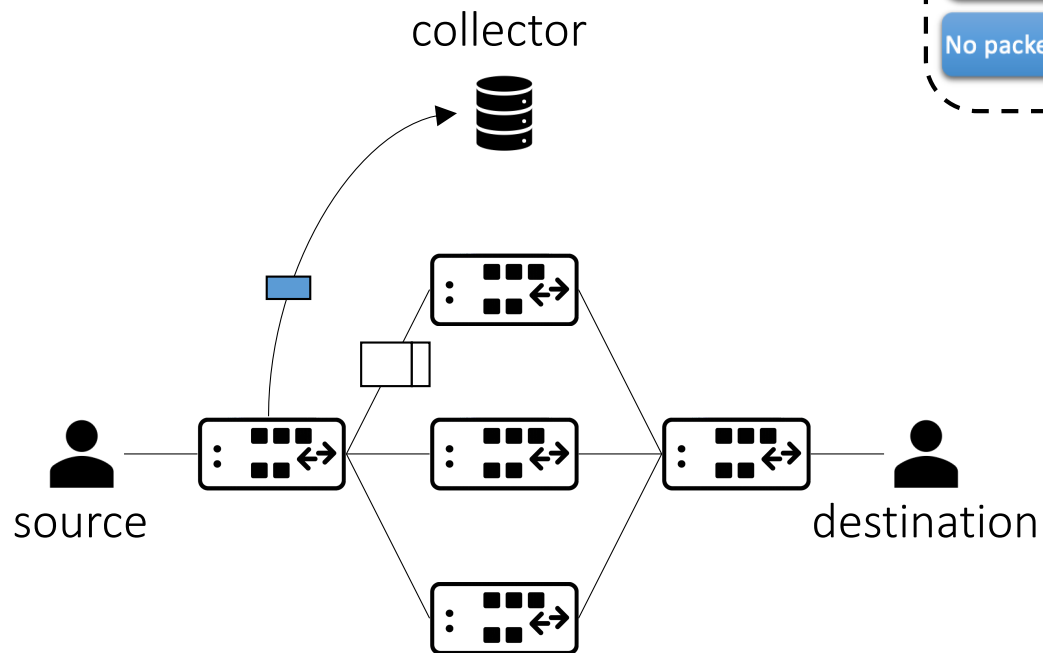
INT modes



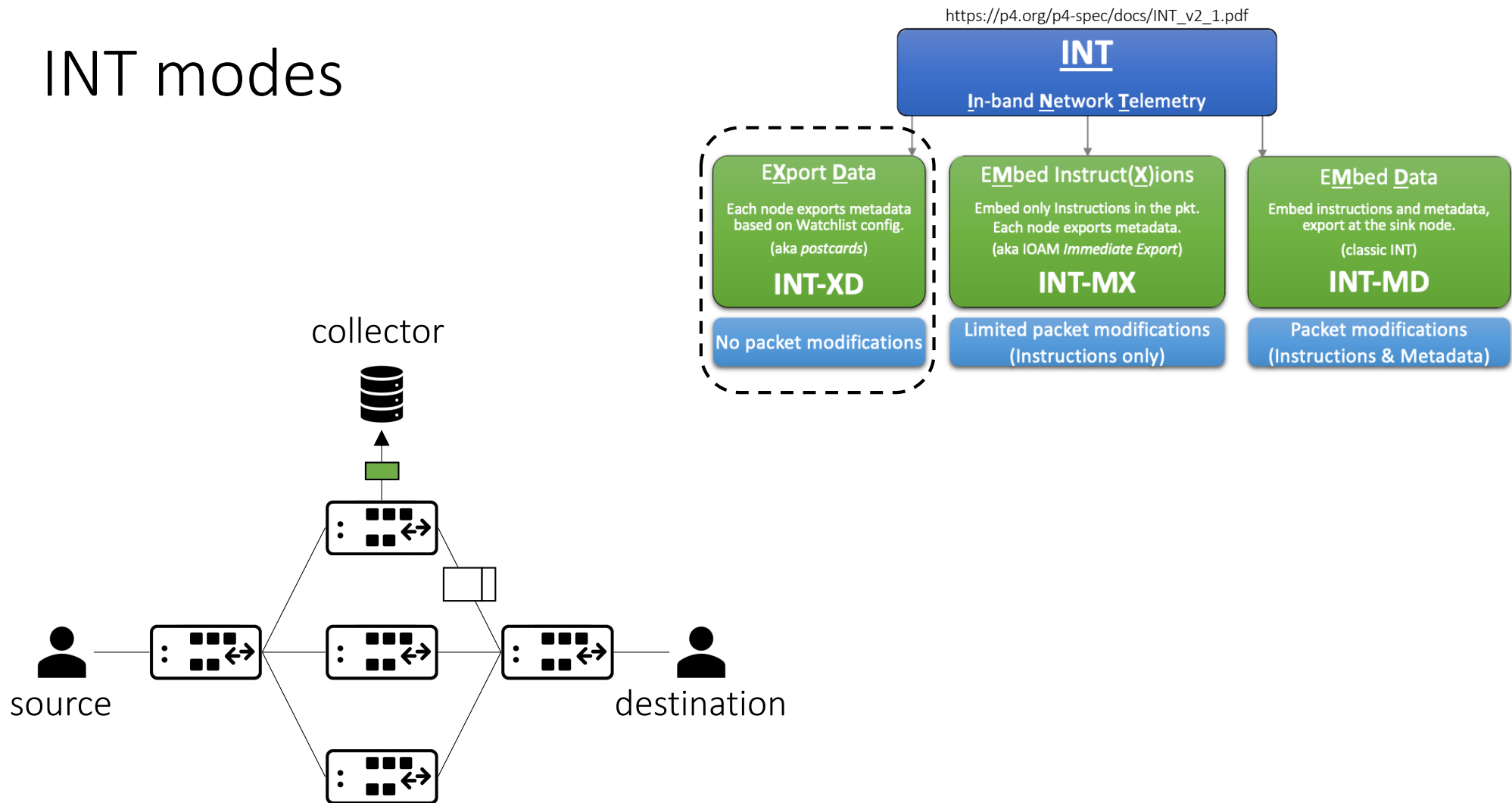
INT modes



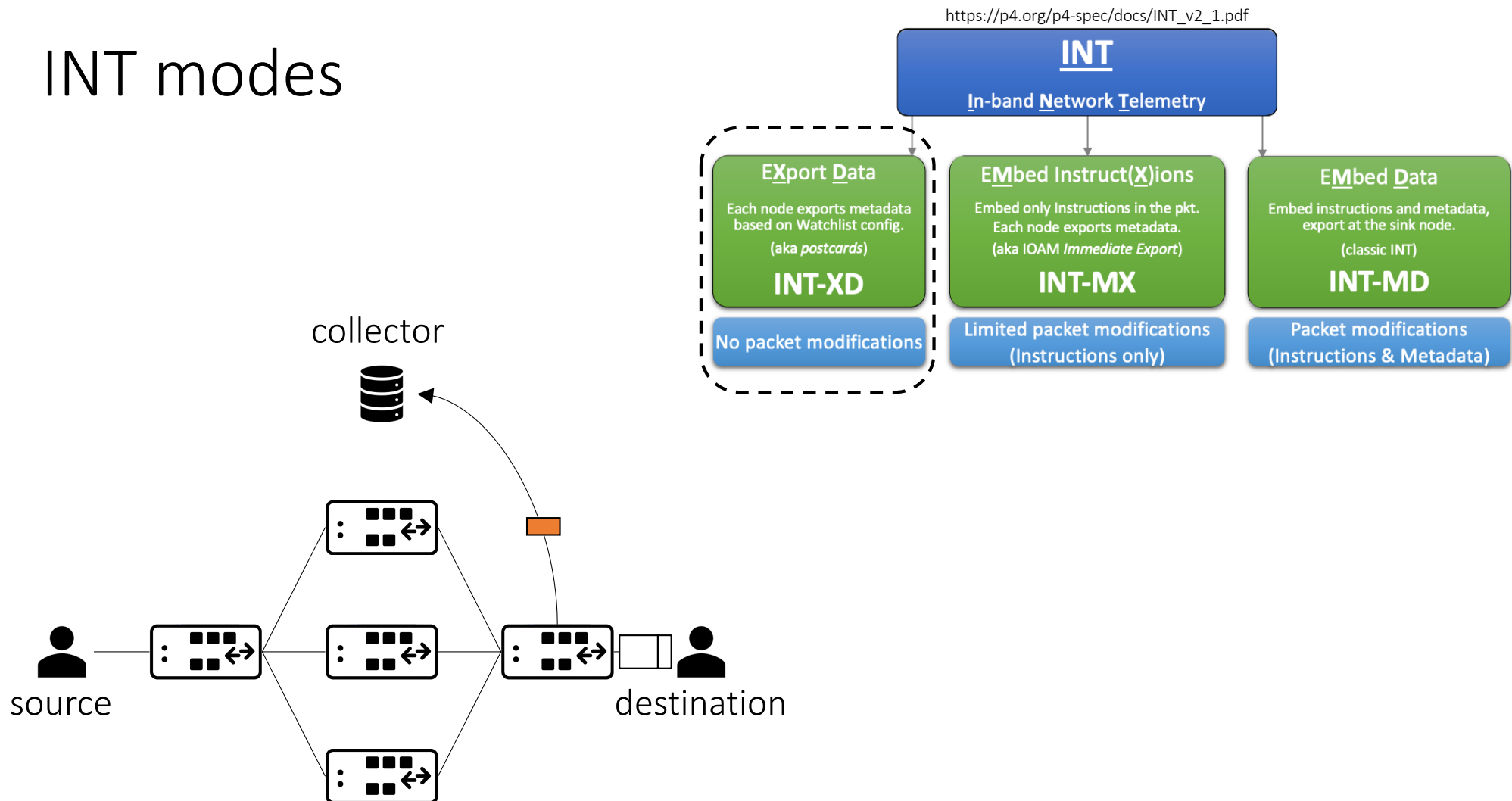
INT modes



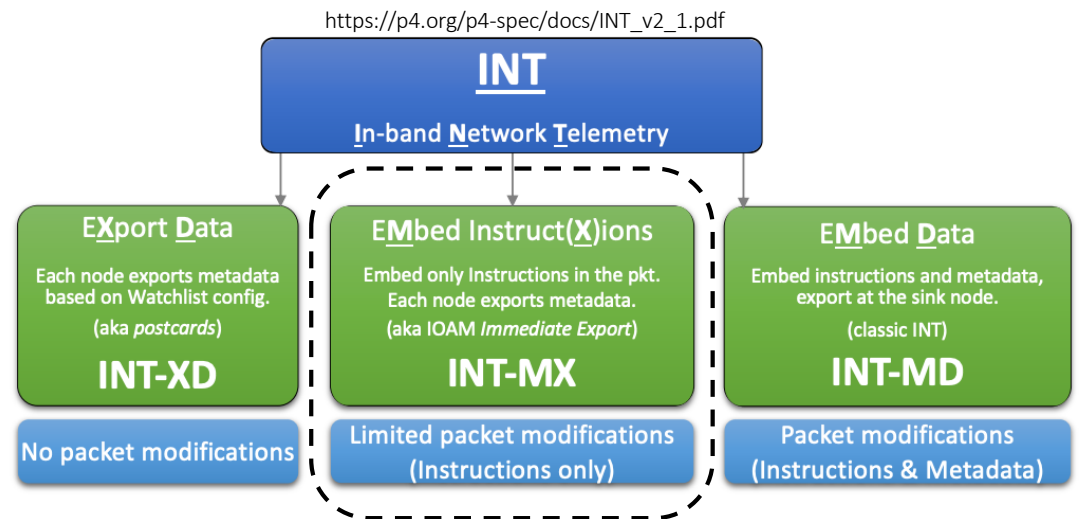
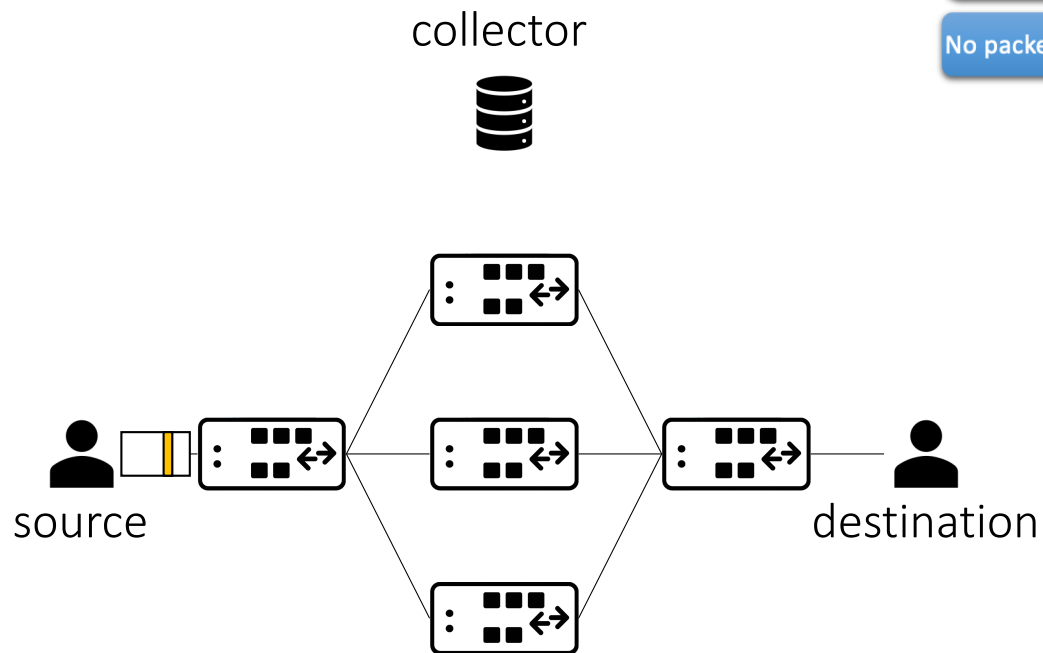
INT modes



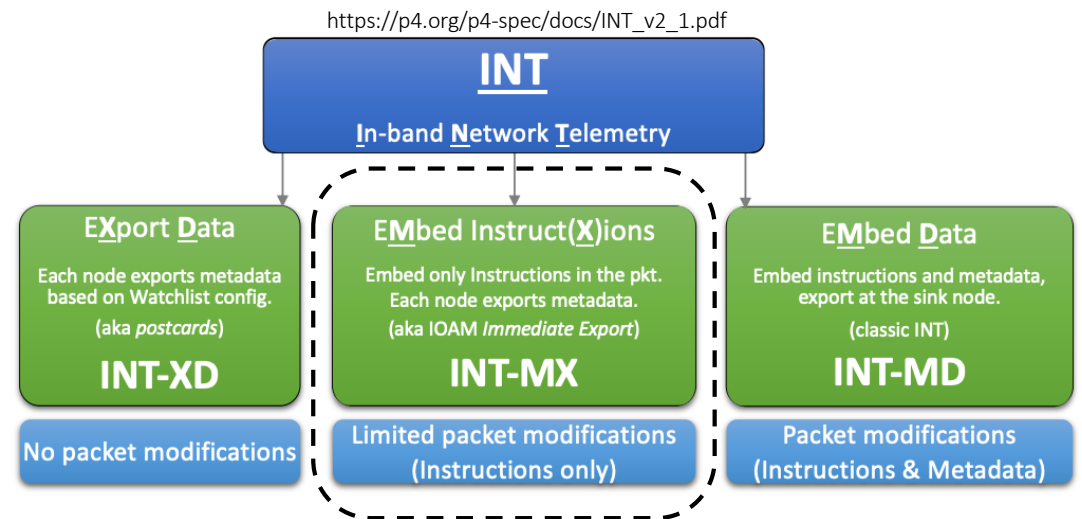
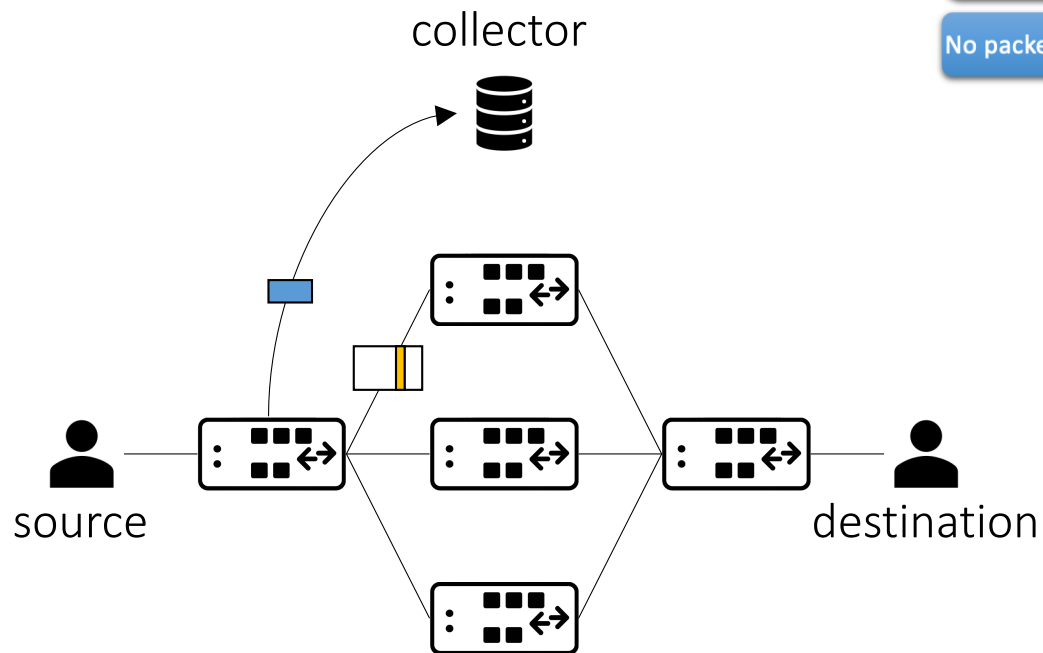
INT modes



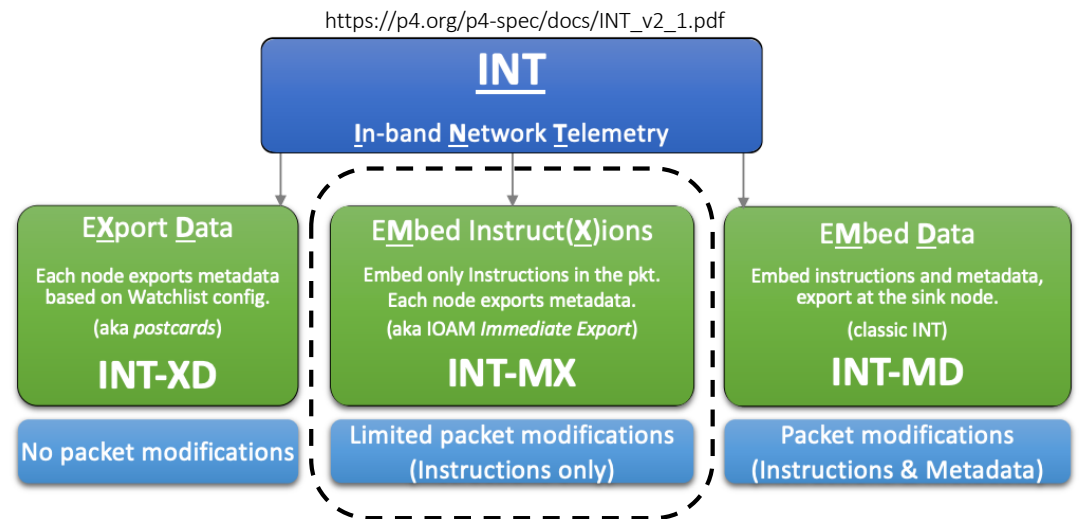
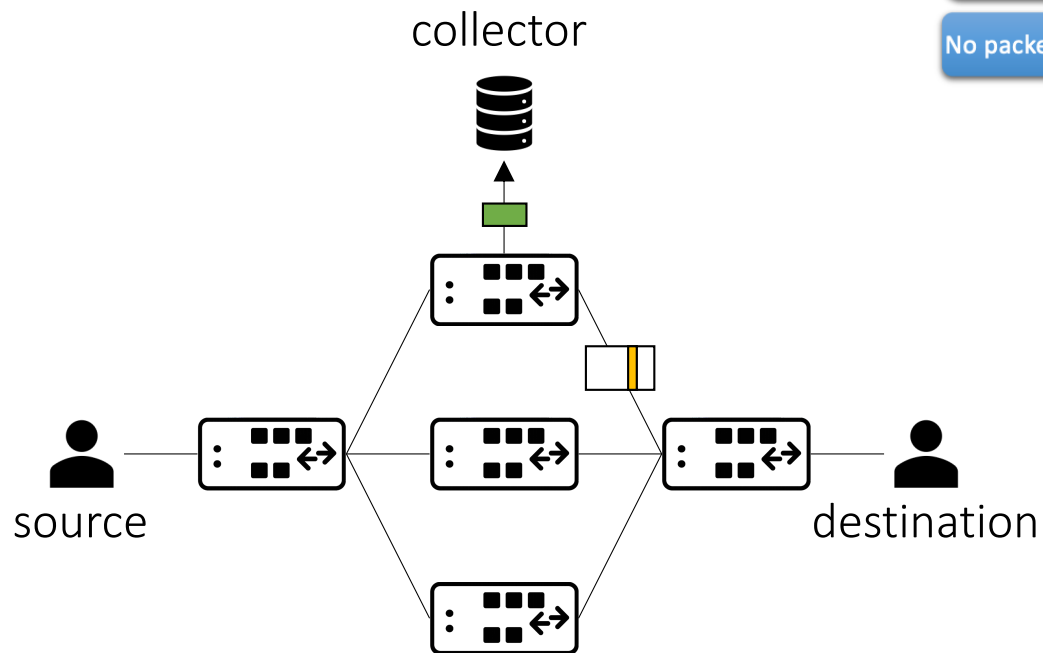
INT modes



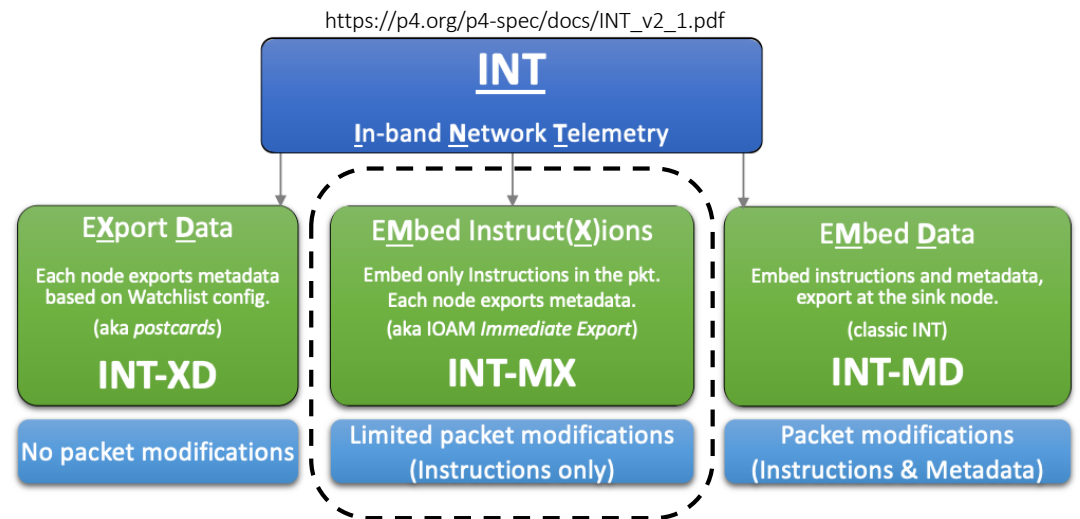
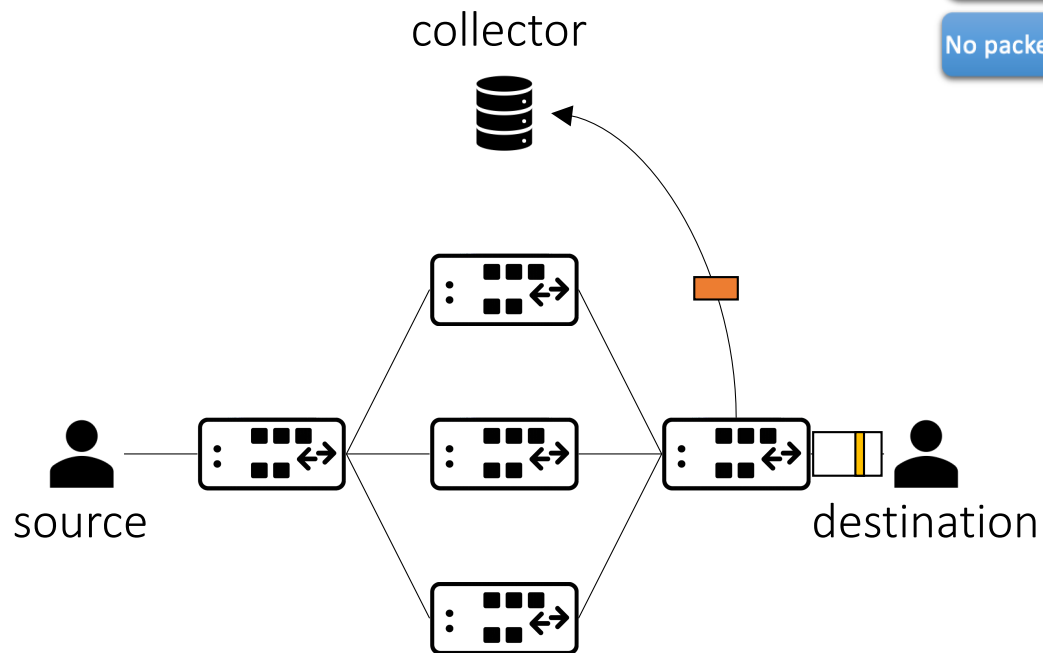
INT modes



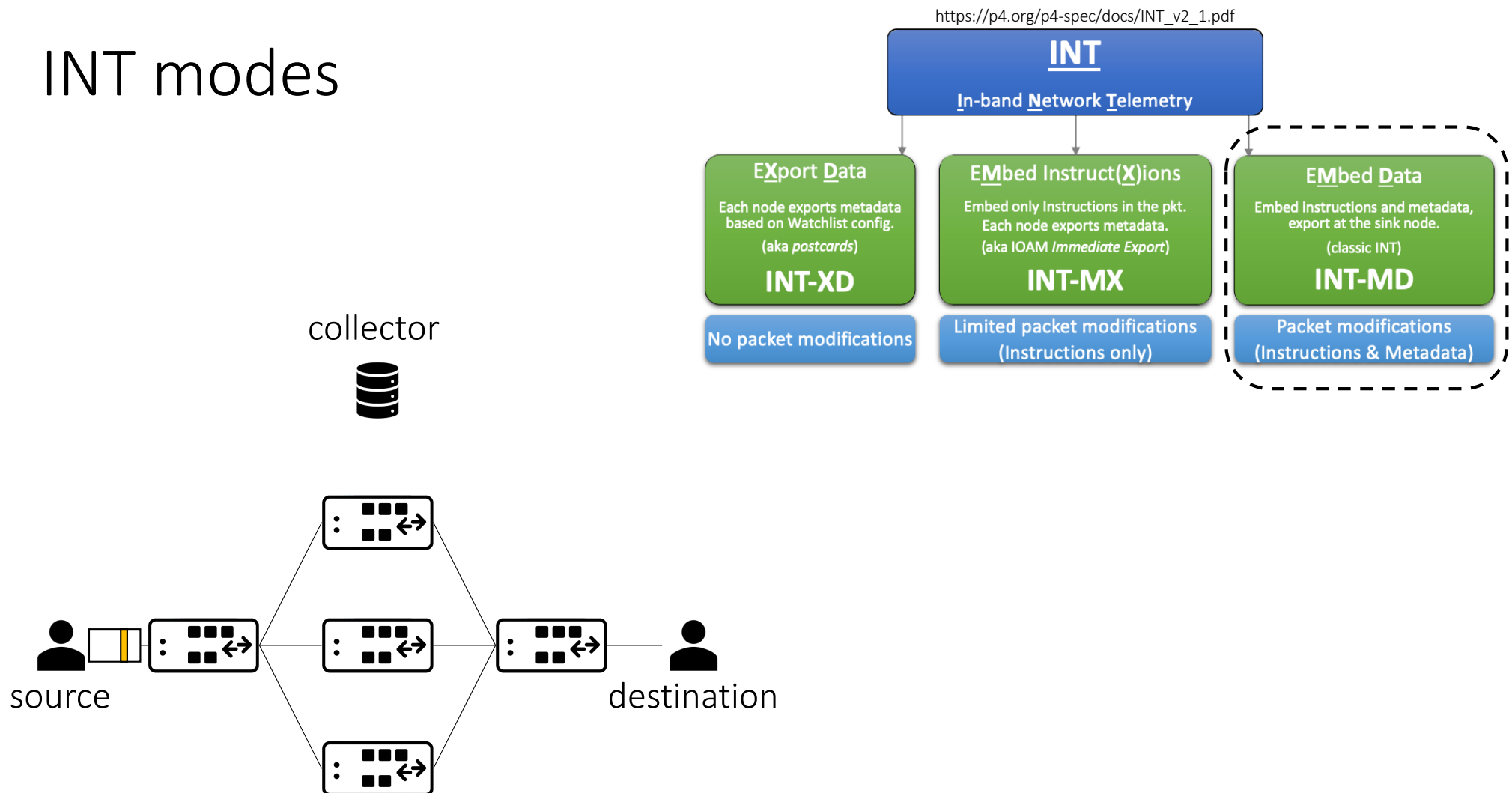
INT modes



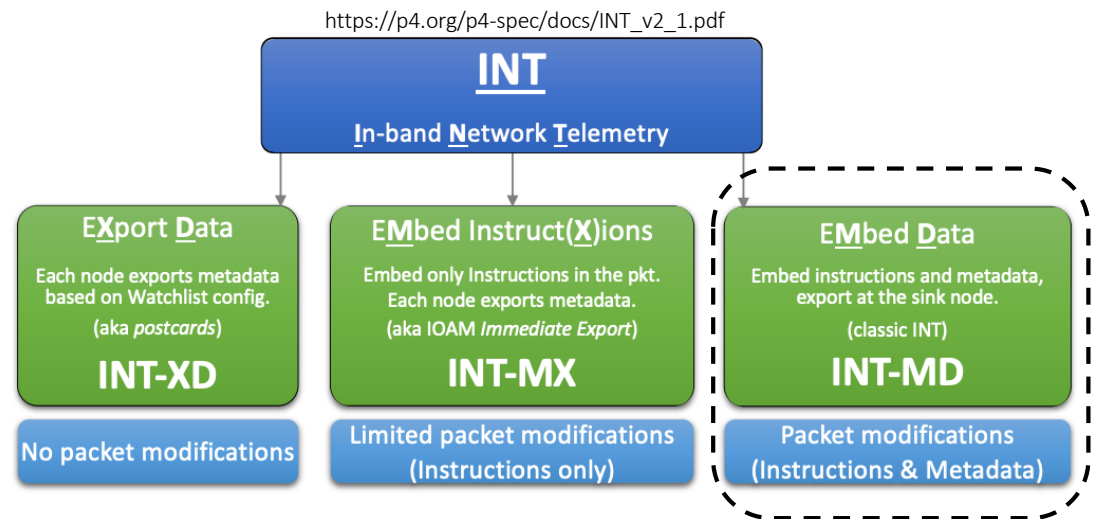
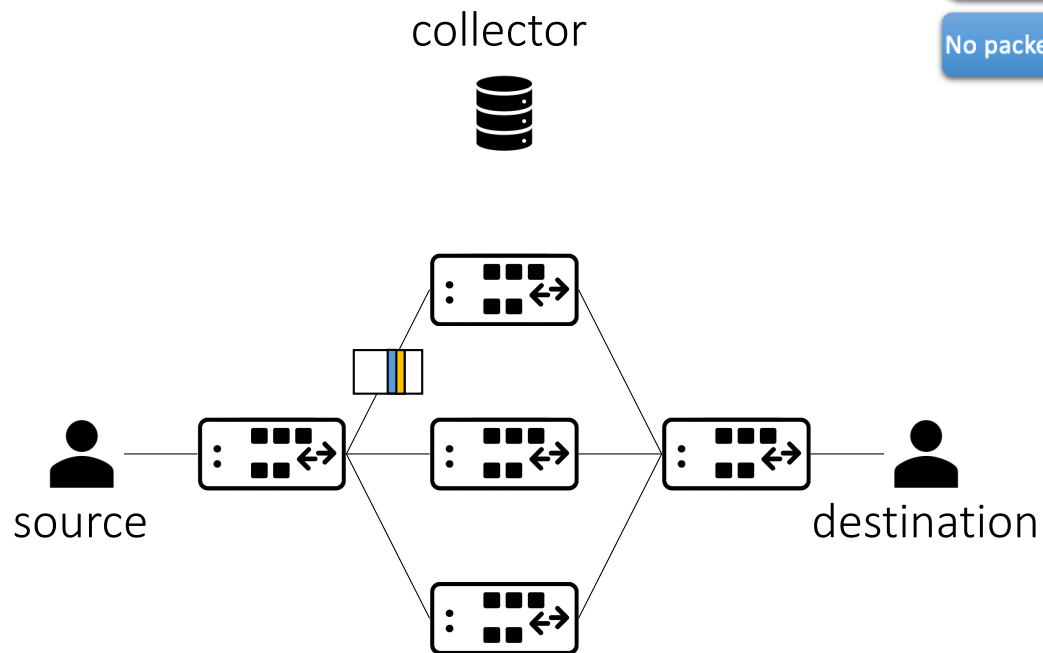
INT modes



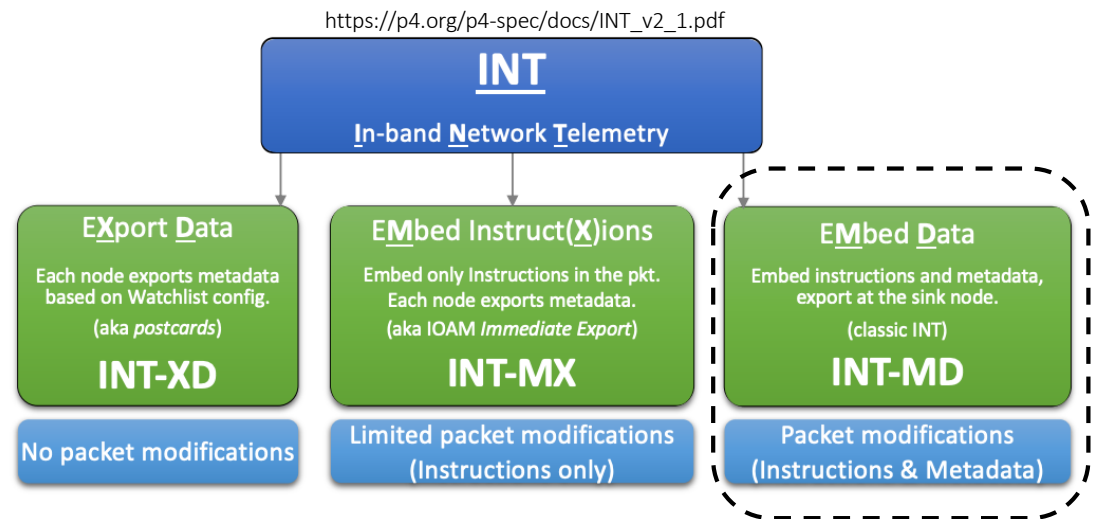
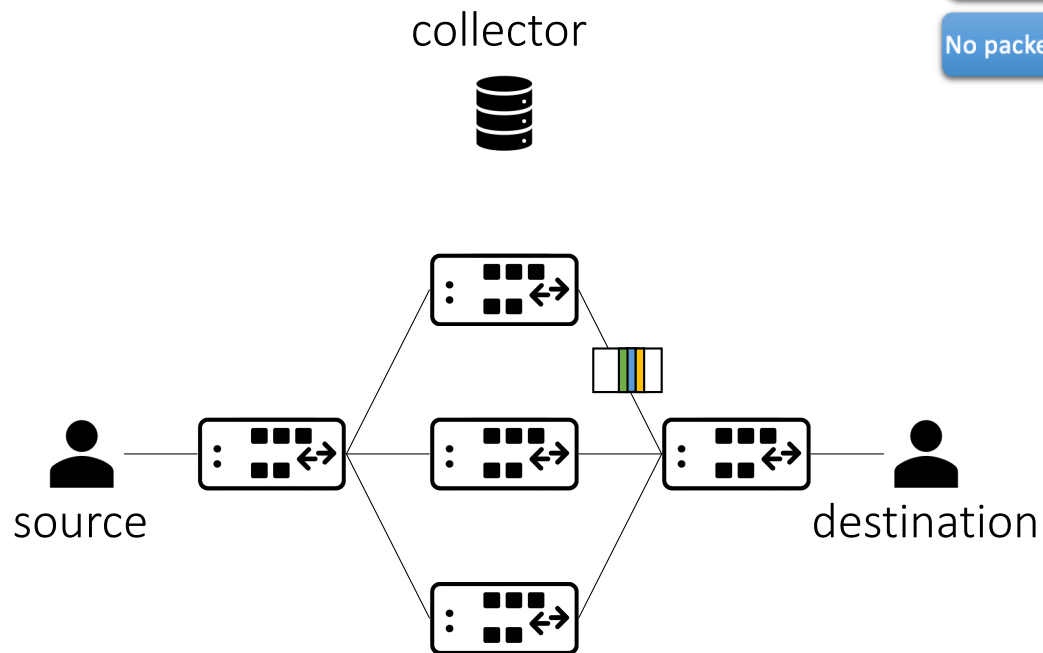
INT modes



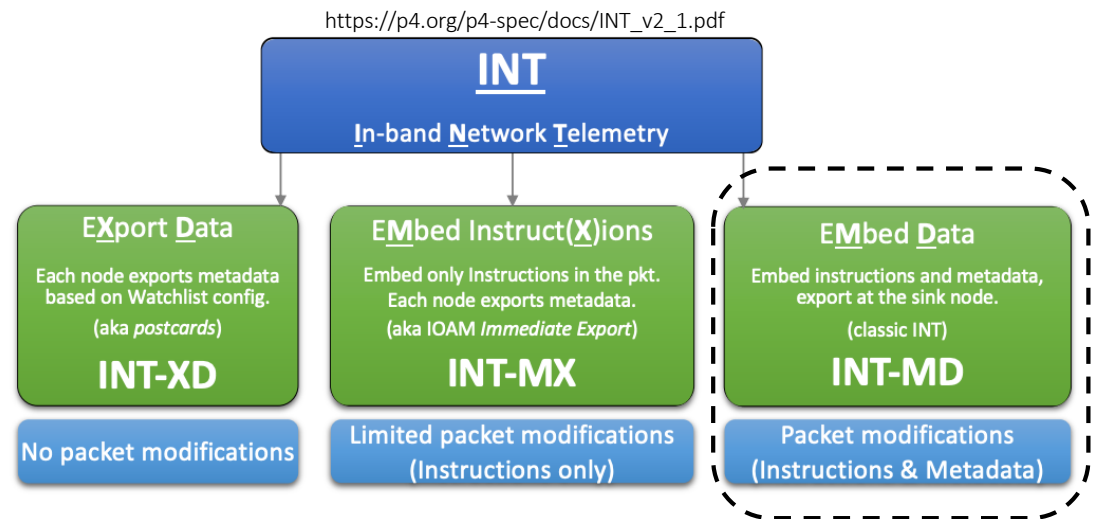
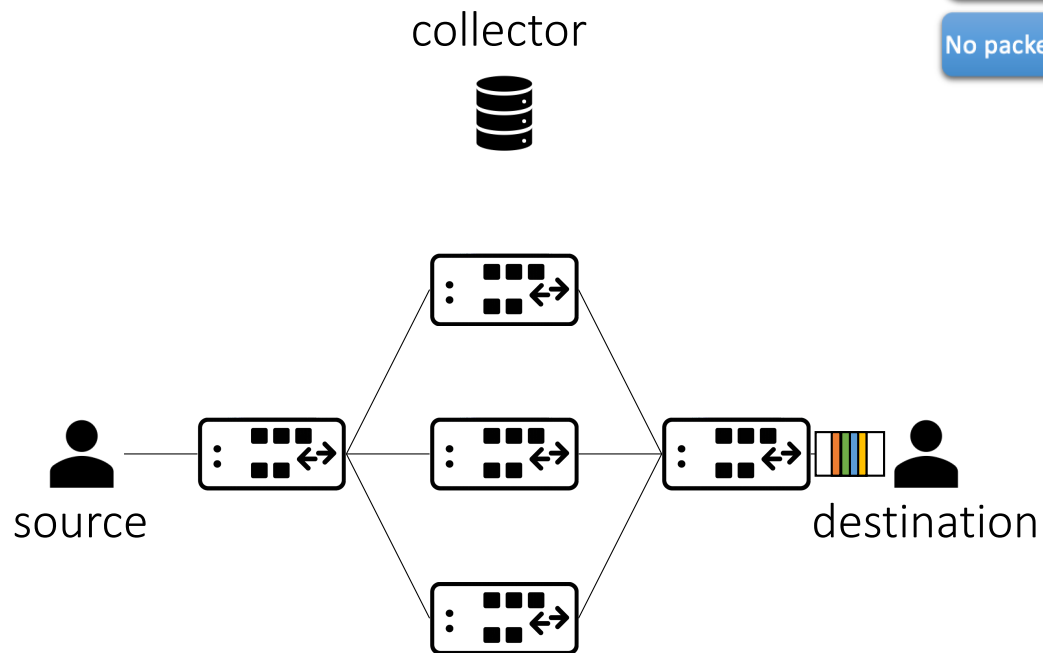
INT modes



INT modes



INT modes

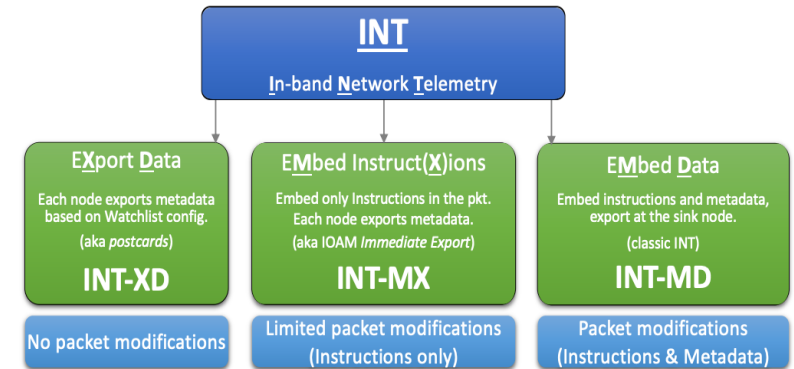


Which mode?

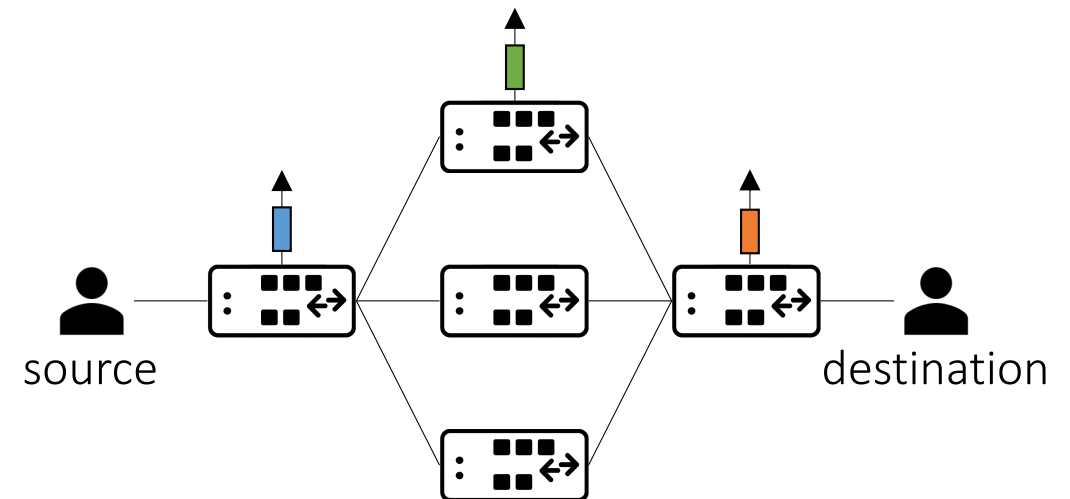
- INT-XD

Good

Bad



https://p4.org/p4-spec/docs/INT_v2_1.pdf

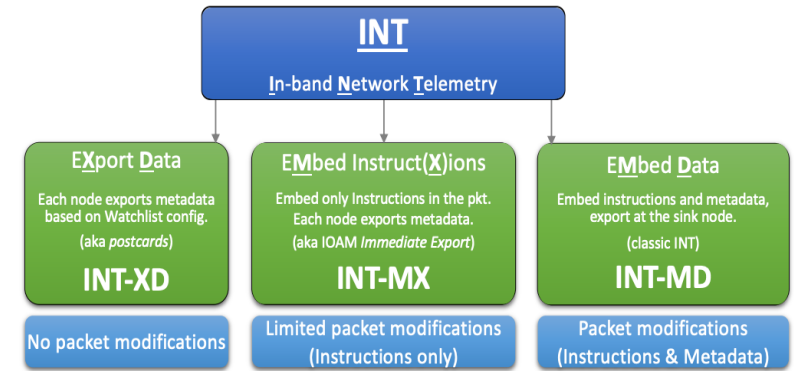


Which mode?

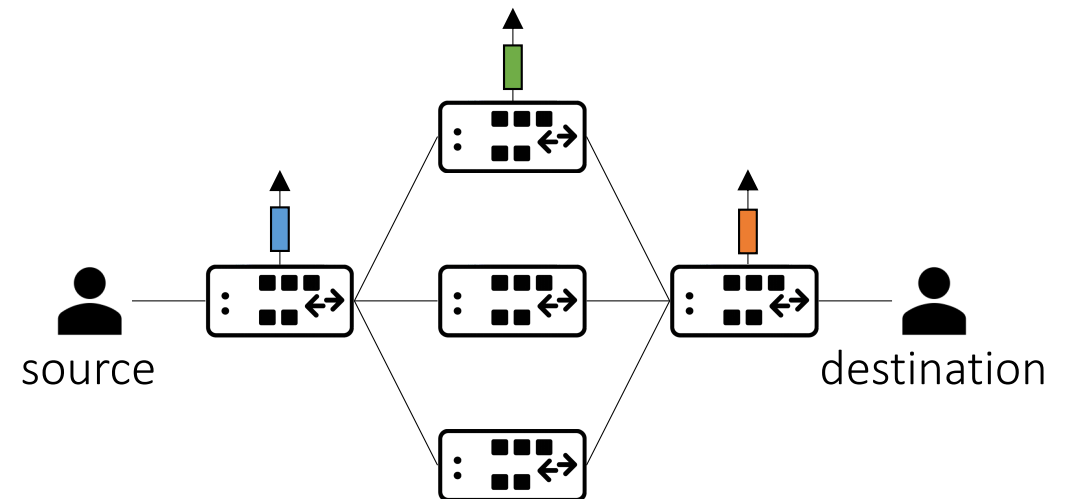
- INT-XD

Good no packet modifications (MTU)

Bad pressure on collectors, query based on switch configuration



https://p4.org/p4-spec/docs/INT_v2_1.pdf



Which mode?

- INT-XD

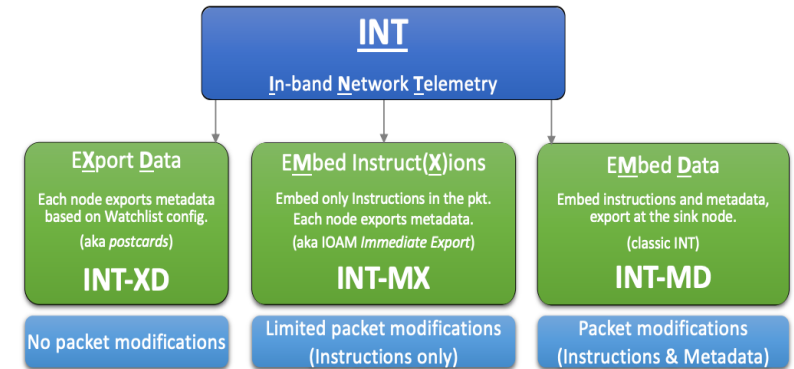
Good no packet modifications (MTU)

Bad pressure on collectors, query based on switch configuration

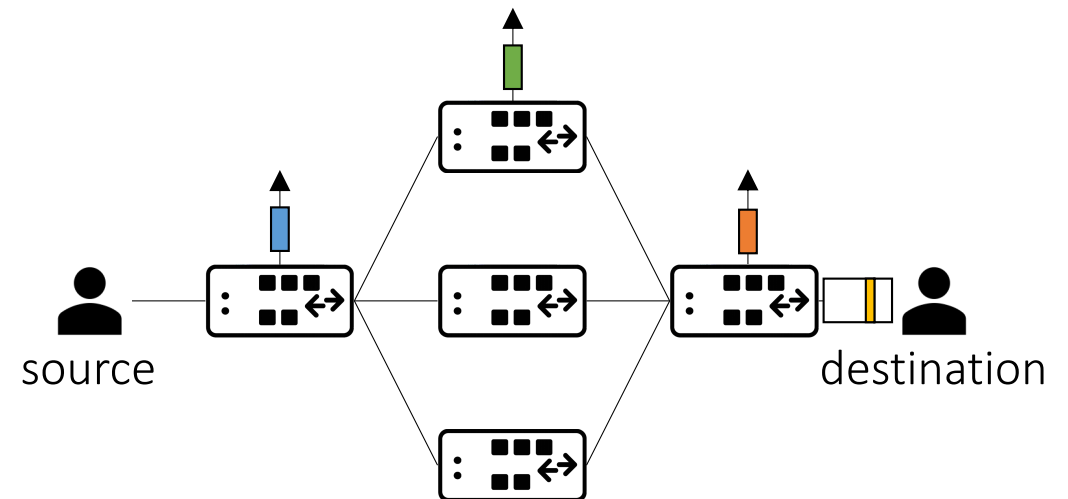
- INT-MK

Good

Bad



https://p4.org/p4-spec/docs/INT_v2_1.pdf



Which mode?

- INT-XD

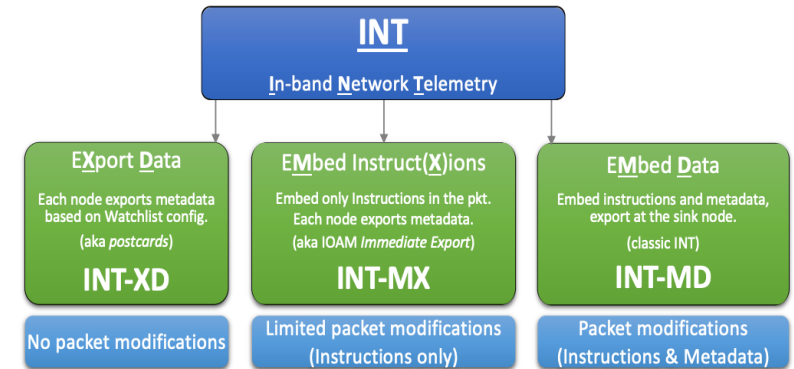
Good no packet modifications (MTU)

Bad pressure on collectors, query based on switch configuration

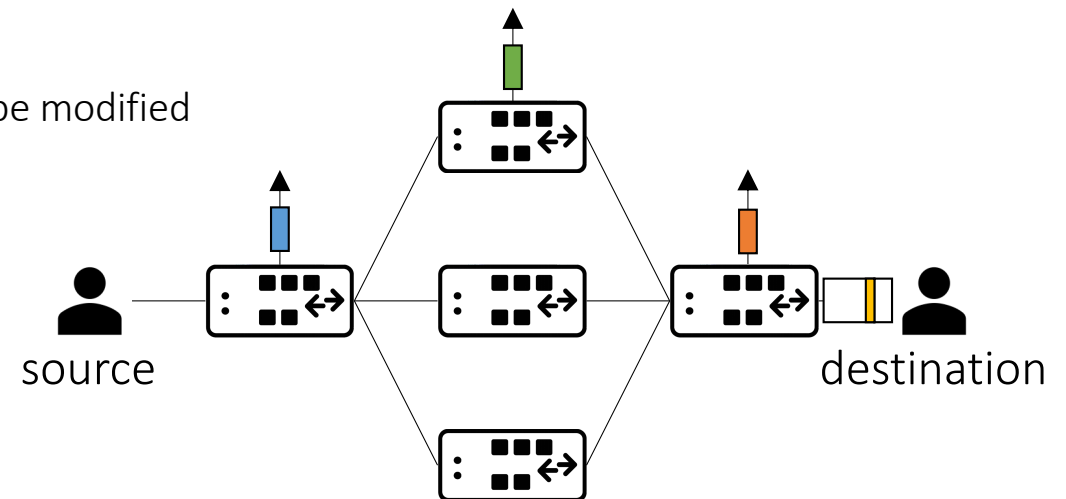
- INT-MK

Good query is packet-dependent

Bad pressure on collectors, packets need to be modified



https://p4.org/p4-spec/docs/INT_v2_1.pdf



Which mode?

- INT-XD

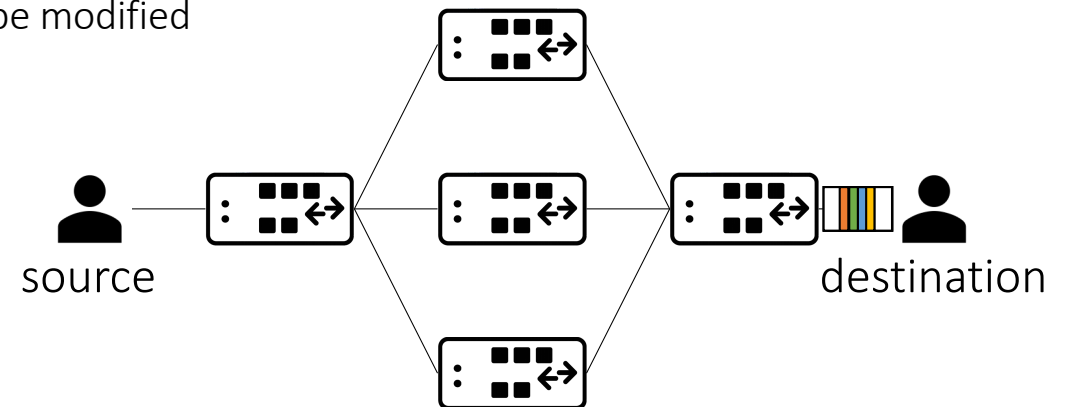
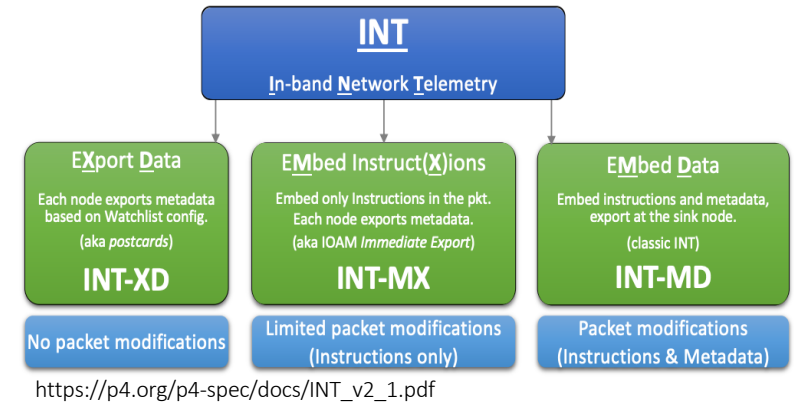
- Good no packet modifications (MTU)
 - Bad pressure on collectors, query based on switch configuration

- INT-MK

- Good query is packet-dependent
 - Bad pressure on collectors, packets need to be modified

- INT-MD

- Good
 - Bad



Which mode?

- INT-XD

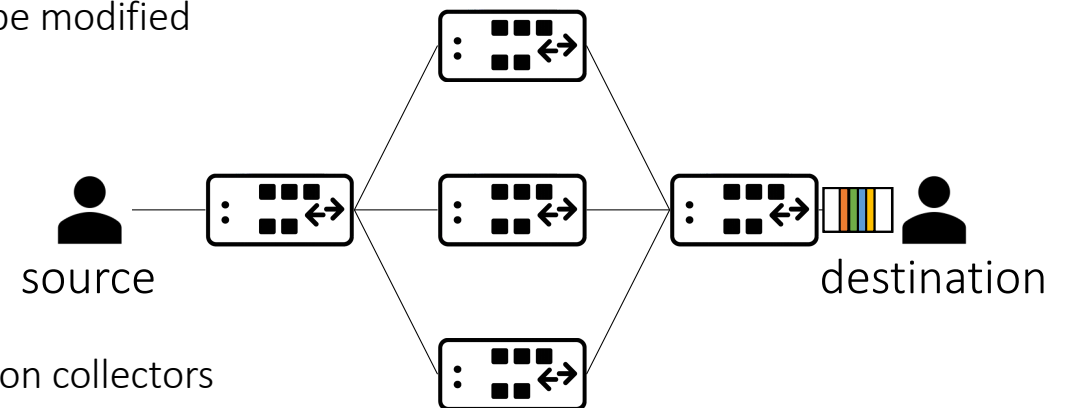
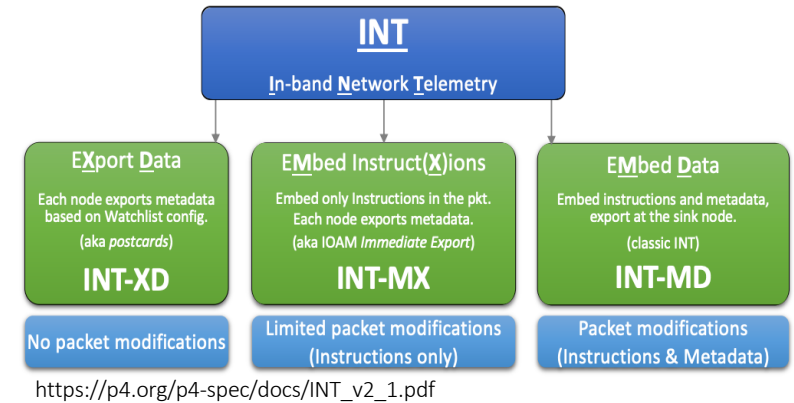
- Good no packet modifications (MTU)
 - Bad pressure on collectors, query based on switch configuration

- INT-MK

- Good query is packet-dependent
 - Bad pressure on collectors, packets need to be modified

- INT-MD

- Good query is packet-dependent, no pressure on collectors
 - Bad impact on packet MTU, packets need to be modified



What to monitor?

- The metadata values can be exported or appended to the packet depending on the INT operation mode
- With those you can answer questions such as:
 - Which path did my packet take?
 - How long did it queue at each switch?
 - Who did it share the queues with?
 - Is the path taken by my packet congested?

Metadata value	Description
Switch ID	ID associated with the switch
Ingress Port ID	Packet input port
Ingress Timestamp	Time when packet is received
Egress Port ID	Packet output port
Hop Latency	Time spent within the device
Egress Port TX utilization	Current utilization of output port
Queue Occupancy	The observed queue build up
Queue Congestion Status	Percentage of queue being used

What to monitor?

```
/* INT: add switch id */
action int_set_header_0() {
    add_header(int_switch_id_header);
    modify_field(int_switch_id_header.switch_id,
                 global_config_metadata.switch_id);
}

/* INT: add ingress timestamp */
action int_set_header_1() {
    add_header(int_ingress_tstamp_header);
    modify_field(int_ingress_tstamp_header.ingress_tstamp,
                 i2e_metadata.ingress_tstamp);
}

/* INT: add egress timestamp */
action int_set_header_2() {
    add_header(int_egress_tstamp_header);
    modify_field(int_egress_tstamp_header.egress_tstamp,
                 eg_intr_md_from_parser_aux.egress_global_tstamp);
}
```

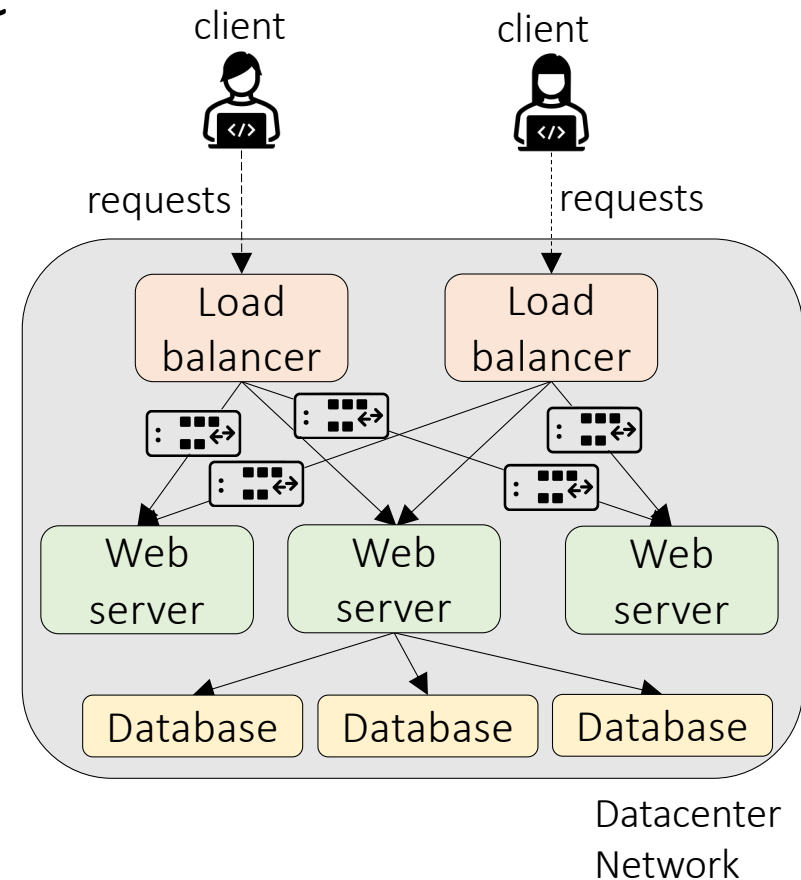
P4 code snippet: Insert switch ID, ingress and egress timestamp

Metadata value	Description
Switch ID	ID associated with the switch
Ingress Port ID	Packet input port
Ingress Timestamp	Time when packet is received
Egress Port ID	Packet output port
Hop Latency	Time spent within the device
Egress Port TX utilization	Current utilization of output port
Queue Occupancy	The observed queue build up
Queue Congestion Status	Percentage of queue being used

Back to the Everflow example

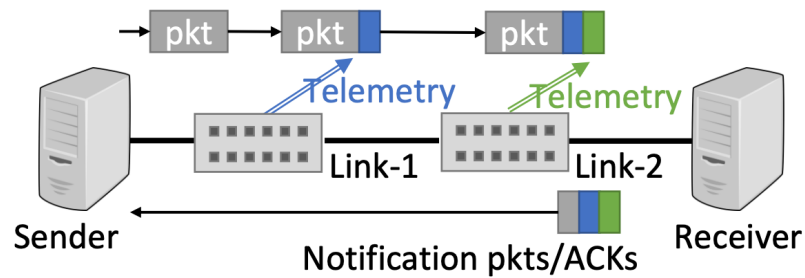
- Many clients of an internal web service reported that a fraction of their connection requests were encountering timeouts
- You could run INT-XD or INT-MX from load balancers to the web servers and detect drops

Metadata value	Description
Switch ID	ID associated with the switch
Ingress Port ID	Packet input port
Ingress Timestamp	Time when packet is received
Egress Port ID	Packet output port
Hop Latency	Time spent within the device
Egress Port TX utilization	Current utilization of output port
Queue Occupancy	The observed queue build up
Queue Congestion Status	Percentage of queue being used



INT as useful building block


Adjust rate
per ACK



INT as a precise feedback to decide how to adjust sender rate

HPCC: High Precision Congestion Control

Yuliang Li[✉], Rui Miao^{*}, Hongqiang Harry Liu^{*}, Yan Zhuang^{*}, Fei Feng^{*}, Lingbo Tang^{*}, Zheng Cao^{*}, Ming Zhang^{*},
Frank Kelly[✉], Mohammad Alizadeh^{*}, Minlan Yu[✉]
Alibaba Group^{}, Harvard University[✉], University of Cambridge[✉], Massachusetts Institute of Technology^{*}*



Intel

Oct 18 · 6 min read · [Listen](#)

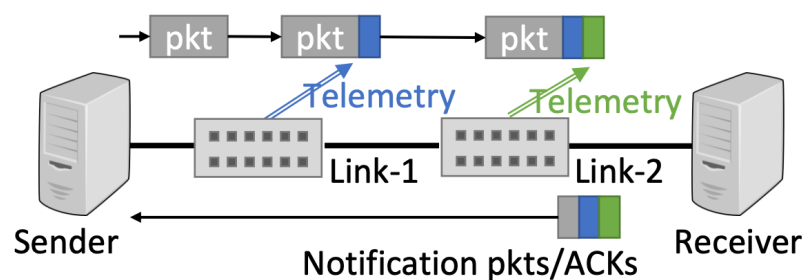


TECHNOLOGY

TCP-INT: Intel's Lightweight Network Telemetry Improves Visibility and Control for TCP Workloads

Simon Wass, Staff Engineer & Jeongkeun Lee, Senior Principal Engineer

INT as useful building block



INT as a precise feedback to decide how to adjust sender rate

+ many more use-cases (e.g., load balancing, security, accounting)

HPCC: High Precision Congestion Control

Yuliang Li[✉], Rui Miao^{*}, Hongqiang Harry Liu^{*}, Yan Zhuang^{*}, Fei Feng^{*}, Lingbo Tang^{*}, Zheng Cao^{*}, Ming Zhang^{*},
Frank Kelly[✉], Mohammad Alizadeh^{*}, Minlan Yu[✉]
Alibaba Group^{}, Harvard University[✉], University of Cambridge[✉], Massachusetts Institute of Technology^{*}*



Intel

Oct 18 · 6 min read · [Listen](#)



TECHNOLOGY

TCP-INT: Intel's Lightweight Network Telemetry Improves Visibility and Control for TCP Workloads

Simon Wass, Staff Engineer & Jeongkeun Lee, Senior Principal Engineer